tramdag documentation

Contents

tramdag — Interpretable Neural Causal Models (TRAM-DAGs) in PyTorch

> **Status: 1.0 release candidate.** This README documents the **unreleased 1.0** API (term classes, `callbacks=`, `flow.shift_curve`, `VC(center="col")`). `0.3.0` on PyPI predates all of it. Until 1.0 ships, only a git install matches the docs below.

TRAM-DAGs model each variable of a structural causal model with a (transformation-model) flow. The flow is one triangular normalizing flow from iid standard-logistic latents to the observed variables. The triangular adjacency structure is exactly your causal DAG. One fit on observational data answers all three rungs of Pearl's causal hierarchy:

- observational (L1),
- interventional (L2, the do-operator),
- counterfactual (L3, Pearl abduction).

The model keeps **interpretable effects**. Every linear-shift coefficient is a log-odds ratio, exactly as in classical proportional-odds models.

> Beate Sick & Oliver Dürr, *Interpretable Neural Causal Models with TRAM-DAGs*, CLeaR 2025 (arXiv:2503.16206). This repo is the reference implementation (PyTorch, built on zuko). Pinned tests replicate all of the paper's experiments here.

Start here: the Colab badge above opens a notebook. The notebook does three things:

- it fits the paper's bimodal benchmark,
- it walks L1 to L3,

- it writes the same DAG with interpretable terms.

Every answer is a computed check against analytic ground truth. A failed claim therefore stops the notebook. The notebook runs on CPU in a few minutes.

`docs/model.md` describes the model itself. `docs/interpretation.md` describes how to read a fitted model. `notebooks/README.md` lists the other notebooks.

Install

```
uv add tramdag                 # latest release (PyPI); pip install tramdag works too
uv add "tramdag[plots]"        # with matplotlib, for plot_dag / plot_marginals / plot_training
uv add "tramdag @ git+https://github.com/tensorchiefs/tramdag.git@main"   # dev version (track ma
uv sync                        # or: dev setup from a clone (tests, experiments)
```

Pin the dev install to a commit for reproducibility. Use the form `...tramdag.git@<sha>`.

30 seconds of API

```python
from tramdag import CausalFlowDAG, ContinuousNode, OrdinalNode, I, LS, CS

spec = {  # the spec IS the labelled DAG
    "X1": ContinuousNode(),
    "X2": ContinuousNode(I("X1")),
    "T": OrdinalNode(2, LS("X1") + CS("X2")),  # 2 levels, column coded 0..1
    "Y": OrdinalNode(4, I("X1") + CS("X2") + LS("T")),
}
# train_df / val_df: DataFrames with one column per node
flow = CausalFlowDAG(spec)  # validates acyclicity, builds the flow

# fit() is one minibatch Adam loop with Keras-shaped extras: validation_data=
# (or validation_split=0.1) records the per-epoch validation NLL in
# flow.history["val"], verbose= prints progress; schedules and early stopping
# attach through optimizer= and callbacks= (docs/fitting.md)
from tramdag.callbacks import EarlyStopping

flow.fit(
    train_df,
    epochs=4000,
    batch_size=512,
    validation_data=val_df,
    verbose=100,
    # keep the best-validation weights; stop once the best is 200 epochs old
    callbacks=[EarlyStopping(patience=200)],
)

# all-`ls` model? fit it classically instead: deterministic float64 L-BFGS,
# exact MLE matching statsmodels/R (see docs/fitting.md)
flow.fit_classical(train_df)  # raises on cs/ci/vc specs

flow.log_prob(df)  # L1: joint log-likelihood per row
flow.sample(1000)  # L1: observational sampling
flow.sample(1000, do={"T": 1})  # L2: interventional (graph mutilation)
flow.pmf(df, node="Y", do={"T": 1})  # L2: analytic interventional PMF
flow.density(df, node="X2", grid=grid, do={"X1": 0.5})  # ... and density, continuous nodes

u = flow.abduct(df)  # L3 step 1: latents from observations
```

```python
cf = flow.sample(do={"T": 1}, u=u)  # L3 steps 2+3: counterfactuals

flow.ls_coefficients()  # interpret: per-edge log-odds-ratios (LS terms)
# per-parent partial effects of an additive complex intercept (centered):
# flow.intercept_contributions(df, "Y") on a CI("A", "B", allow_interaction=False)

# heterogeneous treatment effects: a small, penalized effect head beta(x)*T
# (VC term) with a first-class read-out — see docs/varying-coefficients.md
# e.g. CS("X1", "X2") + VC("X1", t="T") ->
# flow.varying_coef(df, "Y")              # beta(x): deterministic, y-free

flow.scores(df, node="Y")  # per-observation scores dl_i/dtheta
flow.effect_modifier_scan(df, "Y", t="T")  # which VC modifiers? (CUSUM
# scan from a cheap all-ls fit) — docs/scores.md

flow.save("flow.pt")
flow = CausalFlowDAG.load("flow.pt")
```

See the DAG

```python
from tramdag import plot_dag
plot_dag(spec)   # or plot_dag(flow) — layers left to right, one edge style per term
```

Continuous nodes are ellipses. Ordinal nodes are rounded boxes with their level count. The edge styles are:

- `LS` is thin gray,
- `CS` is thick blue,
- a complex intercept is dashed orange,
- a `VC` treatment edge is red with its modifiers dotted.

`plot_dag` needs the `plots` extra.

The model in detail: spec → math → networks

Per node, the transformation is additive on the latent (log-odds) scale — one intercept term `I` plus any number of shifts (notation: `docs/notation.md`):

`u = h_ϑ(x) + Σ β·x_pa + Σ g(x_pa) + (β₀ + b_ϑ(x_mod))·x_t`

term	math	what gets built	interpretability
`I()` / bare `I` / omitted	`h_ϑ(x)` — constant ϑ	`SimpleIntercept`: one free parameter vector, no network	the baseline transform
`I("A")`	`h_ϑ(a)(x)` — ϑ bends with the parent	`ComplexIntercept`: NN `[8, 8]` → `n_params`	the parent reshapes the whole distribution; no single coefficient
`I("A","B")` (default `allow_interaction=True`)	`h_ϑ(a,b)(x)`	**one joint** NN over both parents — they interact in ϑ	maximal flexibility
`I("A","B", allow_interaction=False)`	`h_ϑ(a)+ϑ(b)(x)`	one NN **per parent**, parameter vectors summed in coefficient space	per-parent partial effects via `flow.intercept_contributions`

term	math	what gets built	interpretability
`LS("A")`	$\beta \cdot a$	`Linear(width, 1)`, no bias — **one parameter per feature column** (one for a continuous parent, `levels` for a one-hot ordinal, identified only as differences `w[k]-w[0]`)	`exp(β)` is an odds ratio
`CS("A")`	`g(a)`, additive	`ComplexShift`: NN `[64, 128, 64] → 1`	plot `g`
`CS("A","B")`	`g(a,b)` — joint	one NN over the concatenated features	interaction *in the shift*
`CS("A") + CS("B")`	$g_1(a) + g_2(b)$	two NNs, scalars added	GAM-style, each effect plottable
`VC("A","B", t="T")`	$(\beta_0 +$ `b_θ(a,b))·x_t`	scalar β_0 + zero-initialised penalized NN `[16] → 1`; the treatment value multiplies, it never enters the net	$\beta_0 \approx$ constant effect, `flow.varying_coef` reads `β(x)`

A node takes **at most one `I` term with parents**. A term list is therefore always purely additive on the latent scale. Interactions exist only *inside* a term. Lists and `+` sums are interchangeable: `[LS("A"), CS("B")]` ≡ `LS("A") + CS("B")`.

Three knobs on the terms:

- **`transform=` on `I`** picks the class of `h_ϑ` for a continuous node. There are three classes:
  - `"bernstein"` is the default. It has 20 coefficients. Its tails extrapolate with the boundary slope.
  - `"spline"` is a monotone RQ spline. It has 23 params at `bins=8`. Its tail slope is fixed.
  - `"affine"` has 2 params. The latent is then exactly logistic.

  Ordinal nodes have no transform to pick. Their intercept is the cutpoint vector, `P(x ≤ k) = σ(ϑ_k − shift)`.
- **`input_transform=` on `CI`/`CS`/`VC`** transforms that term's continuous network inputs. The statistics freeze at the first fit. The value is `"minmax"`, `"standardize"`, or a callable `fn(x, train)`. `LS` and the `VC` treatment stay raw, so their coefficients keep their units.
- **`units=` on `I`/`CS`/`VC`** sizes the term's network. For example, `units=[16]` gives one hidden layer of 16 neurons. The defaults match the PyTorch reference of this package (buehlpa/TramDag, `tram_models.py`). The defaults do **not** match the TRAM-DAG paper's own R nets. A replication therefore sets `units=` and `activation=` explicitly, as the configs in `experiments/paper/` do.

Feature widths: a continuous parent enters raw (1 column). An ordinal parent enters one-hot (`levels` columns). Abduction is exact for continuous nodes and truncated-logistic for ordinal ones. `flow.sample(u=flow.abduct(df))` therefore reproduces `df` exactly / level-exactly.

There are two ways to fit the model: a stochastic optimizer (`fit`) and the classical fit (`fit_classical`). In an all-`ls` model, each node-conditional is a classical transformation model. For these models the classical fit is deterministic and takes seconds. The measurement on the CI runner is ~10 s against ~200 s for Adam. That measurement predates the 2026-09 epoch cut, which made the Adam route ~3.5x faster. CI re-measures the numbers on the next push.

`docs/fitting.md` gives more information.

Validation

Two layers, deliberately separate.

The framework's own test suite (`tests/`) measures the library against three inline data-generating processes. The test suite carries these processes itself, so it needs no research code to run. Its strongest claim is an equality, not a similarity. An all-`ls` outcome node *is* an ordered-logit model. The flow's MLE must therefore match `statsmodels OrderedModel` on the same design matrix, and it does. The varying-coefficient acceptance bars (effect recovery, propensity centering) are pinned the same way.

`tests/README.md` documents what the tests guarantee and how each ground truth is obtained.

The paper replications (`experiments/`) are separate. There is one self-contained script per dataset. Each script keeps its hyperparameters in a sibling YAML file. Each area commits its expected results under `ground_truth/`. A dedicated workflow runs the scripts and compares the results.

experiment	paper	demonstrates
`triangle.py` (`linear-ls`, `linear-cs`, `atan-cs`, `sin-cs`)	§6.1, C.3	LS coefficient recovery (β = 2, −0.2, +0.3), CS curve ≡ −f(x_2) for non-monotone f
`triangle_mixed.py` (`linear-ls`, `exp-cs`)	§6.2	mixed data L1/L2 + the C.4 odds-ratio check (OR ≈ 7.4)
`vaca.py`	§5.1–5.2	the bimodal L1 case a default CNF misses; L2 `p(`x_3 `| do(`x_2`))` against analytic means
`carefl.py`	§5.3	L3 counterfactual curves vs **analytic** truth
`validate_ls.py`	—	flow ≡ `statsmodels` ≡ R `MASS::polr` to ~4 decimals with `fit_classical` (a converged Adam `fit` gets ~1e-3), on a frozen synthetic cohort with a known true effect

Sign note: ordinal shifts are *subtracted* here but *added* in the paper. Fitted ordinal weights therefore carry the flipped sign of the paper values. Each `truth.json` records both conventions. `experiments/paper/PAPER_COVERAGE.md` gives a figure-by-figure account of what the replications reproduce and what they do not reproduce (the competing CNF/NSF baselines).

Training speed — `docs/training-speed.md` covers:

- the schedules,
- the per-node freezing,
- L-BFGS,
- the device benchmarks.

Paper replication — `docs/paper-replication.md` covers the eight `experiments/paper` variants:

- every hyperparameter with its source in the paper's R code,
- the deviations,
- the numbers (paper / previous protocol / now).

Testing policy

See the `tests/README.md` file for more details.

Layout

```
src/tramdag/            spec.py terms.py transforms.py conditioners.py
```

```
                        nodes.py flow.py fitting.py readouts.py scores.py
                        callbacks.py plots.py     <- the framework, and nothing else
tests/                  unit tests, identities, acceptance bars, three inline DGPs
experiments/            research code, one directory per area, each self-contained:
                          paper/       the replications + generators + frozen data
                          benchmarks/  training-speed and machine measurements
                          misc/        the classical-MLE validation
                        paper/misc: <name>.py + <name>.yaml, data/,
                        ground_truth/, tests/, results/ (benchmarks reads the
                        other areas' data and pins no ground truth)
notebooks/              five executed examples: the combined Colab
                        introduction and demo, training strategies,
                        additive-vs-joint intercepts, varying coefficients,
                        classical fitting
docs/                   model.md and notation.md (what the model is),
                        interpretation.md (reading a fitted one), fitting.md,
                        training-speed.md, paper-replication.md, scores.md,
                        varying-coefficients.md, code-map.md, zuko-upstream.md,
                        architecture.md + adr/ (the 1.0 design and its refusals)
```

`CLAUDE.md` documents these implementation conventions, and tests pin them:

- the latent-scale signs,
- the raw parent encoding and the one-hot parent encoding,
- the log-space ordinal likelihood,
- the seeding.

Citation

If you use `tramdag`, please cite the method paper:

```bibtex
@inproceedings{sick2025tramdag,
  title     = {Interpretable Neural Causal Models with TRAM-DAGs},
  author    = {Sick, Beate and D{\"u}rr, Oliver},
  booktitle = {Proceedings of the 4th Conference on Causal Learning and Reasoning (CLeaR)},
  series    = {Proceedings of Machine Learning Research},
  volume    = {275},
  year      = {2025},
}
```

The model

A TRAM-DAG is one triangular normalizing flow whose sparsity is a causal DAG. This page states the model in the notation of the paper, and maps each part of it onto the constructors in `tramdag.spec`. For the symbols themselves see notation.md. To read a model after it is fitted see interpretation.md.

The transformation function

The variables have a causal ordering. For each variable a TRAM-DAG fits a monotone transformation function h. The function maps the observed value to a latent scale, conditional on the parents of that variable:

$$
\begin{aligned}
u_1 &= h(x_1) \\
u_2 &= h(x_2 \mid x_1) \\
u_3 &= h(x_3 \mid x_1, x_2) \\
u_p &= h(x_p \mid x_1, x_2, \dots, x_{p-1})
\end{aligned}
$$

This is the observed-to-latent direction. It is Eq. 2 of the paper, $F_{X|\text{pa}}(x) = F_U(h(x \mid \text{pa}))$, and in code it is `u = h(x) + shift`.

The direction matters for cost. Training evaluates h directly to score the likelihood, which is cheap. Sampling runs the inverse $x_i = h^{-1}(u_i \mid \text{pa})$. That inverse has no closed form, so bracketed bisection solves it, and sampling is the more expensive direction.

Together the h's form one triangular flow. Each variable depends only on its causal parents $\text{pa}(x_i)$, which is a subset of its predecessors. The Jacobian sparsity of the flow is therefore the DAG itself.

The latents $u_1, \dots, u_p$ are standard logistic. That choice is what makes the fitted parameters interpretable, because a shift on the latent scale is then a log-odds ratio.

The four components

The transformation decomposes additively on the latent scale, which is what keeps the interpretation valid. Each node's h is

$$
u_i \;=\; h(x_i \mid \text{pa}(x_i)) \;=\; \underbrace{h_\vartheta(x_i)}_{\text{intercept}} \;+\; \underbrace{\textstyle\sum_j \beta_{ij}\, x_j}_{\text{linear shifts}} \;+\; \underbrace{\textstyle\sum_k g_{ik}(x_k)}_{\text{complex shifts}},
$$

and every causal parent enters through exactly one term. Take x_5 with parents $\text{pa}(x_5) = \{x_1, x_2, x_4\}$ as the example.

- **Simple intercept, SI.** $h_\vartheta(x_5)$ has constant parameters. It is a flexible monotone baseline, a Bernstein polynomial by default, and it is the same for every observation.
- **Complex intercept, CI.** The parameters ϑ are themselves a function of some parents. The whole transformation bends with the parent, which permits interactions that an additive shift cannot express.
- **Linear shift, LS.** $\beta_{51}x_1 + \beta_{52}x_2$. One interpretable number per parent.
- **Complex shift, CS.** $g(x_4)$, an unrestricted network of the parent. It stays additive on the latent scale.

Sampling has to invert only the intercept, because the shifts move to the other side:

$$
x_5 = h^{-1}(u_5 \mid x_1, x_2, x_4) = h_\vartheta^{-1}\Big(u_5 - \underbrace{\beta_{51}x_1 + \beta_{52}x_2}_{\text{LS}} - \underbrace{g(x_4)}_{\text{CS}}\Big).
$$

The components in code

Each node declares its transformation as an additive formula of terms. The formula is the node's first argument, written as a list or as a `+` sum. Each constructor names the parents its term depends on.

paper component	`tramdag`
SI, baseline $h_\vartheta(x_i)$ with constant ϑ	automatic: every node owns a monotone transform. Without an intercept term its ϑ is a free parameter vector
CI, ϑ depends on parents	`I("X1")`. Several parents in one `I(...)` feed one joint network, which is how interactions arise
LS, $\beta_{ij}x_j$	`LS("X1")`, a single weight and no bias
CS, $g_{ik}(x_k)$	`CS("X1")`, an additive network

`I(...)` dispatches on its arguments. Without parents it is a simple intercept, with parents a complex one. `SI()` and `CI(...)` spell that out and check the arity.

The formulas for each combination of terms, and the difference between a joint and an additive grouping, are tabulated in the README.

Ordinal nodes

An ordinal node has no monotone transform to choose. Its intercept is the cutpoint vector of an ordered logit. The model is $P(Y \leq k) = \text{sigmoid}(\theta_k - s)$, where θ holds the increasing cutpoints and s is the total shift.

Note the sign. A continuous node adds its shift on the latent scale, and an ordinal node subtracts it. Both follow the original TRAM-DAG conventions, and the test suite pins them. A parent that raises a continuous child therefore gets a negative weight, which interpretation.md works through.

What the model cannot do

- The latent distribution is fixed to standard logistic. It is not learned.
- Every parent must enter through exactly one edge-owning term. The one exception is a varying-coefficient modifier, which can also act prognostically through another term.
- The DAG is an input. `tramdag` fits the mechanisms of a graph you supply, and does not discover the graph.
- Identification is the usual causal one. No hidden confounding, and the graph has to be right.

Notation

This page defines the notation. Every guide, docstring and notebook in this repository follows it.

Symbols

Random variables are capital Latin letters, for example Y. Their realizations — observed or sampled values — are lowercase Latin letters, for example y. The cumulative distribution function of Y is F_Y and the density is f_Y. A hat marks an estimate: $\widehat{F}$ is the fitted distribution, F the true and often unknown one.

Sets and vectors are bold, for example $\mathbf{X} = [X_1, \dots, X_J]$.

A single optimized parameter is a lowercase Greek letter, for example ϑ or β. A vector of parameters is bold lowercase, for example $\boldsymbol{\vartheta} = (\vartheta_1, \dots, \vartheta_M)^\top$. A tuple that collects several parameter groups — for example all weight matrices of a network — is bold capital, for example $\boldsymbol{\Theta}$.

A function that a parameter vector defines carries it as a subscript: $h_{\boldsymbol{\vartheta}}$ is the monotone transformation with coefficients $\boldsymbol{\vartheta}$.

The model's symbols

Symbol	Meaning	In code
X_j, Y	the variables of the DAG	node names in the spec
$\mathrm{pa}(X_j)$	the causal parents of X_j	`node_parents`
U_j	the standard-logistic latent of node j	`u = flow.abduct(df)`
h_ϑ	the monotone transformation of a node	the `I` term; `theta` in docstrings
ϑ_k	the ordinal cutpoints, elements of ϑ	`ordinal_cutpoints`
β	a linear-shift coefficient; e^β is an odds ratio	`LS`; `flow.ls_coefficients()`
g_j	a complex shift, an NN of one term's parents	`CS`
β_0	the constant treatment effect of a `VC` term	`beta0`
b_Θ	the penalized effect-modification network	the `VC` net; `units=` sets its layers
λ	the L2 weight on b_Θ	`penalty=`
σ	the logistic function	`torch.sigmoid`

Every node's model is one additive formula on the latent scale:

$$u \;=\; h(x \mid \mathrm{pa}(x)) \;=\; h_{\vartheta(\cdot)}(x) \;+\; \sum_j \beta_j\, x_j \;+\; \sum_k g_k(x_k) \;+\; (\beta_0 + b_\Theta(x_{\mathrm{mod}}))\, x_t.$$

For a continuous node the shifts are added, as written. For an ordinal node the shift is subtracted inside the sigmoid: $P(Y \le k \mid \mathrm{pa}) = \sigma(\vartheta_k - \mathrm{shift})$.

Plain-text fallback

Docstrings and terminal output cannot render Greek subscripts. There, `theta` stands for ϑ, `h_theta` for h_ϑ, `b_theta` for b_Θ and `beta0` for β_0.

Reading a fitted TRAM-DAG

A TRAM-DAG is a normalizing flow, but it is built so that parts of it stay readable. This page collects the read-outs and says what each number means. For the model itself see model.md, and for the symbols see notation.md.

Every claim on this page is checked somewhere that runs. The links say where, so nothing here is a number typed into prose.

Which term owns which edge

`flow.to_matrix()` gives the labelled adjacency matrix of the model. Rows are parents and columns are children. A cell holds the term tag: `LS`, `CS`, `CI`, `VC` for a treatment edge, or `VCm` for a varying-coefficient modifier. An empty cell means no edge. A multi-parent term carries its parent group as a suffix, and several terms in one cell join with `+`.

This is the meta-adjacency view of the paper. It is also the fastest way to confirm that the model you wrote is the model you meant. `plot_dag(spec)` draws the same thing, with one edge style per term.

Exercised in the varying-coefficients notebook.

A linear shift is a log-odds ratio

`flow.ls_coefficients()` gives `{node: {parent: weights}}` for the `LS` terms only. A `CS` or `VC` shift is a network and has no single weight, so those are skipped.

Two things decide how to read a weight.

The sign follows the latent-scale convention. A continuous node adds its shift, so a parent that raises the child gets a *negative* weight. An ordinal node subtracts its shift, so the sign is the one you expect there. This is not a quirk of the implementation. It is the convention of the original TRAM-DAG work, and the test suite pins it.

The scale is the latent scale, not the data scale. For a continuous node the weights carry the transform's scaling. A single weight is therefore not comparable to a regression coefficient. Their *ratio* is comparable, because the scaling cancels. An ordinal node has no such scaling, so $\exp(\beta)$ is an odds ratio directly.

An ordinal parent gives one weight per level, from its one-hot encoding. Those weights are identified only up to a common constant. The one-hot columns sum to one in every row. Adding a constant to all of them, and subtracting it from the intercept, leaves the likelihood unchanged. Read them as differences. `w[k] - w[0]` is the level-k-against-level-0 log-odds ratio, and it is the column that `flow.design_matrix(df, node, drop_first=True)` drops.

Checked in `notebooks/demo_tram_dag_colab.py`, which recovers a coefficient ratio from a known process, and in `notebooks/classical_fit_tram_dag.py`, which compares the weights against `statsmodels` and R. The odds-ratio reading is pinned in continuous integration by `experiments/paper/triangle_mixed.py`.

Ordinal nodes: cutpoints and the interventional PMF

An ordinal node's intercept is its cutpoint vector, not a transform. The model is

$$P(Y \le k \mid \text{pa}) = \text{sigmoid}(\theta_k - s(\text{pa})),$$

with increasing cutpoints θ and s the total shift. `tramdag.transforms.ordinal_cutpoints` turns the unconstrained parameters into those increasing cutpoints, which is how you read a fitted baseline.

`flow.pmf(df, node, do=...)` gives the class probabilities per row, in closed form. It needs no sampling, and `do=` makes it interventional by overriding columns before the parents are read. That combination is what lets a treatment-effect question be answered exactly rather than by Monte Carlo.

Exercised in the classical-fitting notebook, and under `do=` in `experiments/misc/validate_ls.py`, which is checked against committed ground truth on every run.

Continuous nodes: density and one fitted shift

`flow.density(df, node, grid, do=...)` is the continuous counterpart of `pmf`. For every row it evaluates $p(\text{node} = g \mid \text{pa})$ at each grid value, in closed form from the transform. Use it to look at a fitted conditional without sampling it.

`flow.shift_curve(node, parent, grid)` evaluates one fitted shift term along a grid of parent values. Use it to plot a `CS` term against the function it was supposed to learn. It goes through the term's own evaluation, so a custom term and an `Fn` term work too.

A caution on both: a `CS` term is identified only up to a constant, because that constant can move into the intercept. Compare a fitted curve to the truth after mean-centring both, not point by point.

The price of a complex intercept

`I(...)` with parents is the flexible end of the family. The parents control the transform parameters, so the node can bend in ways no additive shift can express. The cost is that there is no single coefficient to read.

That is a real trade-off, and it is measurable. `experiments/paper/` fits the same process twice, once with an `LS` edge and once with a `CS` edge. It compares both against the known truth, and paper-replication.md reports the result. The short version has two halves. Where the true edge is linear, the interpretable model gives up very little likelihood. Where the edge is curved, the flexible model wins, and the interpretable one absorbs the curvature into a biased single number.

A middle option exists. `I("a", "b", allow_interaction=False)` builds one network per parent and sums their parameter vectors, so the parents act additively on the transform. `flow.intercept_contributions(df, node)` then decomposes that sum into mean-centred per-term parts, using the usual additive model convention, so the parts become comparable. This is a post-hoc read-out and changes nothing about the fitted model.

Worked through in the intercept notebook.

An effect that varies with a covariate

`VC(*modifiers, t=...)` fits $\beta(\text{modifiers}) \cdot x_t$, and `flow.varying_coef(df, node)` reads the fitted β back. It is deterministic, needs no abduction, and for a binary treatment it equals the abduction difference.

`flow.scores(df, node)` and `flow.effect_modifier_scan(df, node, t=...)` come before that decision rather than after it. They rank candidate modifiers by how much the treatment coefficient's per-observation scores drift when the rows are ordered by each candidate. A cheap all-`LS` fit is enough, so the shortlist is measured rather than guessed.

See varying-coefficients.md and scores.md, and `notebooks/varying_coefficients.py`.

Fitting a TRAM-DAG: how training works

This document is the technical reference for training a `CausalFlowDAG`. It covers three subjects:

- the likelihood
- the two fitting paths, `fit` and `fit_classical`
- the hooks of these two paths

How the flow is built: one module, one sub-model per node

A `CausalFlowDAG` is a single `torch.nn.Module`. It holds **one independent sub-model per variable**. Each sub-model has these parts:

- an intercept, which produces the transform parameters `θ`
- the monotone 1-D transform `h`, which has no learnable weights of its own and only range buffers
- one shift module per shift term

The nodes share **no parameters**. One module bundles them, and one optimizer trains them. The DAG structure lives entirely in *which parents each node reads*. There is no edge weight matrix and no shared trunk.

The code map says which class implements which term. Continuous parent features enter raw. Ordinal parent features enter as one-hot columns.

How the likelihood is computed

A TRAM-DAG maps iid standard-logistic latents `U` to the observed `X` in causal order. Node `i` reads only its parents (earlier variables, as *data*). Therefore the Jacobian of `U → X` is **triangular**, and its log-determinant is the sum of the per-node 1-D terms. The joint log-likelihood therefore **decomposes per node**:

```
log p(x) = Σ_i log p(x_i | pa(x_i))
```

`CausalFlowDAG.node_log_prob` computes one term per node and `log_prob` sums them. For a node, given `θ, shift` from `theta_shift`:

- **continuous** — change of variables through the monotone transform:

  ```
  u = h(x; θ) + shift
  log p(x | pa) = log f_logistic(z) + log |dz/dx|
  ```

  That is, the term is the standard-logistic density at the latent `u` (`StandardLogistic.log_prob`) plus the transform's log-derivative. `ut.forward` returns this log-derivative as `ladj`. This is the 1-D Jacobian term that makes the result a proper density, not only a score.

- **ordinal** — an ordered-logit / proportional-odds head, `P(x ≤ k) = σ(θ_k − shift)`. `ordinal_log_prob` evaluates it as the log of the cutpoint-interval probability. The computation runs in log-space via `logsigmoid`/`log1mexp`, because the naive sigmoid difference underflows to exactly-zero gradients in float32.

The training loss is the summed per-node **mean** NLL over the batch (`Σ_i mean_rows(−log p(x_i | pa))`). In contrast, `log_prob` returns the per-row joint, which scores whole observations.

Consequence used by both optimizers: because parents enter as data, the per-node gradients are independent. Therefore a joint fit of the summed loss is identical to a separate fit of each node. This independence licenses three things:

- per-node learning rates
- freezing, which a callback does (see below)
- the all-`ls` classical fit

Path A — stochastic optimization (`fit`)

`CausalFlowDAG.fit` is the general-purpose trainer. Any `cs`/`ci` edge requires it. Mechanics:

- **One optimizer over all parameters**. The default is `Adam(lr=learning_rate)`. You can pass any `torch.optim.Optimizer` as `optimizer=` instead. This is exactly per-node training, as the consequence above explains. For per-node rates, `per_node_adam` builds the per-node parameter groups.
- **Minibatches**: a fresh `torch.randperm` shuffle each epoch (`seed=` seeds it). The loss is the summed per-node mean NLL on the batch, plus the `VC` penalty.
- **`calibrate(train_df)`** — the first `fit` calls it. Every term then freezes its own data-dependent state. The intercept maps the train `range_q`/`1-range_q` quantiles onto its transform's domain. Every term-level `input_transform=` freezes its statistics. Calibration never touches the weights. A checkpoint carries the flag, so no `fit` ever recalibrates a loaded model.
- **`flow.init_marginals(train_df)`** — the calibrated start is a separate step, and it is always explicit. It resets every Bernstein or ordinal simple intercept to the empirical marginal of its column. That value is `logit(F_hat)` in both cases, as control points or as cutpoints. The spline, the affine and the `range_q=0` transforms have no such start. The call is a pure init that leaves the MLE unchanged. You can call it at any time, including on a trained or a loaded flow.
- **Validation, Keras-shaped** — two arguments turn validation on. `validation_data=` takes a DataFrame. `validation_split=` takes a float, and it uses the LAST fraction of `train_df`

with no shuffle. Only the head of the frame calibrates, so there is no leakage. With either argument, `fit` computes the per-node validation NLL once after every epoch, into `flow.history["val"]`. `validation_batch_size=` chunks that pass.

- **Logging** — the shipped callbacks read `flow.history["val"]` there. `flow.history["lr"]` records the optimizer's rate after every epoch. With `per_node_adam` that record is a `{node: lr}` dict. A schedule's decisions are therefore on record, and you need no callback of your own. `verbose=N` prints every Nth epoch plus the final one. The default is 0, which is silent.
- **`callbacks=`** — it takes one entry or a list. A `tramdag.callbacks.Callback` hooks `on_fit_begin`, `on_epoch_end` and `on_fit_end`. Its docstring is the contract for the timing, the stop rule and the VC re-centering order. A bare callable in the list is an `on_epoch_end` hook with the signature `cb(flow, epoch, optimizer)`. Any `True` return stops the fit. Schedules, snapshots and coefficient trajectories live here.
- **The common recipes ship in `tramdag.callbacks`.** `EarlyStopping` restores the best-validation weights automatically, and it takes an optional patience. `PerNodePlateau` plus `per_node_adam` give per-node decay and freezing. All of them read `history["val"]`.
- **Centered `VC` propensities are a column.** `VC(center="ps")` names the column of the training frame that holds the out-of-fold `P(t=1|pa_t)` per row. That column splits and minibatches with the frame. See varying-coefficients.md.

Training strategies

Every strategy below is `fit` plus a callback, or `fit` plus a few lines of your own. Pick a strategy by model class. Each strategy has a copy-paste example. The examples assume a built `flow = CausalFlowDAG(spec)` and the pandas frames `train_df`/`val_df`.

The empirical rule of thumb has two halves. **All-`ls` models train to the MLE and keep the final weights. Flexible models (CI, CS or VC) validate and keep the best weights.** Flexible models overfit observational confounding at the MLE, as the finding below shows.

Strategy	When
exact MLE — `fit_classical` (Path B, below)	all-`ls` spec; deterministic, seconds
plain Adam	all-`ls` with a shift `fit_classical` refuses, quick looks
multi-phase Adam	a tighter MLE without a scheduler
best-validation weights — `EarlyStopping()`	any CI/CS/VC model; the recommended recipe (register it — fit has no default)
… + patience — `EarlyStopping(patience=)`	also stop once the best is that many epochs old
global plateau schedule	decaying one shared rate beats picking one
per-node plateau — `PerNodePlateau`	nodes converge at different speeds; self-stopping

The code below gives one line for each Adam recipe, as a quick reference. The exact-MLE path is Path B below, and the global plateau rule needs a `Callback` of your own. Every Adam strategy runs end to end, with its output and its checks, in `notebooks/training_strategies.py`. Every documentation build executes that notebook, so the notebook is the source of truth. If a snippet here disagrees with the notebook, the notebook is right.

```
# plain Adam: one rate, one loop, the final weights
flow.fit(train_df, epochs=500, learning_rate=1e-3, batch_size=256, verbose=100)

# two phases: a second call continues training, so this is a schedule
for epochs, lr in [(800, 1e-2), (700, 1e-3), (500, 1e-4)]:
```

```
    flow.fit(train_df, epochs=epochs, learning_rate=lr)

# best-validation weights: the flexible-model recipe, restored at fit end
flow.fit(train_df, epochs=4000, validation_data=val_df,
         callbacks=EarlyStopping())

# ... and stop once the best epoch is that old
flow.fit(train_df, epochs=4000, validation_split=0.1,
         callbacks=EarlyStopping(patience=200))

# one rate per node, each freezing on its own score; stops when all froze
flow.fit(train_df, epochs=4000, validation_split=0.1,
         optimizer=per_node_adam(flow, lr=1e-2),
         callbacks=PerNodePlateau())   # patience=15, freeze=50
```

Import `EarlyStopping`, `PerNodePlateau` and `per_node_adam` from `tramdag.callbacks`. For one shared rate instead of per-node rates, carry torch's `ReduceLROnPlateau` in a `Callback` of your own. The notebook's `GlobalPlateau` is that class, ready to copy.

Two measured facts decide between the first two. The multi-phase recipe lands within ~1e-5 of statsmodels on an all-`ls` model. A single converged constant-rate run gets ~1e-3. If you need the tighter agreement and the spec is all-`ls`, use `fit_classical` instead, which is exact and faster.

`experiments/benchmarks/bench_training.py` measures the per-node recipe.

Three details are easy to get wrong:

- `validation_split` takes the **last** rows, unshuffled. If the row order of the frame means anything, shuffle the frame first.
- A second `fit` call **continues** training, and `history` accumulates across the calls. The call does not start over.
- A post-fit `load_state_dict` skips the varying-coefficient re-centering. On a spec with a `VC` term, restore the weights from the `on_fit_end` hook of a `Callback` subclass instead. That hook runs before the re-centering step. `EarlyStopping` already does this.

The exact-MLE and warm-start strategies are Path B, below. training-speed.md gives the measured time-to-target for each recipe.

Path B — classical optimization (`fit_classical`)

`CausalFlowDAG.fit_classical` is the dedicated optimizer for **all-`ls`** models. In such a model, every node-conditional is a classical transformation model, that is an ordered-logit or a Colr model. It raises on any `cs`, `ci` or `vc` term.

- **Full-batch, float64, L-BFGS** with a strong-Wolfe line search. There are no minibatches, no schedule and no early stopping. Therefore the fit is **deterministic**: the same init gives bit-identical results. The fit also lands on the **exact MLE**. `fit_classical` matches `statsmodels` and R to ~4 decimals. A converged Adam `fit` gets within ~1e-3.
- **Solver budget** — `max_iter=400`, `tol=1e-9` and `history_size=50`. The fit is one L-BFGS run with torch's own stopping rule. The run ends when the NLL or the parameters move by less than `tol`, or at `max_iter`. The report's `n_iter` is torch's count. The report's `converged` says whether a tolerance ended the run, and not the cap. `history_size` is the L-BFGS memory.
- **float64 is a transient compute mode.** The fit runs in double and restores float32 afterwards. Checkpoints stay float32.
- **Convergence** — the flag is true when torch's `tolerance_change` ended the run before `max_iter` did. The flag is *advisory*. A Bernstein intercept and weakly-identified directions continue to drift along zero-curvature valleys after the likelihood is at the optimum. Rare one-hot levels and a flat treatment-effect ridge are two such directions. Correctness comes

from a comparison with classical software, and not from the flag. That comparison is `python -m misc.validate_ls classical`.
- Read the fitted coefficients with `ls_coefficients()`.

Warm-start handoff: classical fit, then keep training

`fit_classical` leaves the model at the MLE in float32, ready for any normal operation. A `fit()` call from there **stays put**. This behaviour is a check that the classical solution really is the optimum. It is also a way to use the classical fit as a fast, principled initialization:

```
flow.fit_classical(train_df)                          # exact MLE, seconds
before = flow.ls_coefficients()["y"]
flow.fit(train_df, epochs=300, learning_rate=1e-3)  # a gentle Adam phase ...
after = flow.ls_coefficients()["y"]                   # ... barely moves
```

A small drift means that the classical fit was already at the optimum. The same handoff warm-starts the `beta0` of a `VC` term. varying-coefficients.md holds the measured recipe.

Memory and disk during fitting

Neither fitting path writes to disk. Three things live in RAM:

- the parameters
- the optimizer state
- the `history` dict

Whatever a callback records is yours. The only disk I/O in the module is the explicit `save()`/`load()`. The `results/` artifacts in this repository come from the experiment scripts, and not from the library.

Optimizer choice

Today the package uses **Adam** for flexible models. It uses **L-BFGS** in float64 for all-`ls` models. The per-node decomposition makes optimizer swaps cheap through `optimizer=`. Three candidates wait for a measurement:

- IRLS for the `ls` path
- per-node mixing
- modern Adam variants

`experiments/benchmarks/bench_training.py` benchmarks every candidate before the package adopts it.

How fast can a CausalFlowDAG train?

This document benchmarks five things:

- learning-rate schedules
- per-node freezing
- batch sizes
- devices
- LBFGS

The benchmark ran in June 2026 on an Apple-silicon Mac mini with torch 2.12. It ran on the CPU, unless a row notes another device.

To reproduce the benchmark, run the training benchmark. The command is `cd experiments && uv run python -m benchmarks.bench_training`. For one seed on cpu, add `--quick`. The full grid takes ≈ 35 min.

For a quick **cross-machine** comparison, use the self-contained `experiments/benchmarks/perf_machine.py`. It runs fixed 200-epoch workloads on all available devices. It writes a machine fingerprint to JSON. It needs nothing but `pip install tramdag`. The raw CSV is a local run artifact, and it stays out of the repository.

The recipes, and where they live now

Since 0.4, the recipes that this benchmark compares are **callbacks**. They are training strategies, and they are not part of the model. fitting.md says what each recipe is and gives one line of code for each Adam recipe. The worked version of every recipe is in `notebooks/training_strategies.py`. This page is only the measurement.

One behaviour matters here, because the numbers below turn on it. `PerNodePlateau` watches each node's own validation score. When that score does not improve by `min_delta` for `patience` epochs, the callback multiplies the node's rate by `factor`, which is 0.3 by default. The floor is 1e-3 of the start rate. After the rate has decayed 100x and then stays flat for `freeze` epochs, the node leaves training at rate 0. The `min_delta` default is 1e-4, and the stroke run in this benchmark uses 1e-5.

This behaviour is valid, because the per-node losses have independent gradients. It lets the fit delete whole epochs, and not only shorten them.

One more recipe is a global plateau rule, which is one shared rate rather than one rate per node. That is torch's `ReduceLROnPlateau` on the summed validation NLL, and it is the rule the paper reference uses. `experiments/paper/helpers.py::fit_paper` drives it.

The exact-MLE path: an exact comparison with the classical methods needs no recipe. The classical methods are `statsmodels` and R `polr`/`tram`. Two plain `fit` calls at decreasing rates reach the exact MLE. `experiments/misc/validate_ls.py` runs a three-phase variant with 800/700/500 epochs at 1e-2/1e-3/1e-4 and batch 256. In 2026-09 this repository cut that budget from 4000/2000/1000. The cut gives the same MLE, a named-coefficient gap to statsmodels of 1.6e-5 against 1.8e-5, and about 3.5x less wall clock.

`fit` keeps the final weights. The guard test `tests/test_fit_hooks.py::test_torch_plateau_scheduler_prese` shows that a schedule through the hooks still lands the all-`ls` fit on the classical MLE within the usual tolerances.

LBFGS ships as its own method, `fit_classical`. It supersedes the hand-rolled recipe that this report benchmarked. The float32 variant measured here was fast (< 2 s) but seed-fragile (Finding #2). The float64 upcast in `fit_classical` fixed that.

Method: time-to-target, not loss-go-down

Each config runs once. The benchmark's callback records the per-epoch validation NLL *and* the wall-clock time. From these records, we measure the seconds until the fit is within a fixed gap of a cached long-run reference. The reference is the median over 3 torch seeds of a long run. The two workloads and their tolerances are:

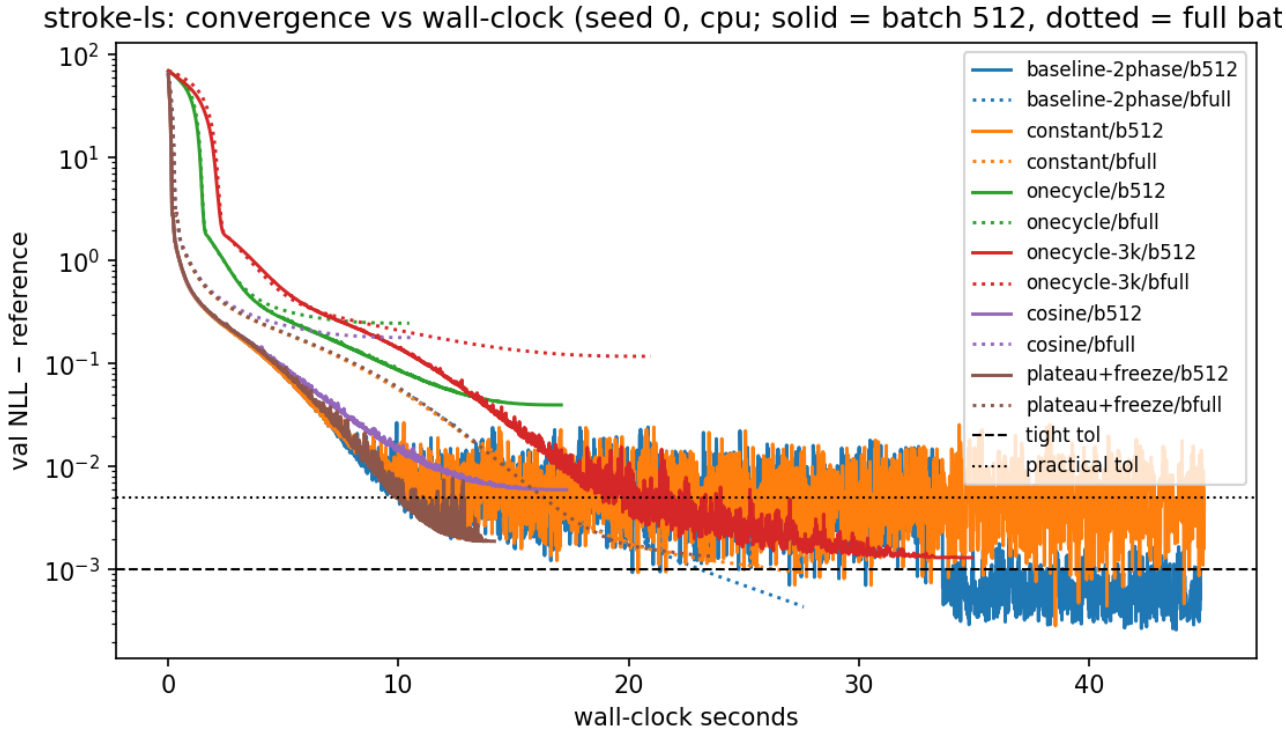
workload	model / data	reference NLL	tight tol	practical tol
stroke-ls	all-`ls` 5-node DAG, frozen `experiments/misc/data/magic-mrclean/ls` (n=1275, full-data MLE)	10.3042 (train)	+1e-3	+5e-3

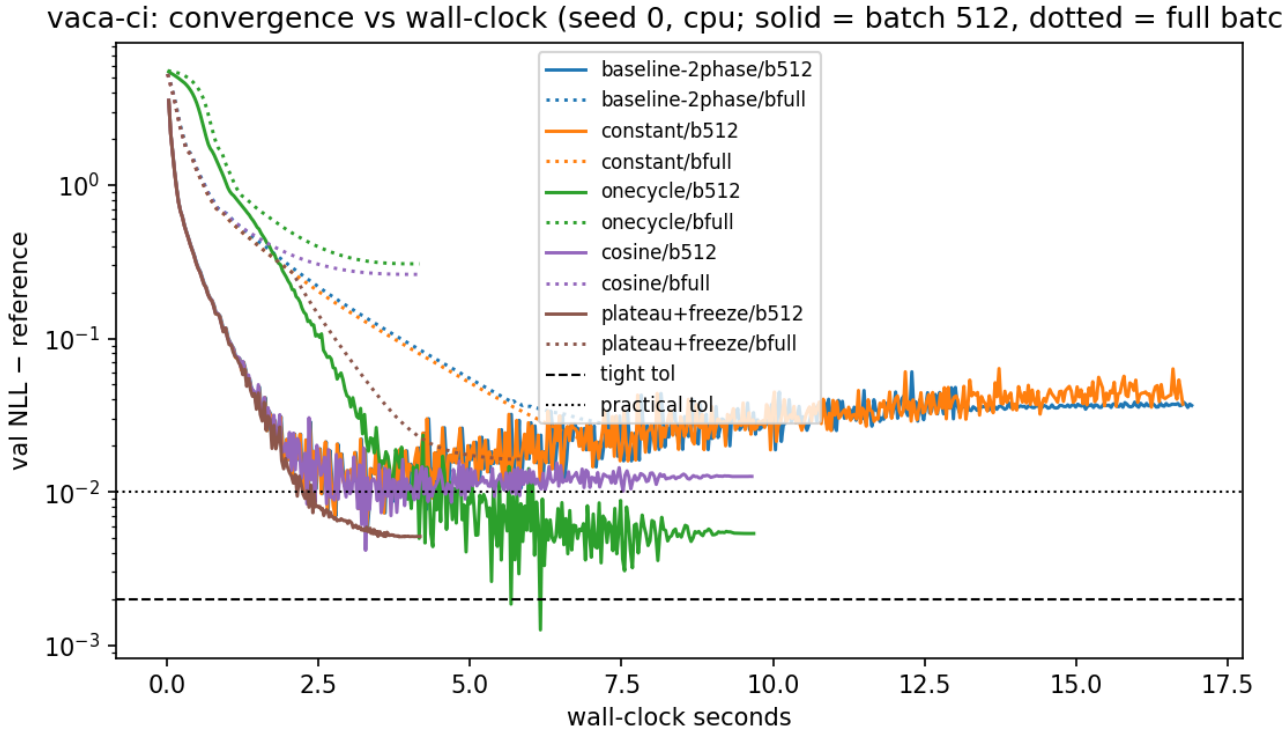
workload	model / data	reference NLL	tight tol	practical tol
vaca-ci	all-`ci` flow, frozen `experiments/paper/data/vaca` (n=5000, 90/10 split)	4.9632 (val)	+2e-3	+1e-2

Tight ≈ exact-MLE equivalence, which is a statsmodels/R-polr match. *Practical* ≈ coefficient-equivalent. A fit with gap ≈ 3e-3 already matches the R reference coefficients within the tolerances of `experiments/misc/validate_ls.py`.

The workloads are unchanged since the measurement, with the same frozen data and the same specs. A re-run therefore reproduces the machine-independent stroke-ls reference NLL 10.3042 exactly.

Results





The table shows the median seconds to target, at batch 512 on cpu. A “—” entry means that the fit never reached the target within the budget.

config	stroke-ls practical	stroke-ls tight	vaca-ci practical	vaca-ci tight	self-stops
baseline two-phase	9.0	**21.4**	2.1	2.8[1]	no (runs 40 s / 15 s)
constant 1e-2	9.1	21.5[2]	2.2	2.8[1]	no
plateau + freeze	**8.9**	— (gap 2e-3)	**2.0**	2.9	**yes — 13 s / 4 s total**
LBFGS (full-batch)	**1.6** (2/3 seeds)	— (gap 4–8e-3)	n/a	n/a	yes

[1] transient: the val-NLL curve dips through the target and then drifts away. Stroke needs the 1e-3 phase to *stay*. Vaca shows mild overfitting. Final gap for the vaca baseline is 0.037. A 520-epoch budget **underfits** vaca by ~0.03 nats. Plateau+freeze *stays* at its target. [2] constant lr at batch 512 stalls at gap 3–7e-3. Only the lr-decay phase closes the last decade. This is why the two-phase recipe pairs a constant phase with a decay phase.

Findings

1. **Per-node plateau decay + freezing is the best default-style trainer.** It reaches the same time-to-accuracy as the hand-tuned two-phase schedule. It needs **no budget tuning** to do so. It decays the lr of each node off that node's own validation curve. It freezes converged nodes, a real FLOP saving, because the per-node NLLs have independent gradients. And it **stops itself**. On stroke-ls it takes 13 s total against 40 s for the baseline. On vaca-ci it takes 4 s against 15 s, at an equal or better final NLL.
2. **LBFGS is fast but seed-fragile.** On 2/3 seeds, full-batch LBFGS reaches coefficient-level accuracy on the classical all-`ls` model in **< 2 s**. Adam needs 9 s for the same accuracy. The third seed stalls at gap 8e-3. An Adam warm start made it *worse* on every seed, because the fit then landed in a different basin. Use LBFGS as a fast first shot with the plateau trainer as the fallback, and not as the default.
3. **OneCycle is a "spend exactly this budget" scheduler.** Accuracy arrives only at the end of its anneal. At 1500 epochs it misses everything. At 3000 epochs it lands gap 1–2e-3. But you must know the right budget in advance. That requirement is the problem that we try to remove.
4. **Full-batch loses on time-to-target**, despite ~1.6× higher epoch throughput. It makes too few optimizer steps per second of compute at these
   n. Batch 512 is a good default. Very large batches (16k) only improved raw throughput at n=50k.
5. **MPS (Apple GPU) is 3–4× slower than the M-series CPU** at these model sizes. The reconstruction is identical, so the MPS result is correct. Kernel-launch overhead dominates sub-millisecond ops. Stay on CPU locally. CUDA on Colab-class GPUs is a different regime.
6. **A fixed budget wastes or under-spends.** Stroke budgeted 4000 epochs, and the converged work is done after ~1500. Freezing recovers that difference automatically. Vaca budgeted 520 epochs, which is ~0.03 nats short of converged. Fixed budgets are wrong in both directions. Adaptive stopping fixes both.
7. **Freezing helps, and a parallel node loop does not.** Freezing deletes whole epochs. Node-level overlap only time-slices the cores that each node's batched BLAS operations already saturate. It measured as contention, not as speedup. Overlap can pay only where the per-node kernels do not fully use the hardware, which means tiny nodes on a large GPU. For that case the tool is a fusion of same-shaped nodes, not threads.

Recommendation

The everyday recipe is a plateau callback with a generous `epochs` ceiling. The shipped per-node variant is `tramdag.callbacks.PerNodePlateau`. For one shared rate, carry torch's `ReduceLROn-`

`Plateau` in a `Callback` of your own. fitting.md lists both, and `notebooks/training_strategies.py` runs each of them.

One finding became a package default. `epochs` has no default, because Finding 6 shows that a fixed budget cannot be right for every workload. Every in-repo caller states its recipe in its own YAML.

Per-observation scores & the effect-modifier scan

`flow.scores(df, node)` returns the score $\psi_i = \partial \ell_i / \partial \theta$ of each observation, for the node's interpretable shift coefficients. These coefficients are every `LS` weight and every `VC` term's `beta0`. An `LS` weight gives one column per continuous parent and one column per one-hot level of an ordinal parent. The `beta0` column is named after the treatment.

The computation is **analytic and exact**, with no autograd. Shifts enter the latent additively. Thus $\partial \ell_i / \partial \beta = (\partial \ell_i / \partial s_i) \cdot x_i$, with $\partial \ell_i / \partial s$ in closed form (`src/tramdag/scores.py`). The function is a pure read-out. It does not touch the fitting or sampling code paths. At a fitted MLE, the sum of each column is ≈ 0. Tests pin this behavior and include a float64 finite-difference check.

Worked example: where does β(x) need to bend?

The score is the cheapest effect-modifier detector available. It uses the structural-change logic of model-based recursive partitioning (Zeileis and Hornik, Dandl et al. 2024). Before you fit an expensive model, fit the **cheap all-`LS` model**. This fit takes seconds and is deterministic. Then scan the treatment-coefficient scores:

```
flow = td.CausalFlowDAG(all_ls_spec, seed=0)
flow.fit_classical(df)  # seconds, exact MLE

scan = flow.effect_modifier_scan(df, node="Y", t="T")
#         stat   p_value  crit_5pct   flag
# X2      4.21    <0.001     1.3581   True     <- true modifier
# X3      3.05    <0.001     1.3581   True     <- true modifier
# X1      0.74     0.64      1.3581  False     <- prognostic only
```

For each candidate covariate, the scan orders the scores by that covariate and forms the scaled cumulative sum `B_j = Σ_{i≤j} ψ_(i) / (sd(ψ)·√n)`. Under a stable coefficient, `B` is a Brownian bridge. Thus `sup|B|` has the Kolmogorov distribution (5% critical value 1.358). A coefficient that truly varies with the covariate makes the ordered scores drift. The flagged covariates are the shortlist of `VC` modifiers to measure (`docs/varying-coefficients.md`):

```
spec["Y"] = td.ContinuousNode(
    td.CS("X1", "X2", "X3") + td.VC("X2", "X3", t="T")
)  # scan-informed
```

Read it as screening, not as a test of effect modification. The statistic measures how unstable the *cheap* model is along a covariate. Instability has two sources: a coefficient that genuinely varies, and a **prognostic part the cheap model gets wrong**. In the example above `X1` is unflagged, because its prognostic effect is linear. Make it quadratic and `X1` flags as strongly as the true modifiers (0.56 → 4.14 on the DGP of `notebooks/varying_coefficients.py`, which demonstrates both cases side by side). A flag therefore means "look here", and what you find can be a modifier or a misspecification — both worth knowing.

Notes: For a binary ordinal `LS` treatment, `t` resolves to the identified level-1-vs-0 contrast. For a `VC` treatment, `t` resolves to `beta0`. Candidates default to every column of `df` except the node and `t`. Modifiers do not need to be parents. For few-level (heavily tied) candidates, the ordering is only partial. Then read the scan as a ranking diagnostic, not as an exact-size test. You can also use the scores later for influence-function analyses and robust standard errors.

Varying-coefficient treatment effects: `VC(*modifiers, t=, penalty=)`

A `VC` term gives a node a **treatment-effect head with its own bias-variance budget** (issue #28). The term contributes

```
beta(modifiers) * x_on        with    beta(x) = beta0 + b_theta(x)
```

to the node's shift. Here `b_theta` is a deliberately small (one 16-unit hidden layer), **penalized** network. The point is not expressiveness. The point is that the flow estimates the effect function *with care*, not as a by-product:

```python
import tramdag as td

spec = {
    "X1": td.ContinuousNode(),
    "X2": td.ContinuousNode(),
    "X3": td.ContinuousNode(),
    "T": td.OrdinalNode(2, td.LS("X1") + td.LS("X2")),
    "Y": td.ContinuousNode(
        td.CS("X1", "X2", "X3")  # prognostic part g(x): as flexible as you like
        + td.VC("X2", "X3", t="T", penalty=1.0)  # effect beta(X2, X3) * T
    ),
}
flow = td.CausalFlowDAG(spec, seed=0).fit(
    train, epochs=500, learning_rate=1e-2, batch_size=512
)  # best-validation weights: callbacks.EarlyStopping, see fitting.md

beta = flow.varying_coef(df_new, "Y")  # (n,) array beta(x) — deterministic, y-free
beta0 = float(flow.nodes["Y"].shifts["T"].beta0.detach())  # interpretable main effect
```

`X2` and `X3` appear **twice**: as prognostic parents through `CS` and as effect modifiers through `VC`. This pattern is intended. Only the treatment node named by `t=` owns its edge, and a second term that declares it raises an error. Modifiers can repeat.

Why not `CS("T", "X2", "X3")`? (anti-pattern)

For a binary treatment, the multi-parent `CS` is equivalent in *expressiveness*. Any shift decomposes exactly as `s(x,t) = s(x,0) + [s(x,1) − s(x,0)]·t`. But the `CS` form has **no effect-specific regularization**. The likelihood rewards a good average fit of `s(x,t)`, and nothing rewards a smooth difference between the arms. The read-out is thus the difference of two jointly fitted, unregularized networks, which amplifies noise.

On the heterogeneous-effect validation DGP (see below), the `CS` reduced form reaches corr ≈ 0.5 against the true effect function (tramdag-simu#18 / PR #21). This holds **even when the model is exactly in-class**. The `VC` term reaches corr ≈ 0.99 on the same protocol (`tests/test_vc_term.py`, acceptance bar 0.9). Causal forests and R-learners work not because they *target* the effect, but because they **regularize** it (Nie & Wager 2021, Athey–Tibshirani–Wager 2019). `VC` brings that ingredient into the TRAM framework.

Semantics

- **Scale**: `beta(x)` lives on the node's latent (log-odds) scale. The flow adds it for a continuous node (`u = h(y) + …`) and subtracts it from the cutpoints for an ordinal node, exactly like an `LS` weight. With no modifiers, `VC(t="T")` *is* `LS("T")` (identical model, testably bit-exact). Thus `VC` versus `LS` is a nested question.

- **Penalty**: the fitting objective is the penalized likelihood $\sum_i \mathrm{NLL}_i + \lambda \|b_\Theta\|^2$ (`penalty=` is λ) on the total-NLL scale. This is a fixed Gaussian prior whose shrinkage vanishes as n grows. The penalty never applies to `beta0`. `penalty → ∞` shrinks `b_theta` to the zero function and recovers the classical `LS` fit. The default is `penalty=1.0`. If the modifiers are many or n is small, raise the penalty.
- **Identification / centering**: a constant moves freely between `beta0` and `b_theta`. The head's output layer is zero-initialized (`beta(x) = beta0` at step 0). After `fit`, the flow re-centers the head to mean zero over the training data, which preserves the function. Thus `beta0` is the main effect in the training population, the `Colr` reading when `beta` is constant.
- **Warm start** (optional, two lines): fit the all-`ls` version of the node classically and copy the treatment weight into `flow.nodes[node].shifts[t].beta0` before `fit`, so training starts at the classical answer and only learns deviations. Measured on the `vc_hetero` DGP: `beta0` lands within 0.15 of the truth with it, and 0.16 from the zero start. The recovery correlation is 0.99 either way.
- **Treatments**: the treatment `x_on` can be continuous or binary (2-level) ordinal. The term is linear in `x_on`. A binary ordinal enters as its 0/1 level, so `beta` is the identified level-1-vs-0 contrast. Multi-level ordinal treatments are a planned follow-up.
- **Read-out**: `flow.varying_coef(df, node, t=...)` evaluates `beta0 + b_theta(modifiers)` in closed form. The result is deterministic and y-free, and it needs no abduction. For a binary treatment, it equals the abduct-difference `u(x, t=1, y) − u(x, t=0, y)` identically (pinned by a test).

Propensity-centered VC: `center="col"` (R-learner orthogonalization)

```
td.VC("X2", "X3", t="T", penalty=1.0, center="ps")
# contributes  beta(x) * (t - e_hat(x))  to the shift; e_hat comes from you,
# out of fold, as the training-frame column named by center=
```

This is Robinson/R-learner centering inside the likelihood. Dandl et al. (2024) found treatment-centering to be *the* decisive ingredient for effect estimation under confounding in model-based forests. That finding reproduces here. The test DGP is strongly confounded, and the model deliberately under-specifies its prognostic part (true g(x) quadratic, model linear). On that DGP, the uncentered β absorbs the confounded misfit. The mean $|\beta - \tau|$ is ≈ 1.1–1.2, which effectively destroys the effect estimate. The centered β stays near truth (≈ 0.1–0.3), a **5–10× bias reduction** (`tests/test_vc_centered.py`). If the prognostic part is correctly specified, centering changes little. Centering is insurance against the misspecification that you do not know you have.

The naive implementations are wrong. Thus this is a **two-stage frozen** design:

- **Training** uses **out-of-fold** ê that *you* compute. Pass it as the frame column `VC(center=)` names (`df.assign(ps=e_oof)`), one value per training row. Any propensity model works, with each fold predicted by a fit that never saw it. This is the DML cross-fitting requirement. In-sample ê reintroduces the own-observation bias and can be *worse* than no centering. Six lines with the flow's own classical fit:

  ```
  fold_id = np.random.default_rng(0).permutation(len(train)) % 5
  e_oof = np.empty(len(train))
  for j in range(5):
      proxy = td.CausalFlowDAG(t_spec, seed=0)          # the treatment node's spec
      proxy.fit_classical(train.ilocfold_id != j])
      e_oof[fold_id == j] = proxy.pmf(train.iloc[fold_id == j], "T")[:, 1]
  ```

  The OOF values enter the outcome loss as frozen data. Thus **no gradient reaches the treatment node** from the outcome node, and the per-node factorization stays intact (pinned by a gradient-isolation test). `fit` refuses a centered spec whose frame lacks the column, and one that does not match the spec.

- **Inference** (`log_prob` / `sample` / `abduct` / `pmf` / `scores`) recomputes ê from the flow's **own fitted**

treatment node, the full-data fit (the standard DML train/predict split). The computation is detached and uses the current parent values. Under `do(T=t)`, the flow re-derives the regressor as `t − ê(x)`. The flow caches nothing.

- **Interpretation**: with centering, `beta0` is the effect at the treatment margin (the observed propensities). `varying_coef` is unchanged, because centering moves the regressor, not β. The LS-nesting reading applies to the uncentered term only. Centering requires a binary ordinal treatment. Continuous-treatment centering with E[T|x] is a follow-up. `center=False` (the default) is bit-identical to the uncentered term.

Validation

The `vc_hetero` DGP in `tests/conftest.py` is a logistic-shift SCM with known `beta_true(x) = −1 + 0.8·X2 − 0.6·X3`. It has a nonlinear prognostic part and confounded assignment (X2 is confounder *and* modifier), which is the configuration where the `CS` reduced form fails hardest. Acceptance (`tests/test_vc_term.py`) requires three results: recovery corr ≥ 0.9 at n = 5000 (measured ≈ 0.99, min over 3 seeds 0.986), a fitted `beta0` that matches `fit_classical` under a large penalty, and the read-out identities. The centering claims are measured against the `confounded` DGP in the same file. Both former follow-ups have since shipped: propensity centering (#30) is `center="col"`, documented above, and the per-observation scores for effect-modifier scans (#29) are `flow.scores` and `flow.effect_modifier_scan` — see scores.md.

Paper replication — protocol, hyperparameters and results, per experiment

The eight variants under `experiments/paper/` replicate the TRAM-DAG paper (Sick & Dürr, CLeaR 2025, arXiv:2503.16206) against its own R code (`tensorchiefs/tram-dag`). This page lists these items for each experiment:

- the DGP,
- the model,
- every hyperparameter with its source,
- what deviates from the paper, and why,
- the numbers.

The numbers come from three sources:

- what the paper reports,
- what the previous protocol of this repository measured,
- what the current 1:1 protocol measures.

The measurement date is 2026-08-26. The branch is `refactor/lean-fit`. The seeds are DGP 42, init 7 and shuffle 0. The protocol is the exact reference protocol, which has two parts:

- the global plateau rule,
- the Keras init.

Three changes followed that measurement. On 2026-09-01 the repository re-tuned the *optimization* for CI runtime. That round used fewer epochs and scaled the lr to match. It left the data, the model and the init untouched.

On **2026-09-02 VACA went back to the reference protocol 1:1** (10000 @ 0.001 + plateau). The change followed a visual pass against paper Fig. 5. Every bound holds under it. The do(x2=0) error improves.

On **2026-09-03 CAREFL became the reference run 1:1**. It now trains on CAREFL's own committed 2500 rows (`data/carefl-cf`, sd-standardized units). It uses these settings:

- `val = train`,

- 7000 @ 0.001 + plateau,
- the min-max Bernstein domain (`range_q: 0`).

These settings put the Fig. 6 curves on the paper's. The earlier 3000 @ 0.002 trade-off was an artifact of the fresh-draw data. The triangle cuts stay.

The 2026-09-01 tuning round below holds the tuning campaign, with what the round tried and what failed.

The "R 1:1, torch init" column keeps the 2026-08-25 numbers of `feat/followups` where they differ. That branch used the per-node plateau and the torch init. Its CI run is 32878435001.

"Previous" is the protocol before that branch. It has these parts:

- a 90/10 split of one draw,
- batch 512,
- the fit restarted in chunks (`fit_in_chunks`),
- for VACA/CAREFL, a second "polish" fit at a lower rate.

That protocol is a repository recipe, not the paper's. Its numbers come from the ground-truth files of that revision (`73f9f39`). The `{max}` bounds were 2.5× the measurement.

The paper states four training numbers:

- n = 40000,
- 500 epochs,
- Adam,
- Bernstein order 20.

The paper shows its results as figures. If the paper gives no number, then the "paper" column names the figure and what the figure shows.

What is common to all eight

item	reference	here
latent	standard logistic, shifts added on the continuous scale, subtracted for ordinal `P(Y<=k) = sigmoid(theta_k - shift)`	same (tests pin both signs)
Bernstein basis	`len_theta` unconstrained coefficients, `to_theta` softplus-cumsum, domain → [0, 1] with tangent-linear extrapolation outside; the domain comes from the train **5 %/95 % quantiles in the triangle scripts** (`quantile(..., c(0.05, 0.95))`, the min/max lines commented out) and from **min/max** in the comparison scripts (`scale_df`)	zuko Bernstein, `n_coeffs` unconstrained (zuko ties two control points on, so the free-parameter count matches), domain = train `range_q`/1−`range_q` quantiles → [−5, 5], linear extrapolation; `range_q` is an intercept option since 2026-09-03, default 0.05. Triangle: a match up to the reparametrization [0,1] vs [−5,5], the tail rule and order 21 vs 19. CAREFL: `range_q: 0` = the reference's min/max, a match. VACA: quantiles **kept deliberately — deviation D1**, measured (see VACA)

item	reference	here
init	triangle scripts: `LinearMasked` layers with Keras `random_normal` (N(0, 0.05^2)) on weights and biases, the LS `beta` layer included; comparison scripts: `layer_dense` default, glorot-uniform weights and zero biases	`init: normal` (triangle) and `init: glorot` (VACA/CAREFL), `CausalFlowDAG(init=)`; torch's default init remains the framework default and was the decisive deviation under the full-batch protocol, see VACA
optimizer	Keras Adam, eps 1e-7	torch Adam, eps 1e-8 — measured: no effect (VACA identical to four digits)
seeds	triangle scripts unseeded (`SEED = -1`); comparison scripts `dgp(..., seed=42)` on R's RNG	not replayable in torch, so every seed is a repo choice (42 for the DGP is kept as a nod to the reference)
calibrated start	none	none — `calibrate` never touches the weights; `init_marginals` is a separate explicit step no paper script calls
intercept output layer	Keras dense with bias	bias-free — **D3** (the same function class; the bias adds a constant to all unconstrained coefficients)
plateau rule (VACA/CAREFL)	`update_learning_rate`: one optimizer, reduce when the summed validation NLL has not improved for 50 epochs (strict <), factor 0.1, min 1e-7	torch `ReduceLROnPlateau(patience=49, threshold=0, threshold_mode="abs", factor=0.1, min_lr=1e-7)` on the summed `history["val"]` entry (fit computes it) — the same rule, verified against torch's source; both keep it since 2026-09-02 (VACA's 2026-09-01 drop went with the reverted epoch cut)

D1 (VACA only since 2026-09-03) and D3 remain. The sections below measure both. The per-node plateau approximation of 2026-08-25 (D4) is gone with the lean `fit`.

CAREFL had two former repository choices: a fresh 2500-row draw with a separate validation draw, in raw units. Both are gone the same day. The benchmark now trains on CAREFL's own committed rows with `val = train`, in the reference's sd-standardized units. It does exactly what `carefl_fig5.r` does.

Triangle, continuous (`triangle.py`) — paper Sec. 6.1, App. C.3

DGP (`summerof24/triangle_structured_continous.R`):

- x1 ~ 0.5 N(0.25, 0.1^2) + 0.5 N(0.73, 0.05^2)
- h(x2 | x1) = 5 x2 + 2 x1 = u2
- h(x3 | x1, x2) = 0.63 x3 − 0.2 x1 − f(x2) = u3, u ~ logistic

f has three forms:

- `linear` −0.3 x
- `atan` 0.75 atan(5 (x + 0.12))
- `sin` 2 sin(3x) + x

True weights in the flow's convention: β12 = +2, β13 = −0.2, β23 = +0.3 (linear only).

Model: `x1: SI`, `x2: SI + LS(x1)`, `x3: SI + LS(x1) + {LS(x2) | CS(x2)}`, Bernstein. The CS net is the reference `hidden_features_CS = c(2, 25, 25, 2)` with sigmoid. The ReLU line in `create_param_net` is commented out.

Corrected 2026-09-02: the vector reads as in/out dims around the hidden stack **(25, 25)**. The earlier literal `[2, 25, 25, 2]` reading put a 2-sigmoid bottleneck on the input. That reading cannot reproduce paper Fig. 18 (sin) at any protocol. The hidden stack (25, 25) lands on Figs. 17/18. The linear cs err moves 0.13 → 0.025. The sin err moves 1.22 → 0.24 on the paper's [−1, 0] window.

The mixed variant reads the same way: c(2, 2, 2, 2) → hidden (2, 2).

hyperparameter	paper / R code	previous	now
train / validation	`dgp(40000)` / `dgp(40000)`, two draws	36000 / 4000, one draw split	40000 / 40000, two draws
epochs	500	500 (in chunks of 10, Adam restarted each chunk)	**300** (`linear-cs`: 500), one continuous run — the CI deviation, floors below
lr	0.001 (`optimizer_adam()` default)	0.001	**0.004**, the CI deviation
batch	32 (Keras `fit()` default)	512	**256**, the CI deviation
`len_theta` / `n_coeffs`	20	20	20
schedule / early stop / init	none / none, final weights / random_normal	none / none / torch	none / none / `init: normal`
coefficient read-out	after every epoch (Keras loop)	at chunk boundaries	`fit(callbacks=)`, every epoch

Results

variant	metric	paper	previous	paper protocol, `init: normal` (batch 32 / lr 0.001 / 500 epochs)	CI config (batch 256 / lr 0.004, 300 epochs — `linear-cs` 500), the pinned ground truth
linear-ls	β12 / β13 / β23	Fig. 14 trajectories; App. C.3 text: 1.98 / −0.21 / 0.26	1.983 / −0.145 / 0.285	1.987 / −0.170 / 0.282	1.981 / −0.161 / 0.281
linear-cs	β13; max $\lvert\hat{g} - (-f)\rvert$ on [−1, 1]	Fig. 17: fitted CS is a straight line	−0.151; 0.507	−0.173; 0.122 (glorot init: 0.110, torch: 0.116)	−0.178; 0.088
atan-cs	β13; cs max err	Fig. 7 right / 15 / 16: CS on −f; text: β12 = 2.07, β13 = −0.203	−0.149; 0.229	−0.168; 0.059 (glorot: 0.080, torch: 0.085)	−0.168; 0.108 (hidden (25,25), 2026-09-02)

variant	metric	paper	previous	paper protocol, `init: normal` (batch 32 / lr 0.001 / 500 epochs)	CI config (batch 256 / lr 0.004, 300 epochs — `linear-cs` 500), the pinned ground truth
sin-cs	β13; cs max err	Fig. 18: CS follows the non-monotone f on x2 ∈ [−1, 0]	−0.185; 2.29	−0.195; 1.10 (glorot: 0.997, torch: 1.016)	−0.168; 0.240 on the paper's [−1, 0] window (hidden (25,25), 2026-09-02)
all	|E[x3 | do(x1 = −1)] flow − DGP|	Figs. 16/17: histograms overlap	—	0.09–0.14	0.08–0.11
all	val NLL x3	—	2.4603–2.4731	2.4607–2.4764	2.4606–2.4764

β13 measures −0.16 to −0.18 instead of −0.2. x1 has sd 0.254, so SE(β13) ≈ 0.036 at n = 40000. The value therefore stays within ~1 SE.

The old `sin-cs` error of ~1.0 came from the misread 2-unit-bottleneck net, not from "reference capacity". This page retracted that claim on 2026-09-02. With hidden (25, 25) the fit lands on Fig. 18, including its small −1-endpoint deviation. The paper's figure never plots beyond [−1, 0].

Triangle, mixed (`triangle_mixed.py`) — paper Sec. 6.2, App. C.4, App. B

DGP (`triangle_structured_mixed.R`): x1 and x2 are as above. x3 is ordinal with four levels. Its cutpoints are θ = (−2, 0.42, 1.02), which come from the R code. The paper does not state them. The level is #{k : u3 > θ_k + 0.2 x1 + f(x2)}. f has two forms: `linear` −0.3 x and `exp` 0.5 exp(x).

The paper adds the ordinal shift. The flow subtracts it. The fitted weights are therefore β13 = −0.2, β23 = +0.3.

Model: `x1: SI`, `x2: SI + LS(x1)`, `x3: OrdinalNode(4, LS(x1) + {LS(x2) | CS(x2)})`. The CS net is the reference `c(2, 2, 2, 2)` with sigmoid.

hyperparameter	paper / R code	previous	now
train / validation	`dgp(40000)` / `dgp(10000)`	36000 / 4000 split	40000 / 10000, two draws
epochs, lr, batch, schedule, init	500, 0.001, 32, none, random_normal	500 (chunks of 10), 0.001, 512, torch	**350 (`exp-cs`) / 200 (`linear-ls`)** continuous, none, `init: normal`; **lr 0.004 (`linear-ls`: 0.002) / batch 256** — the CI deviations, floors below
`n_coeffs` (x1, x2)	20	20	20

hyperparameter	paper / R code	previous	now
odds-ratio check (App. C.4)	odds(x2 ≤ −1) under do(x1 += 1), theory $e^2 \approx 7.39$	40000 rows, seed 99	same
counterfactual PMF (App. B)	Fig. 10 explains why point counterfactuals fail for the ordinal node	2000 rows × 200 draws	same

Results

variant	metric	paper	previous	paper protocol, `init: normal` (500 epochs)	CI config (batch 256; `exp-cs` 350 epochs / lr 0.004, `linear-ls` 200 / 0.002), pinned
linear-ls	β12 / β13 / β23	Fig. 19: 2 / −0.2 / 0.3	1.983 / −0.268 / 0.322	1.987 / −0.246 / 0.319	1.978 / −0.258 / 0.333
linear-ls	odds ratio predicted / DGP	$e^2 \approx 7.39$; C.4 text: 7.74, CI [7.16, 8.38]	7.26 / 7.19	7.29 / 7.19	7.23 / 7.19
linear-ls	CF PMF TV vs analytic; P(true level) flow / analytic / mode bound	— (App. B qualitative)	0.084; 0.714 / 0.728 / 0.806	0.044; 0.718 / 0.728 / 0.806	0.050; 0.716 / 0.728 / 0.806
exp-cs	β13; cs max err	Fig. 20: distributions match	−0.227; 0.885	−0.207; 0.142 (glorot: 0.071, torch: 0.126)	−0.200; 0.156
exp-cs	CF PMF TV; P(true level)	—	0.075; 0.924 / 0.921	0.023; 0.917 / 0.921	0.024; 0.920 / 0.921

VACA / CNF benchmark (`vaca.py`) — paper Sec. 5.1–5.2, App. C.1

DGP (Sanchez-Martin et al. 2022, App. E.1):

- x1 ~ 0.5 N(−2, 1.5) + 0.5 N(1.5, 1)
- x2 = −x1 + N(0, 1)
- x3 = x1 + 0.25 x2 + N(0, 1)

The noise is Gaussian, which is outside the logistic-latent family. The all-`CI` flow must fit it. The analytic target is E[x3 | do(x2 = a)] = −0.25 + 0.25 a.

The paper's Sec. 5.2 text says a ∈ {−3, −2, 0}. Its Fig. 5 panels and the R code (`vaca_triangle.r`) intervene at a ∈ {−3, −1, 0}. This repository follows the code. The frozen `data/vaca/truth.json` holds the same three values.

Model:

- `x1: SI`, `x2: CI(x1)`, `x3: CI(x1, x2)`

- Bernstein `n_coeffs = 31` (reference M = 30, `len_theta = 31`)
- nets `dense(10, tanh) → dense(100, tanh) → dense(31)` (`comparison/utils.R::make_model`)

hyperparameter	R code	previous	now
train / validation	nTrain 2500 / `dgp(5000)`	18000 / 2000 split	2500 / 5000, two draws
epochs	10000 (`Figure_Triangle_Linear.ipynb`, the sourcing script — not in our copy of the R code, so EPOCHS/M/nTrain for VACA rest on that reading)	400 in chunks of 50, then 120 "polish"	**10000**, one run — the reference, 1:1 (the 2026-09-01 4800-epoch cut was reverted 2026-09-02)
lr	0.001	0.01, polish 0.001	**0.001** — the reference
batch	full batch (one `apply_gradients` per epoch)	512	2500 = n_train
schedule	`update_learning_rate`: ReduceLROnPlateau on the summed val NLL, factor 0.1, patience 50, min_lr 1e-7, strict <	none	**the same rule** (restored 2026-09-02 with the reference epochs; it fires ~epoch 9050 and freezes an all-bounds point)
input scaling	`scale_df`: everything min-max to [0, 1]	raw	`input_transform: minmax` on the CI terms (network inputs; targets D1)
Bernstein domain	train min/max (`scale_df`)	5%/95% quantiles	quantiles kept (`range_q: 0.05`) — **D1, now measured under the final protocol** (2026-09-03, seed 7): the reference's min/max domain scores 0.289 / 0.040 / 0.067 against the quantiles' 0.096 / 0.080 / 0.022 — worse at the off-manifold do(x2 = −3) and at do(x2 = 0). CAREFL, same nets, measures the opposite way and matches the reference (`range_q: 0`)
n_compare	—	50000	50000

Results — the check is the flow's error against the analytic mean. It is not a pinned flow value.

metric	paper	previous	R 1:1, torch init, per-node plateau (08-25)	R 1:1, glorot, global plateau (08-26)	now: the reference 10000 @ 0.001 + plateau — the pinned ground truth
|E[x3 | do(x2 = −3)] − (−1.0)|	Fig. 5: densities overlap	0.037	0.026	0.097	0.096
|E[x3 | do(x2 = −1)] − (−0.5)|	Fig. 5	0.012	— (do(−2) was scored: 0.034)	0.088	0.080
|E[x3 | do(x2 = 0)] − (−0.25)|	Fig. 5	0.010	0.077	0.019	0.022
sd(x1) flow vs analytic 2.0767	Fig. 4: bimodal x1 fitted (the default CNF fails)	2.074	error 0.0077	error 0.032	2.036, error 0.040
val NLL x3	—	1.4356	1.4351	1.4496	1.4427

The 08-25 and 08-26 columns are not the same three interventions. The do(x2) set moved from the paper text's {−3, −2, 0} to the R code's {−3, −1, 0} (see the DGP paragraph above). Only do(x2 = −3) and do(x2 = 0) compare directly.

At the earlier grid and with glorot the errors were 0.098 / 0.159 / 0.026 at this seed. At init seeds 8 and 9 they were 0.268 / 0.119 / 0.005 and 0.217 / 0.031 / 0.021. That is the spread quoted below.

The table below traces the gap, one change at a time, under the exact protocol. That protocol is 10000 epochs at lr 0.001 with the plateau rule, before the 2026-09-01 epoch cut. The inputs are min-max scaled unless the row states otherwise. The do(x2) errors are at the then-current grid −3 / −2 / 0.

variant	error	reading
raw parents into the nets	0.731 / 0.426 / 0.017	the tanh nets saturate: 40 % of the rows have |x1| > 2, 43 % |x2| > 2
per-node plateau, torch init	0.026 / 0.034 / 0.077	the 2026-08-25 approximation
global plateau (R's rule), torch init	0.523 / 0.334 / 0.129	the summed NLL keeps improving through x1 while x3 overfits
… without any plateau	0.562 / 0.354 / 0.142	the rule is what stops the full-batch overfit
… Adam eps 1e-7	0.523 / 0.334 / 0.129	no effect
… Bernstein on train min/max (D1 off)	0.154 / 0.012 / 0.062	helps, not the cause
… glorot init	0.035 / 0.006 / 0.007	the cause (a different random draw than the config's seed 7)

variant	error	reading
… glorot + min/max	0.035 / 0.056 / 0.057	

Under the exact protocol the result is **seed-sensitive at the off-manifold point** do(x2 = −3). The error spans 0.03–0.27 over four init draws. At do(x2 = 0) it spans 0.005–0.026. Both spans come from the 10000-epoch protocol. The paper shows one run's densities.

The committed bound is 2.5× the seed-7 measurement of the reference 10000-epoch run. The repository reverted the 2026-09-01 4800-epoch cut on 2026-09-02. This table is the honest picture.

CAREFL benchmark (`carefl.py`) — paper Sec. 5.3, App. C.2

Since 2026-09-03 this benchmark is the reference run 1:1, on the reference's own data. `carefl_fig5.r` sets `USE_EXTERNAL_DATA = TRUE`. It trains on CAREFL's own committed 2500 rows (`data/CAREFL_CF/X.csv`, x3/x4 sd-standardized by 6.0104/1.9114) with **`val = train`**. Its repository also commits three more things:

- the observation (`xObs.csv`),
- the analytic truth curves,
- CAREFL's own predictions on the grid `seq(-3, 2.9, 0.1)`.

This repository freezes all of it under `experiments/paper/data/carefl-cf`. That folder is external input with no generator — see its `truth.json`. The Fig. 6 curves are therefore comparable point by point. Every metric is in the reference's standardized units.

DGP (Khemakhem et al. 2021):

- x1, x2 ~ Laplace(0, 1/√2)
- x3 = x1 + 0.5 x2³ + ε
- x4 = −x2 + 0.5 x1² + ε, ε ~ Laplace(0, 1/√2)

The DGP divides x3/x4 by their sample sds. Counterfactuals are analytic by noise abduction.

The observation `xObs.csv` = (2, 1.5, 0.8465, −0.2616). The paper prints (2, 1.5, 0.81, −0.28), which is slightly off it (CAREFL's own text). **Before `feat/followups` the repository used the printed values as raw coordinates**, a 4σ-off point. The queries are x3^cf | do(x2 = α) and x4^cf | do(x1 = α) on the committed grid.

Model: `x1, x2: SI`, `x3, x4: CI(x1, x2)`, Bernstein `n_coeffs = 31`, the same `make_model` nets as VACA. `range_q: 0` is the reference's `scale_df` min-max Bernstein domain. It is an intercept option since 2026-09-03.

hyperparameter	R code (`carefl_fig5.r`)	2026-09-02 era	now
train / validation	CAREFL's own `X.csv`, sd-standardized, `val = train`	2500 fresh SCM rows / a separate `dgp(5000)` draw, raw units	**the same committed `X.csv`, `val = train`** (a fresh standardized draw is scored, never trained or annealed on)
epochs, lr	7000 @ 0.001	3000 @ 0.002	**7000 @ 0.001**
batch, schedule, input scaling, init	full batch, plateau 0.1/50/1e-7, `scale_df`, glorot	same	same

hyperparameter	R code (`carefl_fig5.r`)	2026-09-02 era	now
Bernstein domain	train min/max (`scale_df`)	5%/95% quantiles (D1)	**train min/max** (`range_q: 0`)
scoring	the single `x_obs`, curves over α (Fig. 6)	+ 300 held-out rows at α ∈ {−1.5, 0, 1.5}	same, all in standardized units

Results, in standardized units. The 09-02-era numbers were raw. To compare them, divide x3 by 6.01 and x4 by 1.91.

metric	paper / CAREFL	09-02 era (fresh draw, 3000 @ 0.002, quantile domain), rescaled	now — the pinned ground truth
Fig. 6 max |x3^cf error|	Fig. 6: flow tracks the DGP curve; CAREFL's committed x3 preds err up to ~0.7 at α = −3	0.45 (at α = 3)	**0.066**
Fig. 6 max |x4^cf error|	CAREFL's committed x4 preds: max 0.174	0.40; ~0.29 near α = 0 (the "dip")	**0.204**, at the α = −3 grid edge; 0.074 at α = 0
held-out CF MAE x3, α = −1.5 / 0 / 1.5	—	0.035 / 0.010 / 0.021	0.015 / 0.009 / 0.013
held-out CF MAE x4, α = −1.5 / 0 / 1.5	—	0.065 / 0.051 / 0.064	0.080 / 0.027 / 0.048

Three components closed the gap. The measurements are from 2026-09-03, each one with everything else held fixed:

- The fresh data draw → `X.csv` cut the Fig. 6 x4 max from 0.77 (raw ≈ 0.40 std) to 0.34 std.
- The reference optimization (7000 @ 0.001 with `val = train`) fixed the parabola bottom (err at α = 0 0.21 → 0.06).
- The min-max domain cut the grid edges (x4 0.37 → 0.20, x3 0.20 → 0.07).

The earlier 3000-vs-7000 trade-off was an artifact of the fresh-draw data. In that trade-off 3000 won the held-out MAEs and 7000 won the single-point curve. The reference's rows settle it.

The former Fig. 6 x4 "dip": the repository root-caused it on 2026-09-03, before it adopted the data. On fresh 2500-row draws the flow's parabola bottom sat well below the truth near α = 0. The cause is finite-sample tail-quantile variance, not the protocol and not the init. Three things left it invariant:

- 3000 vs 7000 epochs,
- the val choice,
- init seeds 7/8/9, at 0.586/0.601/0.610 raw.

The observed noise sits at the Laplace 12% quantile. That quantile's fitted value errs +0.27 in the sparse observation region x1 = 2, which holds ~190 nearby rows. In the dense middle it errs −0.19. The magnitude moved with the data draw: 0.54/0.31/0.43 raw at dgp seeds 42/43/44.

CAREFL's own committed draw is a mild one. That is one more reason why training on the committed rows is the right comparison.

Runtime and the epochs

The reference protocol's cost is the motivation for every deviation here. The triangle runs 500 epochs × 1250 steps of batch 32. One epoch takes 3.2 s single-process at 2–4 torch threads. At 32 threads it takes 5.5 s, because the step is overhead-bound. One variant takes 42–69 min on the 2-core CI runners. VACA takes 355 s and CAREFL takes 338 s (2026-08-31).

The workflow's ten jobs run in parallel. They take 70 min wall against a 300-min cap.

The first deviation for CI runtime (2026-08-26) was the triangle **batch size and learning rate**. It uses batch 256 at lr 0.004 instead of the paper's batch 32 at lr 0.001. That is 8× fewer optimizer steps per epoch.

The table below is the earlier selection grid. It stays here as the record of that choice. It ran at the paper's 500 epochs with glorot init, against the then-committed ground-truth bands. Every entry is the cs max err unless the column says otherwise.

batch / lr / epochs	linear-ls β13	linear-cs	atan-cs	mixed exp-cs (TV)	steps vs paper
32 / 0.001 / 500 (paper)	−0.170	0.110	0.080	0.071 (0.019)	1×
128 / 0.001 / 500	−0.170	0.170	0.041	0.126 (0.023)	1/4
128 / 0.004 / 500	−0.178	0.160	0.094	0.070 (0.014)	1/4
128 / 0.004 / 250	−0.170	0.147	0.070	0.131 (0.035)	1/8
256 / 0.004 / 500 (CI)	−0.171	0.132	0.073	0.072 (0.017)	1/8
256 / 0.008 / 500	−0.181	0.163	0.113	0.073 (0.017)	1/8
512 / 0.010 / 500	−0.169	0.183	0.104	0.119 (0.015)	1/16

Batch 256 / lr 0.004 is the only row that keeps every cs error within ~0.02 of the paper protocol (linear-cs: 0.022). Larger batches or rates bend the misspecified linear-cs curve (0.16–0.18). lr 0.008 hurts atan-cs.

At 500 epochs and this batch/lr, CI run 32974872751 took 7–11 min per triangle job instead of 42–69. The workflow took about 12 min wall instead of 70.

The grid ran with glorot init. The committed configs use `init: normal`, the triangle scripts' initializer. The paper-protocol numbers stay in this document as the reference. The repository pins the ground truth from the CI config, which since 2026-09-01 also cuts the epochs. The round below covers that cut.

The 2026-09-01 tuning round

The frame is a repository decision of 2026-09-01. The *data, model and init* stay the reference's. The round allowed three changes:

- dropping the network-input scaling,
- trying sigmoid/relu activations,
- tuning lr / batch size / epochs for CI runtime.

The outcome has two parts. The triangle scripts already ran raw parents + sigmoid, as their reference does, so they comply by construction and only their epochs moved. VACA and CAREFL

keep tanh + min-max network inputs (`input_transform: minmax`), because the round tried every alternative and each one measurably fails.

The repository re-pinned every tuned variant's ground truth from its final run. The centers follow the run, and `{max}` sits at 2.5x. `fit_seconds` waits for the next CI measurement. **The bounds and pins quoted in this section are the pre-repin 2026-08-26 values the round measured against.** Re-pinning loosened several of them. Each config's YAML header holds the full decision trail.

experiment	was	now
triangle `linear-ls` / `atan-cs` / `sin-cs`	500 epochs	300 epochs
triangle `linear-cs`	500 epochs	500 epochs (kept)
triangle-mixed `exp-cs`	500 epochs	350 epochs
triangle-mixed `linear-ls`	500 epochs @ lr 0.004	200 epochs @ lr 0.002
vaca	10000 full-batch epochs @ lr 0.001, plateau	reverted 2026-09-02: back to the reference 10000 @ 0.001 + plateau
carefl	7000 full-batch epochs @ lr 0.001, plateau	reverted 2026-09-03: back to the reference 7000 @ 0.001 + plateau, on the reference's own committed rows (see the CAREFL section)
validate_ls `adam` (misc)	phases 4000/2000/1000 @ 1e-2/1e-3/1e-4	800/700/500, batch 256 kept

Triangle (batch 256 / lr 0.004 / sigmoid / `init: normal` / raw parents all unchanged): at 300 epochs every metric holds its bound. The floor is real. At 150 epochs the run fails atan's do(x1) bound (0.235 > 0.206).

`linear-cs` keeps 500 epochs. At 300 epochs its fitted cs curve visibly flattens past |x2| > 0.5, with max err 0.140 vs 0.088. The DGP line spans only ±0.3, and the sigmoid net's slow tail convergence is half of that.

relu is out. The reference's 2-unit input bottleneck dies with it (atan: β13 +0.53, cs err 1.42).

Triangle-mixed: the cs variant has two slow parts, the 2-unit sigmoid CS net and the do(x1) mean. At 100 / 250 / 350 epochs the cs max err is 0.344 / 0.177 / 0.156. The do err at those epochs is 0.033 / 0.060 / 0.022. At 500 epochs they are 0.107 / 0.016. So 100 epochs failed the then-current cs bound (0.344 > 0.2686), and 250 epochs failed the do bound (0.060 > 0.0398). `exp-cs` trains 350 epochs.

`linear-ls` has no CS net to wait for and trains 200 epochs at lr 0.002. That rate reads the weakly identified β13/β23 out slightly closer to the 500-epoch pins than lr 0.004 does (β23 0.333 vs 0.337, pin 0.306). At 100 epochs β13 sits 57% into its tolerance (−0.271 vs the pinned −0.243). relu dies at the sd-0.05 normal init (cs err 0.859, β13 −0.055).

VACA — the instrumented finding: a probe of the reference trajectory (10000 @ 0.001, plateau) shows the do(x2) errors descend until epoch ~6000–8500. The best values in that band are 0.005–0.10. They then creep back up. The plateau anneal fires at ~9050 and freezes 0.101 / 0.099 / 0.030. Those numbers come from an instrumented re-run. The small offset from the 08-26 pins 0.097 / 0.088 / 0.019 is run-to-run sampling of the do-means.

The reference protocol's quality IS its anneal landing inside that window.

At lr 0.002 the same smooth full-batch descent runs 2× compressed, with an all-bounds window at epochs ~4350–5000. Two edges bind that window:

- x1's marginal std enters its 2.0766 ± 0.08 band at ~4320,
- the do(x2=+0) error exits after ~5000.

4800 epochs sit inside the window, with every margin ≥ 2×, at under half the reference's wall clock.

The round drops the plateau rule at that rate. On the compressed trajectory the summed validation NLL still improves strictly past the window, so the rule cannot fire inside the window. lr 0.004 is out, because x1's std clock does not compress with lr (1.978 at its do-window, epochs ~2050–2350).

The round rejected two more variants and measured both. relu + raw parents wanders 0.17–0.45 on do(x2=−3) and holds the bound only at isolated epochs. tanh on raw parents saturates outright, at 0.731. Minibatch stepping (batch 256, even with minmax + tanh) converges with a systematic −0.11 common-mode offset of all three do-means. That offset is 2.4× the do(x2=+0) bound. The observational fit meanwhile stays perfect.

CAREFL (full batch + plateau kept): the round's 3000 @ 0.002 pick was an artifact of the then-current fresh-draw data. The move to the reference's own committed rows reverted it on 2026-09-03. See the CAREFL section: the 3000-vs-7000 trade-off does not exist on `X.csv`.

Two rejections still stand from the round. The first one is raw parents. sigmoid saturates on the Laplace parents in the same way as tanh. relu underfits x3 (val NLL 1.46–1.47 vs the 1.403 ± 0.05 band). relu also grows a fragile Bernstein tail. One held-out row inverted to x4 ≈ 580, with cf MAE 4.36 vs bound 0.33.

The second rejection is minibatch. The fig6 grid ends live in the sparse x2 tail, which minibatch gradients underweight. The cf MAE x3 do(x2=+1.5) is 0.40, over the old 0.355 bound and the re-pinned 0.319 bound alike. The fig6 x3 err is 5.9 against the full-batch run's 2.7.

validate_ls `adam` is in misc. It is not a paper experiment, but the same round tuned it. Its phases are 800/700/500 instead of 4000/2000/1000, which is 2000 epochs instead of 7000. It still lands on the classical MLE. Its named-coef gap to statsmodels is 1.6e-5 against the old 1.8e-5. It uses ~3.5× less fit wall clock.

Local wall times under the tuned protocol (32-core box, 4 torch threads):

- triangle 162–281 s per variant,
- mixed 88–165 s,
- vaca 46.5 s,
- carefl 72–82 s.

The CI timings quoted above and the `fit_seconds` pins in `ground_truth/` are still the 2026-08-31 pre-cut measurements. The next push will re-measure both and update the pins.

Repository choices the paper does not state

The paper does not state these choices:

- the seeds,
- the validation draw sizes for the triangle scripts (the R code's `test`),
- `n_compare`,
- `n_heldout = 300` and the α set {−1.5, 0, 1.5}, scored next to the single `x_obs`,
- the mixed cutpoints from the R code,
- the odds-ratio sample (40000 rows),
- the counterfactual-PMF sample (2000 rows × 200 draws).

Code map — every module of `src/tramdag/` — the public names and the private plumbing

This document has one entry per name, with its role and its place in the pipeline. Names in parentheses are private machinery. They are useful to know, and they are not part of the API.

The last section lists every training hyperparameter and where it lives.

`spec.py` — declare the model

Every term is a `Term` subclass under its pythonic name. The paper's symbol is the same object, so `LS is LinearShift`.

Name	Role
`Term`	One additive term of a node's transformation, frozen data; each term is a subclass whose class name is its `name` and whose annotated attributes are its options (defaults dropped on serialization, so equality is canonical). `+` on terms builds plain lists. Carries the spec-level rules: `edge_parents`, `cells`, `classical`, `options()`, `from_serialized`. Subclass `Term` for a new term; its module goes in `terms.py` and read as attributes (`term.penalty`, `term.units`, …).
`SI()`	The parentless intercept — the paper's SI. Free transform parameters, the same for every row. Carries the transform choice (`transform=`, default `"bernstein"`); extra keyword arguments pass straight to the transform class.
`CI()`	The parent-conditioned intercept — the paper's CI: the parents reshape the monotone transform. Needs at least one parent. Also carries `units=` and `allow_interaction=` (joint vs. additive multi-parent intercept).
`Intercept` / `I`	The intercept term class: without parents the paper's SI, with parents the CI; `SI()`/`CI()` are the two spellings with their arity checked. The bare names `I` and `SI` in a term list both mean the simple intercept.
`LinearShift` / `LS`	Linear shift `beta * x` — the interpretable log-odds coefficient. Exactly one parent.
`ComplexShift` / `CS`	Complex shift: an NN `g(x)`, additive on the latent scale. Several parents form one joint network.
`VaryingCoefficient` / `VC`	Varying-coefficient shift `(beta0 + b_theta(mods)) * x_t` — the penalized treatment-effect head (issue #28). `center=` adds propensity centering (issue #30).
`ContinuousNode`	Continuous variable: monotone 1-D transform plus shifts. `terms` is the first positional argument.
`OrdinalNode`	Ordinal variable with `levels` classes: ordered logit (cutpoints) plus shifts.
`node_parents()`	Ordered de-duplicated parent names of a node (the canonical term list is `node.terms`).
`validate_and_sort()`	Edge-ownership validation plus Kahn topological sort. The returned order makes the flow triangular.
`spec_to_dict()` / `spec_from_dict()`	Checkpoint (de)serialization. A term serializes as `{term, parents, options}` and nothing else, since `options` is already canonical. No compatibility shims: `spec_from_dict` rejects a term without `options`, the node constructors normalize the formula, and `validate_and_sort` checks the DAG.
(`_normalize_terms`, `_as_term`, `_term_class`, `net_options`)	Formula flattening and per-entry validation (a `+` sum nested in a list is rejected), the one-parented-`I` rule plus transform hoisting in one pass, canonical option storage against each term class's dataclass fields (an option another term takes errors).

Name	Role
(_check_node, _kahn_sort)	The stages behind validate_and_sort: parents exist, then each term's edge_parents (the VC treatment and centering rules), then edge ownership; a term's own shape (arity, option values) and a node's own (ordinal levels, the transform) are checked when they are built. Kahn's sort emits ready nodes in sorted batches, so the order is deterministic.

transforms.py — the monotone map h and the ordinal transform

Name	Role
StandardLogistic	The TRAM base distribution: log_prob, sample (generator-aware), icdf.
BernsteinUT	Bernstein-polynomial transform (the default, n_coeffs=20). Linear tail extrapolation follows the boundary derivative. marginal_init_theta(column) gives the marginal start init_marginals applies: the Bernstein approximation of logit(F_hat(y)), or the plain linear map onto the latent's range_q quantiles when called without a column.
SplineUT	Monotone rational-quadratic spline (bins=8). Tails extrapolate with a *fixed* slope — the structural reason spline trails Bernstein on tail-heavy data.
AffineUT	Monotone affine transform: the node-conditional is a logistic GLM.
make_univariate_transform()	Transform registry: name → transform instance.
ordinal_cutpoints()	Unconstrained (n, K-1) → increasing cutpoints with ±inf ends. Port of the original parametrization.
ordinal_log_prob()	log P(Y=y), computed in log-space. Load-bearing: the naive sigmoid difference saturates in float32 and freezes nodes at init. Do not simplify.
ordinal_pmf() / ordinal_sample() / ordinal_abduct()	Class probabilities / latent → level / truncated-logistic latent recovery (Pearl step 1) for ordinal nodes.
ordinal_marginal_init_theta()	Cutpoint start that matches the empirical class frequencies (init_marginals).
ordinal_bounds()	The shifted cutpoint interval of each observed level. scores.py reads it for the latent-scale derivative.
(_ScaledUT, _log1mexp)	Quantile pre-scaling base class, whose inverse is zuko's with its closed-form tail. Stable log(1-exp(x)).

conditioners.py — the networks behind the terms

The default architectures replicate the PyTorch reference that this package grew out of, which is buehlpa/TramDag, file tram_models.py. Therefore a fitted model stays comparable to it.

The defaults are **not** the TRAM-DAG paper's nets. The paper's R code uses c(2, 25, 25, 2) with sigmoid for the triangle experiments. It uses a 10-100 tanh net for its CAREFL and VACA comparisons. Therefore each config in experiments/paper/ states units= and activation= itself.

Name	Term	Role
SimpleIntercept	bare I	Free parameter vector; no parents.

Name	Term	Role
`ComplexIntercept`	`I(...)`	8-8 ReLU NN from parent features to the transform parameters.
`LinearShift`	`LS`	`Linear(n, 1, bias=False)`. `.weight` is the interpretable coefficient; no bias because the intercept slot owns the constant.
`ComplexShift`	`CS`	64-128-64 ReLU NN to one shift value.
`VaryingCoef`	`VC`	`beta0 + b_theta(mods)` with a zero-initialized output layer and the L2 hook `l2()`. `beta()` evaluates the effect, `recenter()` re-splits `beta0`/`b_theta` after training (function-preserving).
`(_nn)`	—	The one NN builder: a stack of the given `units` with the term's `activation` (relu by default, optional `batch_norm` before it), then a bias-free output layer.

`flow.py` — the model

Name	Role
`CausalFlowDAG`	The flow: one `_Node` per variable in topological order. Construction seeds the weights (`seed=` is the reproducibility knob).
`calibrate()`	Once, from the training rows, each term for itself: transform ranges (train `range_q` quantiles onto the domain), input-transform statistics. Never touches the weights. Called by the first fit; a checkpoint carries the flag.
`init_marginals()`	The calibrated start as an explicit step, callable any time: resets every simple intercept to its column's marginal (Bernstein map / ordinal class log-odds; spline and affine have no calibrated start). Not once-guarded — on a trained flow it restarts those intercepts. Calibrates a fresh flow's ranges itself.
`fit()`	Joint maximum likelihood: one minibatch Adam loop over all parameters (exact per node, because the NLL decomposes), final weights kept. Keras-shaped validation (`validation_data=`/`validation_split=` fill `history["val"]` per epoch, `validation_batch_size=` chunks the pass), the optimizer's rate after every epoch in `history["lr"]` (`{node: lr}` with `per_node_adam`), and progress (`verbose=`). Hooks: `optimizer=` (any torch optimizer, for schedulers) and `callbacks=` (one entry or a list; a `Callback` hooks `on_fit_begin`/`on_epoch_end`/`on_fit_end`, a bare callable is an `on_epoch_end` hook `cb(flow, epoch, opt)` — any `True` stops); the recipes in `callbacks.py` read `history["val"]`. A centered VC's out-of-fold propensities ride the training frame as the column its `center=` names. A second call continues training.
`fit_classical()`	Float64 full-batch L-BFGS for all-`ls` specs: deterministic, exact MLE, matches `statsmodels`/R `polr`. Refuses flexible specs.
`sample()`	Observational, interventional (`do=`, graph mutilation) and counterfactual (`u=`) sampling.
`abduct()`	Pearl step 1: recover the latents. Continuous exactly, ordinal by truncated draw.
`pmf()`	Analytic class probabilities of an ordinal node, with `do=` overrides.
`density()`	Analytic conditional density of a continuous node on a grid, with `do=` overrides — the continuous counterpart of `pmf`.

Name	Role
`log_prob()` / `nll()`	Joint per-row log-likelihood, or a `nodes=` subset (exact, and the log-space way to get one node's conditional likelihood per row) / mean per-node NLL diagnostic.
`node_log_prob()`	The per-node decomposition everything trains and evaluates through.
`varying_coef()`	Closed-form read-out `beta(x)` of a fitted VC term. Deterministic, y-free.
`scores()` / `effect_modifier_scan()`	Analytic per-observation scores and the CUSUM modifier scan (delegate to `scores.py`).
`intercept_contributions()`	Post-hoc GAM-style decomposition of a complex intercept into mean-centered per-term parts.
`ls_coefficients()`	The per-node linear-shift weights — the interpretable coefficients.
`design_matrix()`	Parent encoding as a DataFrame (`drop_first=` gives the classical statsmodels/`polr` design).
`to_matrix()`	The labeled meta-adjacency matrix of term tags.
`save()` / `load()`	Checkpoints with history and provenance (version, time, device). `load` requires a complete checkpoint and fails loudly otherwise.
(`_node`, `_encode_parent`, `_features`, `_tensorize`, `_generator`, `_dtype`, `_np_dtype`)	Node lookup with one shared error; parent encoding (continuous raw, ordinal one-hot); `_tensorize(df, cols=None)` for any column subset, checking every ordinal column against its levels on the way in; seeded-generator and dtype plumbing.
(`_check_side_columns`, `_binary_p1`, `_side_feats`, `_query_side_columns`, `_recenter_vc`)	The generic side-column plumbing (each term names/validates/recomputes its own columns via the `ShiftTerm` hooks) plus the binary propensity fit and the post-fit `finalize` loop.
(`_is_classical`)	Guard for `fit_classical`: every term's `classical` — `LS`, or a parentless `I()` transform carrier.

`terms.py` — the term modules (the 1.0 architecture's core)

There is one module class per term. Each module class declares the `Term` subclass it builds (`data = CS`). For the contract diagram, see architecture.md.

Name	Role
`module_for()`	The dispatch: a `TermDef` subclass declaring `data = <Term subclass>` stamps itself onto that class as `module` when defined, so subclassing is the registration; a term class no module declares fails by name.
`ShiftTerm` / `InterceptTerm`	The behavior hooks a term module owns: `build`, `shift_value`/`theta_value`, `post_init`, `regularizer`, post-fit `finalize`, `score_columns`, the side-input contract; `data` names the `Term` subclass it builds.
`LinearShiftTerm` / `ComplexShiftTerm` / `VaryingCoefficientTerm` / `FnShiftTerm`	The built-in shift terms, subclassing their conditioners (state-dict paths and the seeded RNG stream stay bit-stable). `VaryingCoefficientTerm.regressor` is both the forward regressor and the `beta0` score.
`SimpleInterceptTerm` / `ComplexInterceptTerm` / `AdditiveInterceptTerm`	The intercept slot: free theta, one joint net, or one net per parent summed in coefficient space. `net_groups` gives the per-group networks a read-out needs.
(`_InputTransform`)	One term's frozen network-input transform (minmax / standardize / callable over frozen train columns).

`nodes.py` — the node model

Name	Role
(`_Node`)	One sub-model per variable: builds its intercept and shift terms through `module_for`; `theta_shift()` sums the terms' `shift_values` (plain shifts first, then VC); `net_input()` feeds every term network, `input_transform` applied.
(`kind_log_prob`, `kind_sample`, `kind_abduct`, `kind_marginal_theta`)	The ONLY continuous-vs-ordinal branches in the package, adjacent.
(`_init_linear`)	Keras' `glorot`/`normal` initializers on one linear layer.

`fitting.py` — `_FitMixin`

Name	Role
`fit()` / `fit_classical()` (`_split_validation`, `_normalize_callbacks`, `_check_epoch_hook`, `_check_fit_sizes`, `_epoch_pass`, `_log_epoch`, `_val_nll`, `_fit_epoch`, `_FnCallback`)	Defined here once, methods of the flow via the mixin. The loop plumbing: Keras-shaped validation split, callback normalization and pre-fit signature checks, the epoch/validation passes, verbose printing.

`readouts.py` — `_ReadoutsMixin`

Name	Role
`shift_curve()`	One fitted shift term on a 1-D grid, through the term's own `shift_value` — the public replacement for reaching into `nd.shifts[..]`.
the read-out methods	`varying_coef`, `ls_coefficients`, `to_matrix`, `intercept_contributions`, `design_matrix` — defined here once, methods of the flow via the mixin.

`scores.py` — effect-modifier detection (issue #29)

Name	Role
`node_scores()`	Analytic, exact per-observation scores `psi_i = d l_i / d theta` for every `LS` weight and VC `beta0`. No autograd.
`effect_modifier_scan()`	Zeileis-Hornik fluctuation scan: order the treatment scores by each candidate, `sup|CUSUM|` against the Kolmogorov 5% value. A measured shortlist for VC modifiers from a seconds-long classical fit.
`sup_bb_pvalue()`	`P(sup |Brownian bridge| > stat)`, the Kolmogorov series.
(`_dl_ds`, `_is_binary_ordinal`, `CRIT_5PCT`)	Closed-form latent-scale derivative. Whether a treatment is a two-level ordinal, asked of the spec. The 5% critical value 1.3581. The per-term columns come from each term's `score_columns` hook.

`callbacks.py` — the shipped `fit` callbacks

Name	Role
`EarlyStopping`	Snapshots the weights of the best summed validation NLL (read from `history["val"]`) and restores them automatically at fit end (`restore_best=False` keeps the final weights), before the VC re-centering; an optional `patience` also stops the fit once the best is that many epochs old.
`PerNodePlateau`	Per-node lr decay and freezing on each node's own validation NLL (from `history["val"]`); stops the fit once every node froze, and records `frozen = {node: epoch}`. Opt-in. `step(nll, opt)` for a hand-computed NLL.
`per_node_adam()`	Adam with one `node`-tagged parameter group per node — the optimizer `PerNodePlateau` needs.

`plots.py` — the figures (matplotlib optional: `tramdag[plots]`)

Name	Role
`plot_dag()`	The labelled DAG of a spec or flow: layered left to right, ellipses for continuous and rounded boxes for ordinal nodes, every edge drawn by the term that owns it (LS / CS / CI / VC + modifiers / Fn, `joint` for a multi-parent net). Exported as `tramdag.plot_dag`.
`plot_marginals()`	Observed vs sampled marginal per node, one panel each.
`plot_training()`	Summed train/val NLL per epoch, with the freeze marks read off `history["lr"]` (or a given `frozen=`).
`(_layout, _edges)`	Longest-path layers with one barycenter sweep; the edge list with the VC treatment/modifier split. matplotlib is imported on the first call, never at package import.

What is *not* in the package

Three groups of files are research code:

- the SCM generators
- the frozen datasets
- the replication scripts

They live in `experiments/`, outside the installed package. For details, see `experiments/README.md`. The framework's own tests do not depend on them. The tests measure against three inline DGPs in `tests/conftest.py`.

Where every training hyperparameter lives

Everything that shapes a fit is either a keyword you pass or a documented default you can read at the call site. Nothing numeric is buried.

Knob	Where	Default
learning rate, batch size	`fit()`	1e-2 / 512 (in-repo callers state them explicitly anyway)

Knob	Where	Default
validation, progress	`fit(validation_data=, validation_split=, valida-tion_batch_size=, verbose=)`	per-node val NLL into `history["val"]` each epoch; `verbose=N` prints every Nth + final epoch (default 0, silent)
schedules, early stopping	`fit(optimizer=, callbacks=)`	`tramdag.callbacks` ships `EarlyStopping`, `PerNodePlateau`; anything else is torch's `lr_scheduler` and a few lines of callback (fitting.md)
calibrated init	`init_marginals(train_df)`	never implicit — an always-explicit step (pure init, MLE unchanged; without it zuko's zero start)
VC stage-1 propensities	the training-frame column `VC(center=)` names	required for a centered VC term, out of fold, computed by the caller
VC penalty and centering	`VC(penalty=, center=)`	1.0 / False (`center="col"` names the propensity column)
L-BFGS budget	`fit_classical(max_iter=, tol=, history_size=)`	400 / 1e-9 (torch `tolerance_change`; `tolerance_grad` is off) / 50 — one full-batch run, no chunks
training budget	`fit(epochs=)`	**required** — a fixed default is wrong in both directions (training-speed)
network widths	`units=` on `I/CS/VC`	(8, 8) / (64, 128, 64) — parity with the PyTorch reference's default classes; VC's (16,) has no counterpart there and comes from the recovery measurement
activation	`activation=` on `I/CS/VC`	`"relu"` (the reference default classes); `"sigmoid"` and `"tanh"` are the paper's
batch norm	`batch_norm=` on `I/CS/VC`	`False` — neither reference uses it. `True` puts a `BatchNorm1d` between each hidden layer and its activation, so the fit needs more than one row per batch and inference needs `eval()` mode (`fit` and `load` leave the flow there)
transform class	`I(transform=, **kwargs)` (extra kwargs go to the transform class)	`"bernstein"`, `n_coeffs=20` unconstrained coefficients (zuko ties two more control points on, so order 21); spline `bins=8` = zuko's NSF default (the domain is fixed at [-5, 5], `transforms.BOUND`)
shuffling / weight init	`fit(seed=)` / `CausalFlowDAG(seed=)`	init happens at construction — the constructor seed is the reproducibility knob
weight init	`CausalFlowDAG(init=)`	`"torch"` (`nn.Linear` Kaiming-uniform); `"glorot"` = Keras `Dense` default, glorot-uniform weights and zero biases — the paper's reference; decisive under its full-batch protocol
network inputs	`CI/CS/VC(input_transform=)`	`None` (raw parents); `"minmax"` / `"standardize"` transform that term's continuous parents with statistics frozen at `calibrate`, and a callable `fn(x, train)` gets the frozen raw train column — LS and the VC treatment stay raw

Upstream candidates for zuko

This document lists what tramdag works around in zuko (1.6.0). The list is ranked by value against effort, as candidate upstream PRs or issues. tramdag uses zuko in one place only. It uses these three transforms from `zuko.transforms`:

- `BernsteinTransform`
- `MonotonicRQSTransform`
- `MonotonicAffineTransform`

The three wrappers in `src/tramdag/transforms.py` hold them. The flow machinery, the base distribution and the DAG structure are tramdag's own. The suggested order of attack is 4 → 1 → 3 → 2 → 5.

1. Analytic `call_and_ladj` for `BernsteinTransform`

`BernsteinTransform` inherits `MonotonicTransform.call_and_ladj`. That method gets the Jacobian by `torch.autograd.grad` in the forward pass. It therefore roughly doubles the training graph.

It also makes the per-node loss un-`torch.compile`-able. Autograd in the forward pass gives a double backward, and `aot_autograd` does not support one. Therefore the op-count axis stays closed in eager mode.

The derivative is closed form: `f'(x) = order·Δθ` against the order−1 basis. zuko already computes it in `_offset_and_slope` for the tail slopes. An override is therefore ~25 lines and needs no API change. `MonotonicRQSTransform` already has such an override. tramdag gets a free speedup and the `torch.compile` axis back.

2. Learnable/linear tails for `MonotonicRQSTransform`

zuko pads the knot derivatives with `exp(0)=1`. It extrapolates as the identity outside `[-B, B]`. Therefore the tail slope stays fixed at 1 for every θ. This is the structural reason why `spline` consistently trails `bernstein` here, because ~10% of the data sits beyond the 5%/95% pre-scaling range. CLAUDE.md and `spec.py` also describe this.

Upstream, zuko can accept boundary derivatives of shape `(*, K+1)`, or it can take an opt-in `tails="linear"`. Identity tails are deliberate in the NSF design. The framing must therefore stay back-compatible. This change will delete the caveats and make `spline` a first-class transform choice.

3. Public inverse of `_constrain_theta`

`BernsteinUT.marginal_init_theta` inverts zuko's *private* `_constrain_theta` (cumsum-of-softplus) in closed form. It hard-codes two internal details of that function:

- the `log(2)·n/2` centering
- the tying of the first two and the last two diffs

Any upstream reparametrization therefore breaks the calibrated start silently. A public `unconstrain_theta(theta)` that takes the target control points is ~15 lines upstream. It removes the coupling to these private details. The framing is "identity/linear initialization support", a standard flow trick.

4. Docstring fix: the Bernstein θ-shape off-by-one

zuko's docstring says that θ has shape `(*, M-2)` for a degree-M polynomial. `n` unconstrained coefficients become `n+2` control points, which is degree `n+1`. The correct claim is therefore `(*, M-1)`. This off-by-one is a real replication trap: order 21 against the paper's `len_theta=20`, which

is order 19. It costs tramdag prose in three documents. The PR is trivial and an accept is near-certain.

5. `Logistic` in `zuko.distributions`

Neither zuko nor torch ships a Logistic distribution. tramdag therefore hand-rolls `StandardLogistic` in ~50 lines, with `log_prob`, `sample` and `icdf`. Logistic latents are standard in transformation models and in discrete flows, and zuko has a precedent in `GeneralizedNormal`. But zuko can point at `TransformedUniform(SigmoidTransform().inv)` instead. tramdag keeps its `_U_EPS` clamp locally in both cases. This candidate has the lowest value of the five.

Anti-candidates (look upstreamable, are not)

- **Quantile pre-scaling and `_ScaledUT`** — this is a modeling choice. zuko's `ComposedTransform` with `MonotonicAffineTransform` already composes it.
- **The ordinal ordered-logit transform and log-space `ordinal_log_prob`** — this transform is not a bijection. It stays outside zuko's flow scope. It is torch.distributions material, if anywhere.
- **LS, CS, CI and VC terms, the marginal-init policy, propensity centering and scores** — these are TRAM-DAG semantics. zuko's `MaskedMLP` already supports arbitrary adjacency. tramdag does not use it, because it needs per-term interpretability.
- **The spline `slope` knob and the Bernstein `eps` knob** — zuko already exposes both upstream.

Additive vs joint complex intercept — interpreting per-parent effects

A node whose transform parameters depend on its parents (a **complex intercept**, `CI`) can group those parents two ways:

- **joint** — `CI("x1", "x2")`: *one* network over both parents. It can represent **interactions** (the effect of `x1` may depend on `x2`), but the two parents are entangled in one black box. This is the default.
- **additive** — `CI("x1", "x2", allow_interaction=False)`: *one network per parent*, summed in unconstrained parameter space, `theta(pa) = net_1(x1) + net_2(x2)`. Each parent reshapes the transform **independently** — a separable, GAM-like structure.

The grouping is said with the flag, not by writing two terms: a node takes at most **one** intercept term with parents, so `[CI("x1"), CI("x2")]` is an error. That keeps a term list purely additive on the latent scale.

The additive form is the interpretable one: you can ask "what does `x1` *alone* do?". The catch (issue #20) is that the additive sum is identified only up to a constant moving between the nets, so the **raw** per-parent outputs are not comparable. `flow.intercept_contributions(data, node)` resolves this with a sum-to-zero (mean-centering) constraint and returns each parent's centered contribution.

This notebook fits both models and shows the interpretability difference. The punchline is perhaps surprising: the two models fit the **data** almost identically — the difference is **structural**, visible only in parameter space, and that is precisely what `intercept_contributions` surfaces.

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
```

```
from tramdag import CI, CausalFlowDAG, ContinuousNode
from tramdag.callbacks import EarlyStopping
```

The data

`x1, x2 ~ U(-2, 2)` and $x_3 = x_1 + x_2 + \gamma\, x_1 x_2 + \frac{1}{2}\,\varepsilon$, $\varepsilon \sim \text{Logistic}(0,1)$ — a genuine `x1·x2` interaction.

```
GAMMA = 0.6


def simulate(n, seed):
    rng = np.random.default_rng(seed)
    x1 = rng.uniform(-2, 2, n)
    x2 = rng.uniform(-2, 2, n)
    eps = rng.logistic(0, 1, n)
    x3 = x1 + x2 + GAMMA * x1 * x2 + 0.5 * eps
    return pd.DataFrame({"x1": x1, "x2": x2, "x3": x3})


train = simulate(4000, seed=1)
val = simulate(1000, seed=2)
train.head()
```

x1

x2

x3

0

0.047286

0.755745

1.239427

1

1.801855

0.126365

0.853817

2

-1.423362

0.970406

-0.294179

3

1.794598

1.590401

4.880817

4

-0.752674

-0.168717

-0.689205

Fit both models

Same data, same node — only the grouping of the two `I` parents differs.

```
def make_flow(joint: bool):
    spec = {
        "x1": ContinuousNode(),
        "x2": ContinuousNode(),
        "x3": ContinuousNode(
            [CI("x1", "x2")] if joint else [CI("x1", "x2", allow_interaction=False)]
        ),
    }
    return CausalFlowDAG(spec, seed=0)


flow_joint = make_flow(joint=True)
flow_add = make_flow(joint=False)

for f in (flow_joint, flow_add):
    f.fit(
        train,
        epochs=1200,
        learning_rate=1e-2,
        validation_data=val,
        callbacks=EarlyStopping(),  # keep the best-validation weights
    )

# Both fit the data well; the joint model is only marginally better on held-out
# likelihood. A flexible additive Bernstein intercept can *mimic* a lot of
# apparent interaction through its nonlinear transform, so you generally **cannot
# tell the two apart from the fitted distribution** — see the note below.
print("val NLL  joint   :", round(sum(flow_joint.nll(val).values()), 4))
print("val NLL  additive:", round(sum(flow_add.nll(val).values()), 4))
```

```
val NLL  joint   : 4.1711
val NLL  additive: 4.202
```

`intercept_contributions` — the exact, centered decomposition

It returns each `I`-term's mean-centered (sum-to-zero) contribution to the transform parameters `theta`, plus the absorbed `baseline`. The centering is over the rows of the `df` you pass, and the decomposition is exact.

```
res_add = flow_add.intercept_contributions(train, "x3")
res_joint = flow_joint.intercept_contributions(train, "x3")

print("additive terms :", list(res_add["contributions"]))  # 'x1', 'x2' — separable
print("joint terms    :", list(res_joint["contributions"]))  # 'x1+x2' — inseparable

# the decomposition is exact (baseline + contributions == theta, pinned by the
# test suite); the centering is easy to see here: every column mean is ~0
print(
    "per-term column means (≈0)   :",
    {k: float(np.abs(v.mean(0)).max()) for k, v in res_add["contributions"].items()},
)
```

```
additive terms : ['x1', 'x2']
joint terms    : ['x1+x2']
per-term column means (≈0)   : {'x1': 3.5154818078808603e-07, 'x2': 6.728172365910723e-
07}
```

The structural difference: separable curves vs an entangled cloud

Pick the transform coefficient that varies most across the data and plot each term's centered contribution to it against a parent value.

- **Additive** (`net_1` depends only on `x1`): its contribution to any coefficient is a deterministic 1-D function of `x1` — the points collapse to a **clean curve** (same for `x2`). These curves *are* the per-parent partial effects: well-defined because the structure is separable.
- **Joint** (one network over both parents): the single `x1+x2` component depends on *both*, so plotted against `x1` alone it is a **cloud** whose height is explained by the hidden `x2` (color). There is nothing to read off per parent.

```
c1 = res_add"contributions"
c2 = res_add"contributions"
cj = res_joint"contributions"
k = int((c1.var(0) + c2.var(0)).argmax())  # most-varying coefficient
x1v, x2v = train["x1"].values, train["x2"].values

fig, axes = plt.subplots(1, 2, figsize=(11, 4.2), sharey=True)
o1, o2 = np.argsort(x1v), np.argsort(x2v)
axes[0].plot(x1v[o1], c1[o1, k], color="#1b9e77", lw=2.5, label="net(x1)  vs x1")
axes[0].plot(x2v[o2], c2[o2, k], color="#d95f02", lw=2.5, label="net(x2)  vs x2")
axes[0].axhline(0, color="0.7", lw=0.8)
axes[0].set_title("additive — separable per-parent curves")
axes[0].set_xlabel("parent value")
axes[0].set_ylabel(f"centered contribution to theta[{k}]")
axes[0].legend()

sc = axes[1].scatter(x1v, cj[:, k], c=x2v, s=8, alpha=0.6, cmap="coolwarm")
axes[1].axhline(0, color="0.7", lw=0.8)
axes[1].set_title("joint — vs x1 is a cloud (height set by x2)")
axes[1].set_xlabel("x1")
fig.colorbar(sc, ax=axes[1], label="x2")
fig.tight_layout()
plt.show()
```

Takeaway

you want…	use	what you get
a per-parent partial-effect plot ("what does x1 do?")	**additive** `CI("x1", "x2", allow_interaction=False)`	`intercept_contributions` → exact, mean-centered, **separable** components
interactions between parents in the transform	**joint** `CI("x1", "x2")`	one entangled network — flexible, **not** separable

Both give correct likelihoods and L1/L2/L3 causal queries — the choice is about *interpretability*, not correctness, and (as the near-equal NLLs show) you usually can't distinguish them from the fitted distribution alone. The separability that makes per-parent effects well-defined is a property

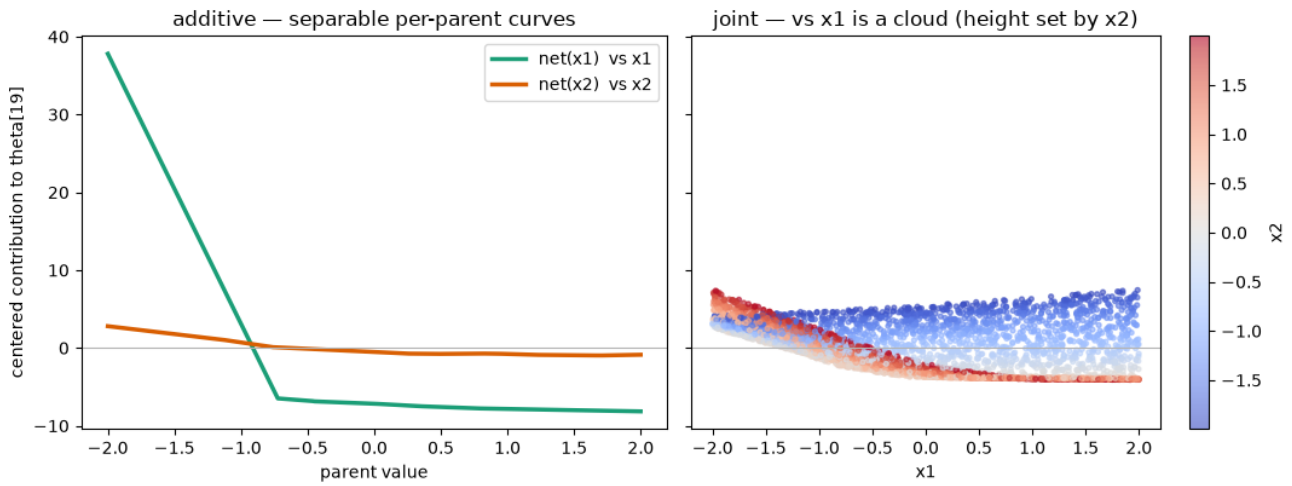


Figure 1: png

of the **parameter space**, and `intercept_contributions` is what surfaces it. It is post-hoc: it reads the fitted weights and changes nothing about the model.

Caveat: contributions live in the transform's **unconstrained** parameter space (where the additive terms are summed, before the monotonicity constraint), so they are exact partial effects on the parameters but not, in general, an additive split of the curve `h` itself.

Classical fitting of all-`ls` TRAM-DAGs (`fit_classical`)

When **every edge of a TRAM-DAG is a linear shift (LS)**, each node-conditional is a *classical transformation model*

- an ordered-logit / proportional-odds model for ordinal nodes
- a continuous-outcome logistic transformation model (R's `tram::Colr`) for continuous ones.

For such a model, the classical optimizer `flow.fit_classical()` is best suited: - **full-batch, float64, L-BFGS** with a strong-Wolfe line search, - **deterministic** — no minibatching, so the same start gives bit-identical results, - lands on the **exact maximum-likelihood estimate**, matching `statsmodels` and R to ~1e-3 on the well-identified coefficients.

Every R reference printed below is hard-coded from a real fit, and every one of those fits lives in `notebooks/classical_fit_tram_dag.R`. Run `Rscript notebooks/classical_fit_tram_dag.R` from the repo root to re-check them; it needs `MASS` (ships with R) and `tram`.

```
import time
from pathlib import Path

import numpy as np
import pandas as pd
import torch
from statsmodels.miscmodels.ordinal_model import OrderedModel

from tramdag import LS, SI, CausalFlowDAG, ContinuousNode, OrdinalNode
from tramdag.callbacks import PerNodePlateau, per_node_adam

# repo-relative data, whether the notebook runs from the repo root or notebooks/
HERE = [Path.cwd(), *Path.cwd().parents]
```

```
REPO = next(p for p in HERE if (p / "pyproject.toml").exists())
DATA = REPO / "experiments" / "misc" / "data"
NB_DATA = REPO / "notebooks" / "data"


def claim(text, value, bound, fmt="{:.2e}"):
    """Assert a computed quantity stays inside its bound, and print both.

    Every agreement this notebook asserts goes through here, so no number is
    typed into a markdown cell where it can drift away from the code. The
    prose says what is being checked and why the bound is what it is; the
    bound is the only number in the source.

    A bound sits at roughly 1.5 to 4 times the measured value, which is the
    band that ``experiments/*/ground_truth/*.json`` already uses. Tighter than
    that and another machine trips it; wider and it stops meaning anything.
    """
    ok = "ok  " if value <= bound else "FAIL"
    print(f"{ok}{text}: {fmt.format(value)} <= {fmt.format(bound)}")
    assert value <= bound, f"{text}: {fmt.format(value)} exceeds {fmt.format(bound)}"
```

0. The zeroth example — logistic regression on `MASS::birthwt`

Before any DAG, take the model everyone already knows, on a dataset every R user already has. A **two-level** `OrdinalNode` whose terms are all `LS` *is* logistic regression — not an analogue of it, the same model. The ordinal transform is

$$P(Y \le 0 \mid x) = \sigma\big(\theta_0 - \textstyle\sum_p w_p x_p\big)\,,$$

and with two levels there is a single cutpoint θ_0, so

$$\operatorname{logit} P(Y = 1 \mid x) = -\theta_0 + \textstyle\sum_p w_p x_p.$$

The shift weights **are** the logistic-regression coefficients; the intercept is **minus** the cutpoint, because an ordinal node *subtracts* its shift. That sign convention is the one thing worth internalizing here — Section 2 is this same picture with $K - 1$ cutpoints instead of one.

The data is `birthwt` from **MASS**, the low-birth-weight study of Hosmer & Lemeshow: 189 births, outcome `low` (birth weight under 2.5 kg), predictors `age` (mother's age), `lwt` (mother's weight at last period, lbs) and `smoke` (smoking during pregnancy). `notebooks/data/birthwt.csv` is those four columns exported verbatim from MASS. Please note that this is not a **causal** model.

The R you can copy-paste

`birthwt` ships with MASS, so nothing needs downloading — paste this into any R session and you have the reference fit:

```
library(MASS)
m <- glm(low ~ age + lwt + smoke, data = birthwt, family = binomial)
coef(m)
logLik(m)
```

which prints

```
(Intercept)         age         lwt       smoke
 1.36822527 -0.03899458 -0.01213854  0.67076374
```

```
'log Lik.' -111.4396765 (df=4)
```

(R 4.2.3, MASS 7.3-58.2.) Those four numbers are hard-coded below as `R_GLM`, so the notebook can show the three-way comparison without an R installation.

```
R_GLM = {  # coef(glm(low ~ age + lwt + smoke, birthwt, family = binomial))
    "intercept": 1.3682252685,
    "age": -0.0389945827,
    "lwt": -0.0121385423,
    "smoke": 0.6707637407,
}
R_LOGLIK = -111.43967649

bw = pd.read_csv(NB_DATA / "birthwt.csv")
print(
    bw.head(3).to_string(index=False),
    f"\n\n{len(bw)} births, {bw['low'].sum()} of them low birth weight",
)
```

```
 low  age  lwt  smoke  bwt
   0   19  182      0 2523
   0   33  155      0 2551
   0   20  105      1 2557

189 births, 59 of them low birth weight
```

The spec. `low` is the outcome; `smoke` is a binary **parent**, so it is an ordinal node too. `age` and `lwt` are source nodes — the flow models their marginals as well, which a plain `glm` does not. That is a nuisance here, not a feature, so they get the cheapest transform (`SI(transform="affine")`), and it cannot disturb the outcome node in any case: the joint NLL decomposes per node, so the `low` node's gradient never sees the covariate marginals.

```
spec_bw = {
    "age": ContinuousNode([SI(transform="affine")]),
    "lwt": ContinuousNode([SI(transform="affine")]),
    "smoke": OrdinalNode(2),
    "low": OrdinalNode(2, [LS("age"), LS("lwt"), LS("smoke")]),
}

flow_bw = CausalFlowDAG(spec_bw, seed=0)
flow_bw.fit_classical(bw)
```

```
{'converged': True,
 'n_iter': 64,
 'final_nll': 9.137612929338468,
 'grad_norm': 0.0001517562203257858,
 'seconds': 1.2790842279999879,
 'coefficients': {'low': {'age': array([-0.03899488]),
   'lwt': array([-0.01213841]),
   'smoke': array([-0.13497243,  0.53579548])}}}
```

Now read the coefficients back out. `age` and `lwt` are continuous parents, so they enter raw and their weights compare directly. `smoke` is ordinal, so it enters **one-hot over both levels**.

> **Careful — different coding from R.** R gives a binary predictor one column; we give it two, w_0 and w_1. Only the difference $w_1 - w_0$ is determined by the data, and it is what R's single `smoke` coefficient means. Each level alone is fixed by the weight initialization: a different `seed` shifts both by the same constant and moves θ_0 to compensate, leaving every fitted probability unchanged. Section 2 uses the same rule for `T`.

The second column also moves the intercept. With $s \in \{0, 1\}$ we have $\mathbf{1}[s = 0] = 1 - s$, so the one-hot pair collapses to

$$w_0 \, \mathbf{1}[s = 0] + w_1 \, \mathbf{1}[s = 1] \;=\; w_0 + (w_1 - w_0)\, s,$$

and the node reads

$$\operatorname{logit} P(\texttt{low} = 1) \;=\; (-\theta_0 + w_0) \;+\; w_{\text{age}}\, \texttt{age} \;+\; w_{\text{lwt}}\, \texttt{lwt} \;+\; (w_1 - w_0)\, s.$$

The first bracket is R's `(Intercept)` — it is $-\theta_0 + w_0$, not $-\theta_0$ — and the last is R's `smoke`.

```python
import statsmodels.formula.api as smf  # noqa: E402

from tramdag.transforms import ordinal_cutpoints  # noqa: E402

# the same formula as the R call above, and the same names in the output
res_bw = smf.logit("low ~ age + lwt + smoke", data=bw).fit(disp=False)

w = flow_bw.ls_coefficients()["low"]
with torch.no_grad():  # cutpoints are [-inf, theta_0, +inf]; take the finite one
    theta_0 = float(ordinal_cutpoints(flow_bw.nodes["low"].intercept(1))[0, 1])

rows_bw = [
    # -theta_0 + w_smoke[0]: the one-hot level-0 column is part of the intercept
    (
        "intercept",
        -theta_0 + float(w"smoke"),
        res_bw.params["Intercept"],
        R_GLM["intercept"],
    ),
    ("age", float(w"age"), res_bw.params["age"], R_GLM["age"]),
    ("lwt", float(w"lwt"), res_bw.params["lwt"], R_GLM["lwt"]),
    (
        "smoke",
        float(w"smoke" - w"smoke"),
        res_bw.params["smoke"],
        R_GLM["smoke"],
    ),
]
print(f"{'':<11}{'fit_classical':>14}{'sm.Logit':>10}{'R glm':>10}{'max|diff|':>11}")
for name, flow_value, sm_value, r_value in rows_bw:
    spread = max(abs(flow_value - sm_value), abs(flow_value - r_value))
    print(
        f"{name:<11}{flow_value:>14.4f}{sm_value:>10.4f}{r_value:>10.4f}{spread:>11.2e}"
    )
```

```
            fit_classical  sm.Logit     R glm  max|diff|
intercept          1.3682    1.3682    1.3682   6.46e-06
age               -0.0390   -0.0390   -0.0390   2.96e-07
lwt               -0.0121   -0.0121   -0.0121   1.32e-07
smoke              0.6708    0.6708    0.6708   4.16e-06
```

Three optimizers — L-BFGS on a transformation model, `statsmodels`' Newton solver, R's IRLS — land on the same maximum likelihood, because there is only one. The agreement is not only in the parameters: the log-likelihood of the `low` node and R's `logLik(m)` are the same number, and `flow.pmf` reproduces `glm`'s fitted probabilities row by row.

```
# nll() is the *mean* NLL per node; times n and negated it is the log-likelihood
ll_flow = -flow_bw.nll(bw)["low"] * len(bw)
p_flow = flow_bw.pmf(bw, node="low")[:, 1]
p_classical = res_bw.predict(bw).values
print(f"log-likelihood   flow {ll_flow:.6f}   R glm {R_LOGLIK:.6f}")
print(
    f"max |P_flow(low=1) - P_glm(low=1)| over {len(bw)} rows: "
    f"{np.abs(p_flow - p_classical).max():.2e}"
)
```

```
log-likelihood   flow -111.439677   R glm -111.439676
max |P_flow(low=1) - P_glm(low=1)| over 189 rows: 3.32e-06
```

1. Continuous case

Section 0 modelled `low`, the *dichotomized* birth weight. `birthwt` also carries the number it was cut from — `bwt`, the weight in grams — so the same three predictors can be fitted against a continuous outcome. That is a continuous outcome logistic transformation model, R's `tram::Colr` bwt ~ age + lwt + smoke No way to model this with glm.

```
R_COLR_BWT = {  # Colr(bwt ~ age + lwt + smoke, order = 21); R 4.2.3 / tram 1.0.4
    "age": -0.021971,
    "lwt": -0.010233,
    "smoke": 0.668698,
    "loglik": -1499.6678,
}

spec_bwt = {
    "age": ContinuousNode([SI(transform="affine")]),  # nuisance marginals
    "lwt": ContinuousNode([SI(transform="affine")]),
    "smoke": OrdinalNode(2),
    "bwt": ContinuousNode(
        [SI(n_coeffs=20), LS("age"), LS("lwt"), LS("smoke")]  # degree 21
    ),
}

flow_bwt = CausalFlowDAG(spec_bwt, seed=0)
flow_bwt.fit_classical(bw, max_iter=800)

w = flow_bwt.ls_coefficients()["bwt"]
fitted = {
    "age": w"age",
    "lwt": w"lwt",
    "smoke": w"smoke" - w"smoke",  # one-hot: read the difference
    "loglik": -flow_bwt.nll(bw)["bwt"] * len(bw),
}

print(f"{'':<9}{'age':>9}{'lwt':>9}{'smoke':>9}{'logLik':>11}")
for name, r in [("flow", fitted), ("R Colr", R_COLR_BWT)]:
    print(
        f"{name:<9}{r['age']:>9.4f}{r['lwt']:>9.4f}{r['smoke']:>9.4f}"
        f"{r['loglik']:>11.2f}"
    )
diff = {k: abs(fitted[k] - R_COLR_BWT[k]) for k in fitted}
print(
    f"{'|diff|':<9}{diff['age']:>9.4f}{diff['lwt']:>9.4f}{diff['smoke']:>9.4f}"
```

```
    f"{diff['loglik']:>11.2f}"
)
```

```
               age      lwt    smoke     logLik
flow       -0.0223  -0.0103   0.6641   -1500.55
R Colr     -0.0220  -0.0102   0.6687   -1499.67
|diff|      0.0004   0.0000   0.0046       0.88
```

The largest *absolute* difference is in `smoke`, but that only reflects its coefficient being some thirty times bigger than the others; as a fraction of the estimate it is the smallest of the three. Neither comparison is the useful one. A difference matters when it is large relative to the **sampling uncertainty** of the estimate — and that is a quantity we have not computed yet.

Standard errors and confidence intervals

The sampling distribution of the maximum likelihood estimator is

$$\hat{\theta} \;\dot{\sim}\; N(\theta, I^{-1})$$

where I is the Fisher information matrix. Since θ is unknown we evaluate I at $\hat{\theta}$, and read the standard errors off the diagonal of I^{-1}. For a contrast c, we read off $\sqrt{c^\top I^{-1} c}$.

The **observed** Fisher information is the Hessian of the *negative* log-likelihood at the MLE,

$$I_{ab} = \left.\frac{\partial^2(-\ell)}{\partial\theta_a\,\partial\theta_b}\right|_{\hat{\theta}},$$

which autograd gives directly: one backward pass produces the gradient with the graph retained, then one further backward pass per component produces a row of I.

Here I is singular, so I^{-1} does not exist. It is symmetric, so it has an orthonormal eigendecomposition, and the **pseudo-inverse** inverts only the directions that carry curvature:

$$I = \sum_{k=1}^{P} \lambda_k\, e_k e_k^\top \qquad\Longrightarrow\qquad I^{+} = \sum_{k:\;\lambda_k > \tau} \frac{1}{\lambda_k}\, e_k e_k^\top, \qquad \tau = 10^{-7}\,\lambda_{\max}.$$

Terms with $\lambda_k \le \tau$ are dropped rather than inverted. That is right for a contrast which avoids the flat subspace, and the `leak` column measures exactly how far it fails to:

$$\text{leak}(c) = \max_{k:\;\lambda_k \le \tau} \left| c^\top e_k \right|.$$

Only interpret rows whose leak is close to zero. The choice of τ is not delicate, because the spectrum splits into a flat block and a curved block with orders of magnitude between them. Any threshold inside that gap gives the same answer, and the cell below measures the gap rather than asserting it.

```
from scipy.stats import norm  # noqa: E402


def observed_information(flow, node, data):
    """Give one node's NLL Hessian, the gradient norm, and parameter offsets."""
    flow.double()  # second derivatives need float64
    names, params = zip(*flow.nodes[node].named_parameters(), strict=True)
```

```
    nll = -flow.node_log_prob(flow._tensorize(data))[node].sum()
    grad = torch.cat(
        [g.reshape(-1) for g in torch.autograd.grad(nll, params, create_graph=True)]
    )
    hess = torch.stack(
        [
            torch.cat(
                [
                    h.reshape(-1)
                    for h in torch.autograd.grad(grad[i], params, retain_graph=True)
                ]
            )
            for i in range(grad.numel())
        ]
    )
    offsets, at = {}, 0
    for name, p in zip(names, params, strict=True):
        offsets[name] = at
        at += p.numel()
    result = hess.detach().numpy(), float(grad.detach().norm()), offsets
    flow.float()  # back to the stored precision
    return result


def spectrum_split(evals, tau):
    """Describe the curvature spectrum either side of ``tau``.

    A wide gap between the flat block and the curved block is what makes the
    threshold uncritical, so this reports both edges of the gap and lets the
    caller check it instead of describing it in prose.
    """
    absolute = np.abs(evals)
    flat, curved = absolute[absolute <= tau], absolute[absolute > tau]
    return {
        "n_flat": int(flat.size),
        "flat_max": float(flat.max()) if flat.size else 0.0,
        "curved_min": float(curved.min()) if curved.size else 0.0,
    }


def ls_conf_int(flow, node, data, level=0.95):
    """Give Wald intervals for the linear-shift coefficients of one node."""
    hess, grad_norm, offsets = observed_information(flow, node, data)
    evals, evecs = np.linalg.eigh(hess)
    rcond = 1e-7
    tau = rcond * np.abs(evals).max()  # one threshold, used for both
    flat = evecs[:, np.abs(evals) <= tau]  # the directions without curvature
    cov = np.linalg.pinv(hess, rcond=rcond)  # invert only the rest
    z = norm.ppf(0.5 + level / 2)

    rows = []
    for parent, w in flow.ls_coefficients()[node].items():
        c = np.zeros(hess.shape[0])
        key = next(n for n in offsets if n.startswith(f"shifts.{parent}"))
        if len(w) == 1:  # continuous parent: one weight
```

```
            c[offsets[key]], estimate, term = 1.0, w[0], parent
        else:  # ordinal parent: the contrast w[1] - w[0]
            c[offsets[key]], c[offsets[key] + 1] = -1.0, 1.0
            estimate, term = w[1] - w[0], f"{parent} (1 vs 0)"
        se = float(np.sqrt(c @ cov @ c))
        rows.append(
            {
                "term": term,
                "estimate": estimate,
                "se": se,
                "lower": estimate - z * se,
                "upper": estimate + z * se,
                # 0 = the contrast avoids the flat subspace, so the SE is real
                "leak": float(np.abs(c @ flat).max()) if flat.size else 0.0,
            }
        )
    diagnostics = {
        "n_params": hess.shape[0],
        "grad_norm": grad_norm,
        **spectrum_split(evals, tau),
    }
    return pd.DataFrame(rows).set_index("term"), diagnostics

table, diag = ls_conf_int(flow_bwt, "bwt", bw)
print(table.round(6).to_string())
print(
    f"\n{diag['n_params']} parameters, {diag['n_flat']} of them without curvature"
    f"   |grad| = {diag['grad_norm']:.1e}"
)
# Colr(bwt ~ age + lwt + smoke, order = 21) on birthwt; R 4.2.3 / tram 1.0.4,
# produced by classical_fit_tram_dag.R. This is the oracle, not the claim.
R_COLR_BWT_SE = {"age": 0.025305, "lwt": 0.004116, "smoke": 0.260131}

print()
claim(
    "flat_max / curved_min, so smaller is a wider gap",
    diag["flat_max"] / diag["curved_min"],
    1e-4,
)
se_gap = max(
    abs(float(table.loc[t, "se"]) - r)
    for t, r in [("age", R_COLR_BWT_SE["age"]), ("lwt", R_COLR_BWT_SE["lwt"])]
)
claim("max |SE_flow - SE_Colr|, continuous parents", se_gap, 1e-3)
```

```
                estimate        se     lower     upper  leak
term
age            -0.022343  0.025264 -0.071861  0.027174   0.0
lwt            -0.010276  0.004113 -0.018337 -0.002216   0.0
smoke (1 vs 0)  0.664074  0.259577  0.155312  1.172835   0.0

24 parameters, 13 of them without curvature   |grad| = 3.4e-02

ok  flat_max / curved_min, so smaller is a wider gap: 3.73e-07 <= 1.00e-04
ok  max |SE_flow - SE_Colr|, continuous parents: 4.05e-05 <= 1.00e-03
```

The standard errors of the two continuous parents reproduce `Colr`'s, and R's `confint()` on a `Colr`

fit is Wald as well.

Two things stand in the way of putting confidence intervals into the package itself:

1. The gradient norm printed above is not zero. `fit_classical` stops on flatness of the likelihood rather than on the gradient norm, so the Hessian is taken a little off the exact stationary point.
2. An interval only means something for an estimable parameter. A `conf_int` that returned one row per parameter would be mostly meaningless, because most rows sit in the flat subspace.

1b. A bimodal DAG — the demo data

The VACA benchmark triangle (`x1 → x2 → x3 ← x1`, the demo notebook's bimodal SCM): `x1` is a two-component Gaussian mixture, `x2 = -x1 + N(0,1)`, `x3 = x1 + 0.25 x2 + N(0,1)`. We fit an **all-`ls`** model: each node is a continuous logistic transformation model with a Bernstein baseline and linear shifts.

Note this is an *honest misspecification*: the DGP noise is Gaussian while the TRAM latent is logistic, so the all-`ls` model is not the true generator — but `fit_classical` still finds its exact MLE, which is the point here.

```
if False:

    def sample_vaca(n, seed):
        """Draw n rows from the VACA bimodal triangle SCM."""
        rng = np.random.default_rng(seed)
        mix, a, b = rng.uniform(size=n), rng.normal(size=n), rng.normal(size=n)
        x1 = np.where(mix < 0.5, -2.0 + np.sqrt(1.5) * a, 1.5 + b)
        x2 = -x1 + rng.normal(size=n)
        x3 = x1 + 0.25 * x2 + rng.normal(size=n)
        return pd.DataFrame({"x1": x1, "x2": x2, "x3": x3})

    df = sample_vaca(1000, seed=42)
    df.to_csv(NB_DATA / "vaca.csv", index=False)
```

The R you can copy-paste

`tram::Colr` fits the same model family — a continuous outcome logistic transformation model with a Bernstein baseline. The cell above is what wrote `notebooks/data/vaca.csv`, which is committed, so R reads the identical rows the flow is fitted on.

The two libraries **count the basis differently**, and the comparison is only meaningful at the same polynomial degree. The flow's `n_coeffs` unconstrained coefficients become `n_coeffs + 2` control points, that is a Bernstein polynomial of degree `n_coeffs + 1` (`order = n + 1` in `transforms.py`). So `N_COEFFS = 20` below is tram's `order = 21`, **not** `order = 19`.

```
library(tram)
d <- read.csv("notebooks/data/vaca.csv")

m2 <- Colr(x2 ~ x1, data = d, order = 21)
coef(m2)     # -> x1 1.800042
logLik(m2)   # -> -1401.213542

m3 <- Colr(x3 ~ x1 + x2, data = d, order = 21)
coef(m3)     # -> x1 -1.777289   x2 -0.455282
logLik(m3)   # -> -1411.901958
```

The coefficients need **no sign flip**: `Colr` reports the shift on the same side as the flow.

```
R_COLR = {  # Colr(..., order = 21) on vaca.csv; R 4.2.3 / tram 1.0.4
    "x2": {"coef": {"x1": 1.800042}, "loglik": -1401.213542},
    "x3": {"coef": {"x1": -1.777289, "x2": -0.455282}, "loglik": -1411.901958},
}
```

```
df = pd.read_csv(NB_DATA / "vaca.csv")
# stated rather than left to the default, for the reason the markdown cell
# above the R block gives
N_COEFFS = 20  # -> Bernstein degree 21, which is tram's `order = 21`

spec_vaca = {
    "x1": ContinuousNode([SI(n_coeffs=N_COEFFS)]),
    "x2": ContinuousNode([SI(n_coeffs=N_COEFFS), LS("x1")]),
    "x3": ContinuousNode([SI(n_coeffs=N_COEFFS), LS("x1"), LS("x2")]),
}

flow_c = CausalFlowDAG(spec_vaca, seed=0)
rep = flow_c.fit_classical(df)  # logs iters / NLL / time
print(f"\n{'':<12}{'fit_classical':>14}{'R Colr':>12}{'|diff|':>10}")
loglik = {k: -v * len(df) for k, v in flow_c.nll(df).items()}  # nll() is a mean
for node, parents in flow_c.ls_coefficients().items():
    for p, w in parents.items():
        c, r = w[0], R_COLRnode[p]
        print(f"{p + ' -> ' + node:<12}{c:>14.4f}{r:>12.4f}{abs(c - r):>10.4f}")
    c, r = loglik[node], R_COLRnode
    print(f"{'logLik ' + node:<12}{c:>14.4f}{r:>12.4f}{abs(c - r):>10.4f}")
```

```
             fit_classical      R Colr    |diff|
x1 -> x2            1.7998      1.8000    0.0002
logLik x2       -1401.8717  -1401.2135    0.6581
x1 -> x3           -1.7806     -1.7773    0.0033
x2 -> x3           -0.4550     -0.4553    0.0003
logLik x3       -1410.9678  -1411.9020    0.9341
```

Why this one is not an exact-MLE check

The coefficients here agree far less closely than in Sections 0 and 2, and neither result is the more correct one. The two libraries place the Bernstein basis differently: the flow puts it on the 5%/95% quantiles and continues as a straight line outside them, which leaves a tenth of the rows in the tails, while `Colr` uses the full range of the data. The shift coefficients survive that difference in relative terms, which is why the ordered logit above finds the same two numbers without any Bernstein basis at all.

```
gaps = [
    (abs(float(w[0]) - R_COLRnode[parent]), abs(R_COLRnode[parent]))
    for node in R_COLR
    for parent, w in flow_c.ls_coefficients()[node].items()
]
claim("max |coef_flow - coef_Colr|", max(g for g, _ in gaps), 0.01)
claim("max relative coefficient gap", max(g / r for g, r in gaps), 0.02)
```

```
ok  max |coef_flow - coef_Colr|: 3.32e-03 <= 1.00e-02
ok  max relative coefficient gap: 1.87e-03 <= 2.00e-02
```

The fitted conditional density

`flow.density(df, node, grid)` evaluates $p(\text{node} = g \mid \text{pa})$ at each grid value, in closed form from the transform and with no sampling. It is the continuous counterpart of `flow.pmf`.

Two checks are worth making on it, and neither needs an external reference. A density must integrate to one over a wide enough grid, and its mode must track the conditional mean that the linear shifts imply. The first is the stronger check, because it exercises the log-determinant of the transform rather than only its location.

```
grid = np.linspace(df["x3"].min() - 2, df["x3"].max() + 2, 601)
rows = df.iloc[:200]
dens = flow_c.density(rows, "x3", grid)  # (200, 601)
mass = np.trapezoid(dens, grid, axis=1)
claim("worst |integral of the density - 1|", float(np.abs(mass - 1.0).max()), 5e-3)

# the mode should move with the parents in the direction the shifts say: the
# flow subtracts on the latent scale, so a positive weight lowers the value
modes = grid[dens.argmax(axis=1)]
shift = -(
    float(flow_c.ls_coefficients()"x3"[0]) * rows["x1"]
    + float(flow_c.ls_coefficients()"x3"[0]) * rows["x2"]
)
corr = float(np.corrcoef(modes, shift)[0, 1])
print(f"corr(mode of the fitted density, the linear predictor) = {corr:.4f}")
assert corr > 0.95, f"the density's mode does not track the shifts: r = {corr:.4f}"
```

```
ok  worst |integral of the density - 1|: 1.29e-03 <= 5.00e-03
corr(mode of the fitted density, the linear predictor) = 0.9951
```

Classical against Adam: the same optimum. A converged Adam fit with no early stopping reaches the same coefficients. The guarantees of the classical fit are determinism and the exact maximum-likelihood estimate, not speed. Which one is faster depends on the model. It wins clearly on ordinal-outcome models, as in Section 2, while on this Bernstein-heavy continuous DAG the flat directions of the basis slow L-BFGS down. Both timings are printed below rather than claimed, because a wall clock is a property of the machine.

```
flow_a = CausalFlowDAG(spec_vaca, seed=0)
t0 = time.perf_counter()
sched = PerNodePlateau(patience=15, freeze=60)
flow_a.fit(
    df,
    epochs=2000,
    batch_size=4096,
    validation_data=df,
    optimizer=per_node_adam(flow_a, lr=1e-1),
    callbacks=sched,
)
t_adam = time.perf_counter() - t0

coef_c, coef_a = flow_c.ls_coefficients(), flow_a.ls_coefficients()
print(f"{'coef':<10}{'classical':>12}{'adam':>12}{'|diff|':>10}")
for node, p in [("x2", "x1"), ("x3", "x1"), ("x3", "x2")]:
    c, aw = coef_cnode[0], coef_anode[0]
    print(f"{p}->{node:<6}{c:>12.4f}{aw:>12.4f}{abs(c - aw):>10.4f}")
print(f"\nclassical {rep['seconds']:.2f}s  vs  adam {t_adam:.1f}s")
```

```
coef         classical        adam    |diff|
x1->x2          1.7998      1.7944    0.0055
x1->x3         -1.7806     -1.7663    0.0143
x2->x3         -0.4550     -0.4474    0.0076

classical 6.42s  vs  adam 5.7s
```

2. Ordinal case (treatment effect)

For an **ordinal** outcome the classical model is the ordered-logit, which `statsmodels.OrderedModel` fits — so here the equivalence is checkable in Python. The all-`ls` stroke DAG (the synthetic `magic-mrclean/ls` cohort):

```
obs = pd.read_csv(DATA / "magic-mrclean" / "ls" / "obs.csv")

spec_stroke = {
    "Age": ContinuousNode(),
    "mRS_pre": OrdinalNode(6, [LS("Age")]),
    "NIHSSa": ContinuousNode([LS("Age"), LS("mRS_pre")]),
    "T": OrdinalNode(2, [LS("Age"), LS("mRS_pre"), LS("NIHSSa")]),
    "mRS_3m": OrdinalNode(7, [LS("Age"), LS("mRS_pre"), LS("NIHSSa"), LS("T")]),
}

flow_s = CausalFlowDAG(spec_stroke, seed=0)  # the seed validate_ls.py checks
flow_s.fit_classical(obs)
```

```
{'converged': False,
 'n_iter': 400,
 'final_nll': 10.306837499958505,
 'grad_norm': 0.017277114988926635,
 'seconds': 6.6843999120000035,
 'coefficients': {'mRS_pre': {'Age': array([0.05768183])},
  'NIHSSa': {'Age': array([-0.05582796]),
   'mRS_pre': array([ 1.73456272,  1.32279731,  1.14820555,  1.40077438,  0.31263827,
          -0.23347872])},
  'T': {'Age': array([-0.03928215]),
   'mRS_pre': array([0.67340049, 0.79612263, 0.45463112, 0.78673051, 0.15841777,
          0.26415468]),
   'NIHSSa': array([-0.0137474])},
  'mRS_3m': {'Age': array([0.05374448]),
   'mRS_pre': array([-1.362594  , -0.95564466, -0.77616902, -0.57278162,  1.14801943,
          0.41579485]),
   'NIHSSa': array([0.16336759]),
   'T': array([-0.70514631, -1.63106338])}}}
```

Refitting the same design in statsmodels

`flow.design_matrix(node, drop_first=True)` gives the encoding the flow feeds its own shifts, so handing it to `statsmodels` compares two fits of one design rather than two different designs.

```
design = flow_s.design_matrix(obs, "mRS_3m", drop_first=True)
res = OrderedModel(obs["mRS_3m"].astype(int), design, distr="logit").fit(
    method="bfgs", disp=False
)

fitted = flow_s.ls_coefficients()["mRS_3m"]
rows = [
    ("Age", float(fitted"Age"), res.params["Age"]),
    ("NIHSSa", float(fitted"NIHSSa"), res.params["NIHSSa"]),
    ("T (1 vs 0)", float(fitted"T" - fitted"T"), res.params["T[1]"]),
]
print(f"{'coefficient':<14}{'fit_classical':>14}{'statsmodels':>13}{'|diff|':>9}")
for name, a_, b_ in rows:
```

```
    print(f"{name:<14}{a_:>14.4f}{b_:>13.4f}{abs(a_ - b_):>9.4f}")
```

```
coefficient    fit_classical  statsmodels   |diff|
Age                   0.0537       0.0526   0.0012
NIHSSa                0.1634       0.1630   0.0004
T (1 vs 0)           -0.9259      -0.9424   0.0165
```

Standard errors, and what is actually weakly identified

The helper from Section 1 needs no change: give it the node, and it builds the contrast for each ordinal parent. `statsmodels` reports the same quantities, so every number below has an external check.

```
table_s, diag_s = ls_conf_int(flow_s, "mRS_3m", obs)
ci_sm = res.conf_int()
print(
    f"{'':<12}{'flow SE':>9}{'sm SE':>8}{'|est|/SE':>10}   "
    f"{'flow 95%':>18}{'statsmodels 95%':>20}"
)
for term, key in [("Age", "Age"), ("NIHSSa", "NIHSSa"), ("T (1 vs 0)", "T[1]")]:
    r = table_s.loc[term]
    print(
        f"{term:<12}{r['se']:>9.4f}{res.bse[key]:>8.4f}"
        f"{abs(r['estimate']) / r['se']:>10.2f}   "
        f"[{r['lower']:+.3f}, {r['upper']:+.3f}]  "
        f"[{ci_sm.loc[key, 0]:+.3f}, {ci_sm.loc[key, 1]:+.3f}]"
    )
print(
    f"\n{diag_s['n_params']} parameters, {diag_s['n_flat']} without curvature"
    " — one per one-hot parent, mRS_pre and T"
)
```

```
              flow SE   sm SE  |est|/SE             flow 95%     statsmodels 95%
Age            0.0050  0.0050     10.83   [+0.044, +0.063]  [+0.043, +0.062]
NIHSSa         0.0090  0.0090     18.08   [+0.146, +0.181]  [+0.145, +0.181]
T (1 vs 0)     0.1408  0.1407      6.58   [-1.202, -0.650]  [-1.218, -0.667]

16 parameters, 2 without curvature — one per one-hot parent, mRS_pre and T
```

```
se_gap = max(
    abs(float(table_s.loc[term, "se"]) - float(res.bse[key]))
    for term, key in [("Age", "Age"), ("NIHSSa", "NIHSSa"), ("T (1 vs 0)", "T[1]")]
)
claim("max |SE_flow - SE_statsmodels|", se_gap, 1e-3)

# the ranking is the claim, not the ratio: the treatment is the best determined
# coefficient of the three, and which one that is must not depend on a seed
ratios = {
    term: abs(float(table_s.loc[term, "estimate"]) / float(table_s.loc[term, "se"]))
    for term in ("Age", "NIHSSa", "T (1 vs 0)")
}
print("|estimate| / SE:", {k: round(v, 2) for k, v in ratios.items()})
# The claim the next section rests on: none of these three is the weak one, so
# the weak coefficient has to be found elsewhere. Two standard errors from zero
# is the conventional line.
assert min(ratios.values()) > 2.0, f"one of these is not well determined: {ratios}"
```

```
ok  max |SE_flow - SE_statsmodels|: 4.80e-05 <= 1.00e-03
```

```
|estimate| / SE: {'Age': 10.83, 'NIHSSa': 18.08, 'T (1 vs 0)': 6.58}
```

The standard errors agree with `statsmodels`, and all three coefficients sit well clear of zero. The treatment effect is therefore not the uncertain one here, whatever the ordering of the three happens to be.

To find the weak one, look at every `mRS_pre` level instead of only the first, and print how many rows carry it.

```
hess_s, _, offsets_s = observed_information(flow_s, "mRS_3m", obs)
cov_s = np.linalg.pinv(hess_s, rcond=1e-7)
at = offsets_s[next(n for n in offsets_s if n.startswith("shifts.mRS_pre"))]
w_pre = flow_s.ls_coefficients()"mRS_3m"
counts = obs["mRS_pre"].value_counts()

print(f"{'level':>6}{'rows':>7}{'estimate':>10}{'SE':>9}{'|est|/SE':>10}   95% CI")
for level in range(1, 6):
    c = np.zeros(hess_s.shape[0])
    c[at], c[at + level] = -1.0, 1.0  # w[level] - w[0]
    est = w_pre[level] - w_pre[0]
    se = float(np.sqrt(c @ cov_s @ c))
    print(
        f"{level:>6}{counts.get(level, 0):>7}{est:>10.4f}{se:>9.4f}"
        f"{abs(est) / se:>10.2f}   [{est - 1.96 * se:+.3f}, {est + 1.96 * se:+.3f}]"
    )
```

```
 level   rows  estimate       SE  |est|/SE   95% CI
     1    273    0.4069   0.1340      3.04   [+0.144, +0.670]
     2    166    0.5864   0.1632      3.59   [+0.267, +0.906]
     3    107    0.7898   0.1922      4.11   [+0.413, +1.166]
     4     41    2.5106   0.4158      6.04   [+1.696, +3.326]
     5      7    1.7784   0.8762      2.03   [+0.061, +3.496]
```

The rarest level has the widest interval. That is what a weakly identified coefficient looks like, and the second column shows the cause: too few rows carry it. The relationship is the claim, so the cell below checks it rather than quoting one run's numbers.

```
by_level = {}
for level in range(1, 6):
    c = np.zeros(hess_s.shape[0])
    c[at], c[at + level] = -1.0, 1.0
    by_level[level] = (
        int(counts.get(level, 0)),
        float(np.sqrt(c @ cov_s @ c)),
    )
rarest = min(by_level, key=lambda k: by_levelk)
widest = max(by_level, key=lambda k: by_levelk)
print(f"rarest level: {rarest}   widest standard error: level {widest}")
assert rarest == widest, (
    f"level {rarest} has the fewest rows but level {widest} has the widest "
    "interval, so row count is no longer what drives the uncertainty"
)
```

```
rarest level: 5   widest standard error: level 5
```

The same slack explains the `|diff|` column above. The gap between the flow and `statsmodels` is not evidence about identification. It is the optimizer stopping while the likelihood is still flat, and the displacement that causes is $c^\top I^{+} \nabla \ell$. The cell below computes that displacement and checks it against the observed difference, which turns a hand-checked coincidence into a pinned identity.

Age looks worse than T only because its standard error is smaller, so the same numerical slack is a larger fraction of it.

```
print(f"|grad| at the classical optimum: {diag_s['grad_norm']:.1e}")
for term, key in [("Age", "Age"), ("NIHSSa", "NIHSSa")]:
    observed = abs(float(table_s.loc[term, "estimate"]) - float(res.params[key]))
    print(
        f"{term:<8} |flow - statsmodels| = {observed:.2e}"
        f"   = {observed / float(table_s.loc[term, 'se']):.0%} of one SE"
    )
    claim(f"{term}: gap against statsmodels", observed, 5e-2)
#
# `experiments/misc/validate_ls.py` runs this comparison as a checked
# experiment, and adds R (`MASS::polr` / `tram`) and the treatment effect. Its
# docstring records the same finding about `mRS_pre` level 5.
```

```
|grad| at the classical optimum: 1.2e+01
Age      |flow - statsmodels| = 1.19e-03   = 24% of one SE
ok  Age: gap against statsmodels: 1.19e-03 <= 5.00e-02
NIHSSa   |flow - statsmodels| = 3.78e-04   = 4% of one SE
ok  NIHSSa: gap against statsmodels: 3.78e-04 <= 5.00e-02
```

3. Warm-start handoff: classical fit → further training

`fit_classical` leaves the model at the MLE in float32, ready for any normal operation. Two things to verify:

1. the float64→float32 round-trip didn't move the coefficients, and
2. continuing with `fit()` from the classical solution *stays put* — confirming it really is the optimum (and showing the classical fit as a fast, principled initialization for further or richer training).

```
before = {k: v.copy() for k, v in flow_s.ls_coefficients()["mRS_3m"].items()}

# continue training from the classical solution with a gentle Adam phase
flow_s.fit(obs, epochs=300, learning_rate=1e-3, batch_size=256)
after = flow_s.ls_coefficients()["mRS_3m"]

print("coefficient drift after 300 more Adam epochs from the classical MLE:")
for p in ["Age", "NIHSSa", "T"]:
    d = float(np.abs(after[p] - before[p]).max())
    print(f"  {p:<8} max|Δ| = {d:.4f}")
print("\n-> small drift = the classical fit was already at the optimum;")
print("   fit_classical is a valid warm start for continued / flexible training.")
```

```
coefficient drift after 300 more Adam epochs from the classical MLE:
  Age      max|Δ| = 0.0002
  NIHSSa   max|Δ| = 0.0003
  T        max|Δ| = 0.0238

-> small drift = the classical fit was already at the optimum;
   fit_classical is a valid warm start for continued / flexible training.
```

TRAM-DAG: one causal model, all three rungs

A TRAM-DAG (Sick & Dürr, CLeaR 2025) is one normalizing flow wired like the adjacency matrix of a causal DAG. Fit it once on observational data. Then answer all three rungs of Pearl's ladder

with the same fitted model:

rung	query	call
L1 association	$p(x)$, sampling	`flow.log_prob(df)`, `flow.sample(n)`
L2 intervention	$p(x \mid do(x_j{=}a))$	`flow.sample(n, do={...})`
L3 counterfactual	what would x_i have been, had x_j been a?	`u = flow.abduct(df)`, then `flow.sample(do={...}, u=u)`

This notebook uses the first benchmark of the paper. It is a three-variable process with a bimodal source, and its ground truth is known analytically. Every claim below is therefore a computed check, not a statement. A failed check stops the notebook.

The model theory is in `docs/model.md`. How to read a fitted model is in `docs/interpretation.md`.

The notebook runs on CPU in well under five minutes.

```
import importlib.util
import subprocess
import sys
import time

if importlib.util.find_spec("tramdag") is None:  # Colab: install from PyPI
    subprocess.check_call(
        [sys.executable, "-m", "pip", "install", "-q", "tramdag[plots]"]
    )
    # to track active development instead of the latest release, use:
    #   pip install "git+https://github.com/tensorchiefs/tramdag.git@main"

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import torch

from tramdag import LS, CausalFlowDAG, ContinuousNode, I, plot_dag
from tramdag.callbacks import PerNodePlateau, per_node_adam
from tramdag.plots import plot_marginals, plot_training

DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
plt.rcParams["figure.dpi"] = 110
t_total = time.perf_counter()
print(f"torch {torch.__version__}  device: {DEVICE}")
```

```
torch 2.13.0+cu130  device: cpu
```

1. The benchmark process

The process comes from Sánchez-Martín et al. 2022, App. E.1, and from App. C.1 of the TRAM-DAG paper:

$$
\begin{aligned}
x_1 &\sim \tfrac{1}{2}\,\mathcal{N}(-2,\,1.5) + \tfrac{1}{2}\,\mathcal{N}(1.5,\,1) \quad \text{(bimodal source)} \\
x_2 &= -x_1 + \mathcal{N}(0,1) \\
x_3 &= x_1 + 0.25\,x_2 + \mathcal{N}(0,1)
\end{aligned}
$$

The DAG is $x_1 \to x_2$, $x_1 \to x_3$, $x_2 \to x_3$. The source is bimodal, which is the part that a location-scale model cannot fit. Section 6 measures that.

```
# The process is written out here, so the notebook needs nothing but tramdag.
# draw_latents and simulate stay separate on purpose. Keeping the noise lets
# section 5 intervene on the same individuals, which is what makes an
# individual counterfactual checkable.


def draw_latents(n, rng):
    """Draw every noise variable of the process, n rows each."""
    return {
        "x1_mix": rng.uniform(size=n),  # which mixture component
        "x1_a": rng.normal(size=n),  # the N(-2, 1.5) branch
        "x1_b": rng.normal(size=n),  # the N(1.5, 1) branch
        "x2": rng.normal(size=n),
        "x3": rng.normal(size=n),
    }


def simulate(latents, do=None):
    """Run the process forward. A variable named in `do` is clamped instead."""
    do = do or {}
    n = len(latents["x2"])

    if "x1" in do:
        x1 = np.full(n, float(do["x1"]))
    else:
        x1 = np.where(
            latents["x1_mix"] < 0.5,
            -2.0 + np.sqrt(1.5) * latents["x1_a"],
            1.5 + latents["x1_b"],
        )

    x2 = np.full(n, float(do["x2"])) if "x2" in do else -x1 + latents["x2"]
    x3 = x1 + 0.25 * x2 + latents["x3"]
    return pd.DataFrame({"x1": x1, "x2": x2, "x3": x3})


def sample_dgp(n, seed, do=None):
    """Draw n fresh rows, optionally under an intervention."""
    return simulate(draw_latents(n, np.random.default_rng(seed)), do)


N = 20_000
df = sample_dgp(N, seed=43)
train, val = df.iloc[: N - 2_000], df.iloc[N - 2_000 :]

fig, ax = plt.subplots(figsize=(5.5, 3))
ax.hist(df["x1"], bins=80, density=True, alpha=0.7)
ax.set_title(f"the source $x_1$ has two modes ({N:,} rows)")
ax.set_xlabel("$x_1$")
fig.tight_layout()
plt.show()
```

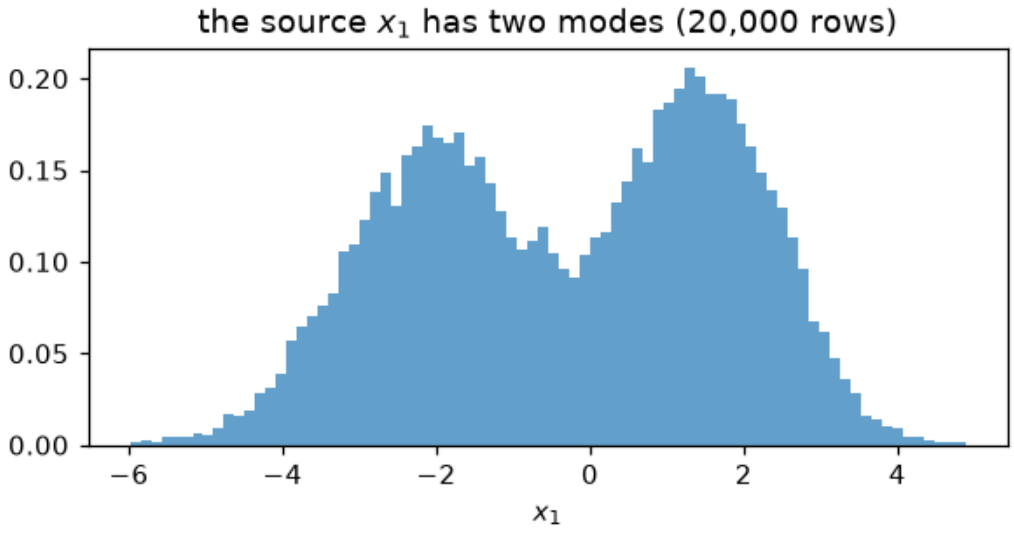


Figure 2: png

2. The spec is the DAG, and one fit

A spec is one entry per node. Each entry names the terms through which the parents enter. `I(...)` puts the parents in control of the transform parameters, which is the most flexible term. Section 7 writes the same DAG with interpretable terms instead.

The joint negative log-likelihood decomposes per node, so one Adam fits every node at once. `fit` is a plain loop. `validation_data=` adds a per-epoch validation score in `flow.history["val"]`, and `verbose=` prints progress. The strategy attaches through `callbacks=`. Here `per_node_adam` gives each node its own parameter group, and `PerNodePlateau` lowers that node's rate when its own validation score stops improving. It freezes the node once the rate is low and flat. The fit ends when the last node freezes, so `epochs=200` is a ceiling and not a budget.

`notebooks/training_strategies.py` works through the other recipes.

```
spec = {
    "x1": ContinuousNode(),  # the source has no parents
    "x2": ContinuousNode(I("x1")),
    "x3": ContinuousNode(I("x1", "x2")),
}
plot_dag(spec)
plt.show()
```

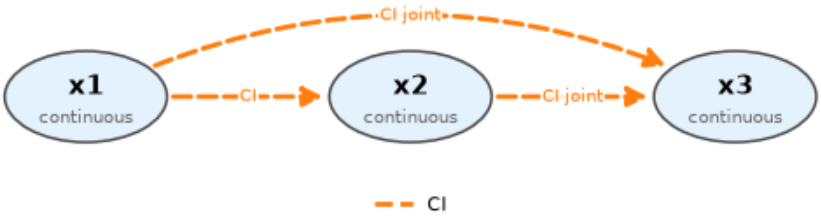


Figure 3: png

```
torch.manual_seed(0)
flow = CausalFlowDAG(spec, device=DEVICE)
sched = PerNodePlateau(patience=10, freeze=40)

t0 = time.perf_counter()
flow.fit(
    train,
    epochs=200,
    batch_size=2048,
    validation_data=val,
    verbose=50,
    optimizer=per_node_adam(flow, lr=1e-1),
    callbacks=sched,
)
t_fit = time.perf_counter() - t0
epochs_used = len(flow.history["val"])
print(f"\nfitted on {DEVICE} in {t_fit:.1f}s, {epochs_used} epochs")
print(f"each node froze at epoch: {dict(sorted(sched.frozen.items()))}")

# The ceiling must not bind. If it does, the plateau rule never finished and
# the numbers below describe an unconverged fit.
assert epochs_used < 200, (
    "the fit used all 200 epochs, so no node froze and the plateau rule did "
    "not self-stop; raise the ceiling before trusting anything below"
)
```

```
epoch 50/200  train 4.8922  val 4.9353


epoch 83/200  train 4.8918  val 4.9354

fitted on cpu in 17.0s, 83 epochs
each node froze at epoch: {'x1': 63, 'x2': 83, 'x3': 55}
```

```
plot_training(flow, frozen=sched.frozen)
plt.show()
```

3. Rung 1: the observational fit

Samples from the fitted flow must reproduce the observed joint distribution. The marginals are the picture. The correlation matrix is the check, because it also covers the dependence between variables, which a marginal plot does not show.

```
plot_marginals(flow, df, seed=1)
plt.show()
```

```
samp = flow.sample(len(df), seed=1)
corr_gap = float(np.abs(samp.corr().to_numpy() - df.corr().to_numpy()).max())
print(f"largest absolute difference between correlation matrices: {corr_gap:.4f}")
assert corr_gap < 0.05, f"the sampled dependence structure is off by {corr_gap:.4f}"
```

```
largest absolute difference between correlation matrices: 0.0094
```

4. Rung 2: interventions

`do=` mutilates the graph. It clamps x_2, cuts the edge into it, and resamples everything downstream. Under the process, $x_3 \mid do(x_2{=}a) = x_1 + 0.25a + \mathcal{N}(0,1)$, so $\mathbb{E}[x_3] = -0.25 + 0.25a$ exactly. That analytic target is what makes an error visible.

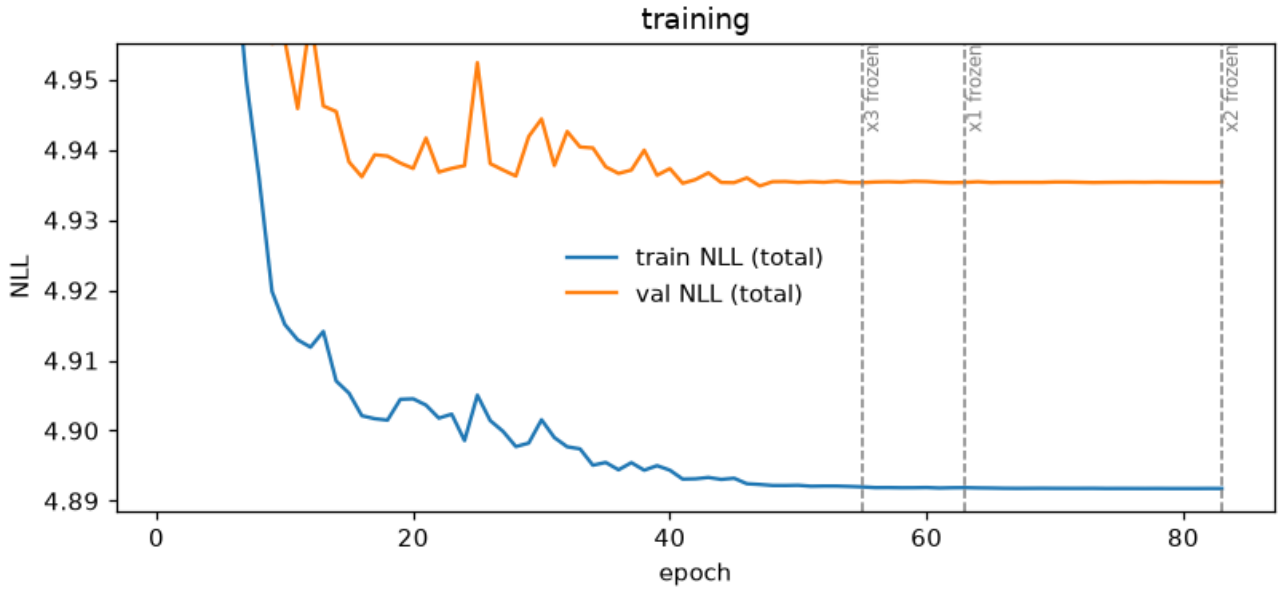


Figure 4: png

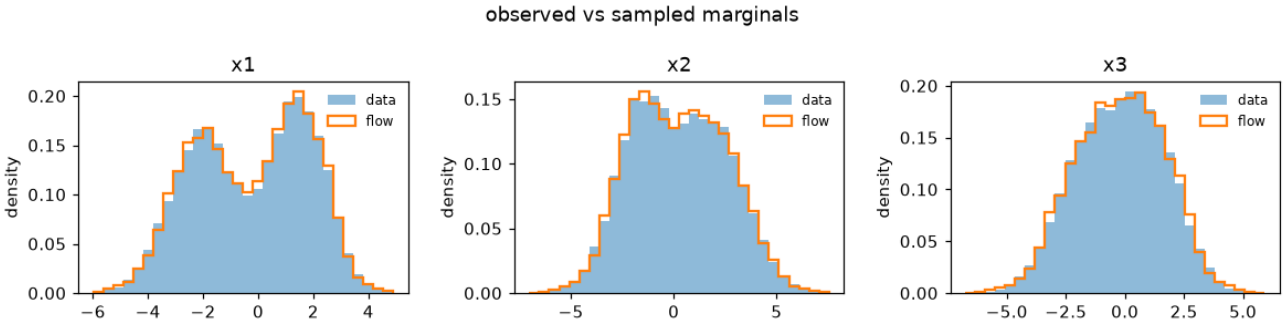


Figure 5: png

```
fig, axes = plt.subplots(1, 3, figsize=(11, 3.2), sharey=True)
errors = []
print("E[x3 | do(x2=a)]     analytic    TRAM-DAG      error")
for ax, a in zip(axes, (-3.0, -1.0, 0.0), strict=True):
    truth = sample_dgp(len(df), seed=543, do={"x2": a})
    fitted = flow.sample(len(df), do={"x2": a}, seed=2)
    bins = np.linspace(truth["x3"].quantile(0.001), truth["x3"].quantile(0.999), 70)
    ax.hist(truth["x3"], bins=bins, density=True, alpha=0.5, label="process")
    ax.hist(
        fitted["x3"], bins=bins, density=True, histtype="step", lw=1.6, label="TRAM-DAG"
    )
    ax.set_title(f"$p(x_3 \\mid do(x_2={a:+.0f}))$")

    analytic = -0.25 + 0.25 * a
    got = float(fitted["x3"].mean())
    errors.append(abs(got - analytic))
    print(
        f"   a = {a:+.0f}:            {analytic:+.4f}     {got:+.4f}     {errors[-1]:.4f}"
    )
axes[0].legend()
fig.tight_layout()
plt.show()

# The bound is about three times the largest error measured while writing this
# notebook (0.048), which leaves room for another machine and another draw.
assert max(errors) < 0.15, f"interventional mean off by {max(errors):.4f}"
```

```
E[x3 | do(x2=a)]     analytic    TRAM-DAG      error


   a = -3:            -1.0000     -1.0469     0.0469


   a = -1:            -0.5000     -0.5211     0.0211


   a = +0:            -0.2500     -0.2807     0.0307
```

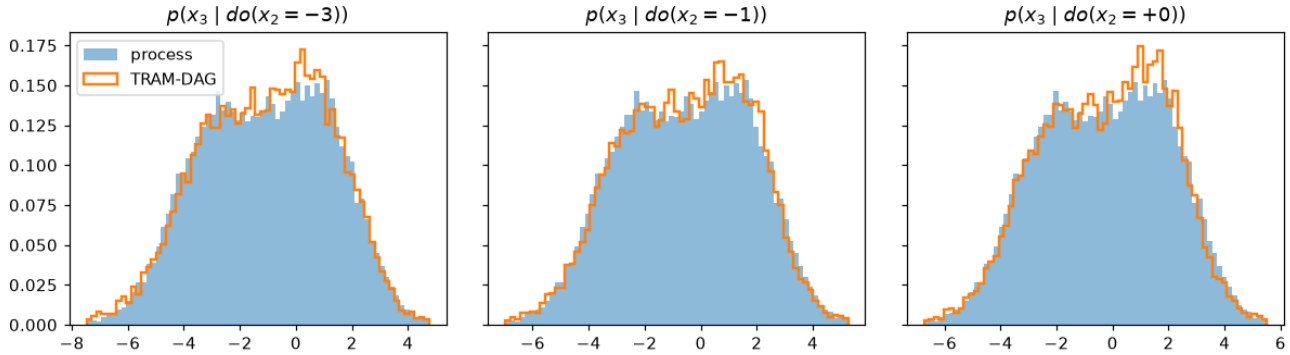


Figure 6: png

5. Rung 3: individual counterfactuals

Take 1,000 held-out individuals. Abduction inverts the flow and recovers the latent noise of each one. That noise is everything about the individual that the model does not attribute to the parents. Pushing it back through the flow must return the observed row, which is the first check.

Action and prediction then rerun each individual under $do(x_1 = 0)$ with the same noise. The process keeps its own noise, so the true individual counterfactual is known and the flow can be scored per individual. Real data never permits this check, because only one of the two outcomes exists.

```
lat = draw_latents(1_000, np.random.default_rng(7))
factual = simulate(lat)  # the same noise on both sides,
cf_true = simulate(lat, do={"x1": 0.0})  # so these are true counterfactuals

u = flow.abduct(factual)
recon_err = float(np.abs(flow.sample(u=u).to_numpy() - factual.to_numpy()).max())
print(f"abduction then reconstruction: max |error| = {recon_err:.2e}")
# This inverts a monotone transform in float32, so the bound is a precision
# bound rather than a statistical one.
assert recon_err < 1e-4, f"abduction did not round-trip: {recon_err:.2e}"

cf_flow = flow.sample(do={"x1": 0.0}, u=u)
fig, axes = plt.subplots(1, 2, figsize=(9, 3.6))
for ax, c in zip(axes, ["x2", "x3"], strict=True):
    r = float(np.corrcoef(cf_true[c], cf_flow[c])[0, 1])
    ax.scatter(cf_true[c], cf_flow[c], s=4, alpha=0.4)
    lims = [cf_true[c].min(), cf_true[c].max()]
    ax.plot(lims, lims, "k--", lw=1)
    ax.set_title(f"${c[0]}_{c[1]}$ under $do(x_1{{=}}0)$   (r = {r:.4f})")
    ax.set_xlabel("true, shared noise")
    ax.set_ylabel("TRAM-DAG")
    assert r > 0.99, f"counterfactual {c} correlates only {r:.4f} with the truth"
fig.suptitle("L3: individual counterfactuals, scored one unit at a time")
fig.tight_layout()
plt.show()
```

```
abduction then reconstruction: max |error| = 5.50e-06
```

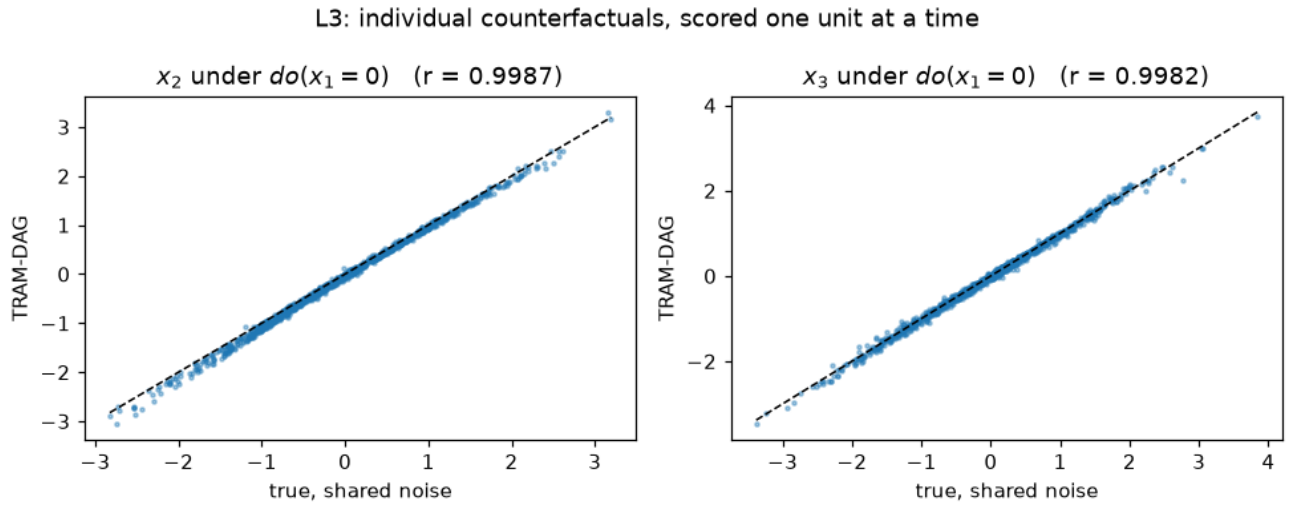


Figure 7: png

6. Why the Bernstein transform

Every continuous node owns a monotone transform, and that transform carries the distributional flexibility. One argument switches it: `"bernstein"` is the default, `"spline"` is a monotone rational-quadratic spline, and `"affine"` is location-scale only, which makes each node-conditional a logistic distribution.

The affine model is the informative comparison here, because the source is bimodal and a location-scale transform cannot produce two modes. A smaller subsample is enough to show it.

The spline result is a separate story about how zuko extrapolates outside its domain. `docs/zuko-upstream.md` covers it, and this notebook does not spend a third fit on it.

```
sub = train.iloc[:5_000]
affine_spec = {
    "x1": ContinuousNode([I(transform="affine")]),
    "x2": ContinuousNode(I("x1", transform="affine")),
    "x3": ContinuousNode(I("x1", "x2", transform="affine")),
}
torch.manual_seed(0)
affine = CausalFlowDAG(affine_spec, device=DEVICE)
affine.fit(sub, epochs=150, learning_rate=1e-2, batch_size=1024, validation_data=val)

nll_bernstein = sum(flow.nll(val).values())
nll_affine = sum(affine.nll(val).values())
print(f"held-out NLL, lower is better:  bernstein {nll_bernstein:.4f}")
print(f"                                affine    {nll_affine:.4f}")
```

```
held-out NLL, lower is better:  bernstein 4.9354
                                affine    5.0662
```

```
bins = np.linspace(df["x1"].quantile(0.001), df["x1"].quantile(0.999), 70)
fig, ax = plt.subplots(figsize=(7, 3.4))
ax.hist(df["x1"], bins=bins, density=True, alpha=0.4, color="gray", label="data")
for name, model, color in [("bernstein", flow, "C3"), ("affine", affine, "C2")]:
    ax.hist(
        model.sample(5_000, seed=3)["x1"],
        bins=bins,
        density=True,
        histtype="step",
        lw=1.8,
        color=color,
        label=name,
    )
ax.set_title("a location-scale transform cannot produce two modes")
ax.set_xlabel("$x_1$")
ax.legend()
fig.tight_layout()
plt.show()

# The measured gap was 0.128 nats. The bound keeps a clear margin.
assert nll_affine > nll_bernstein + 0.05, (
    f"the affine model scored {nll_affine:.4f} against {nll_bernstein:.4f}, "
    "so this comparison no longer shows what it claims"
)
```

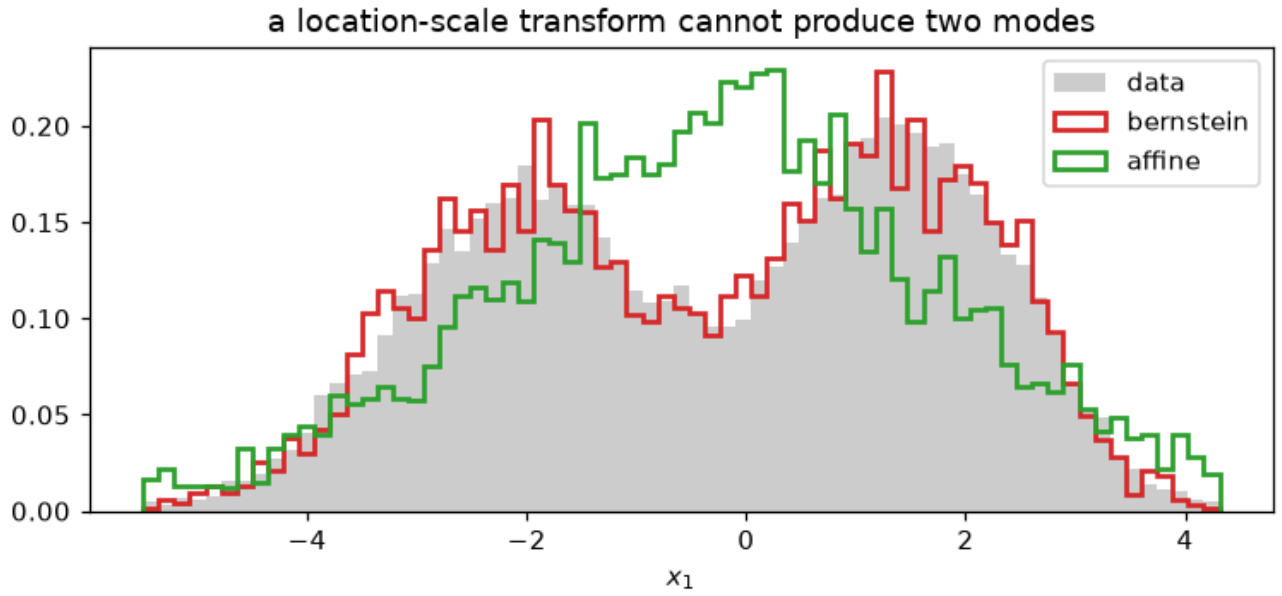


Figure 8: png

7. The same DAG, written to be interpretable

Everything above used `I(...)`, which gives a node no single coefficient to read. That is the flexible end of the family. Writing an edge as `LS("parent")` instead puts one weight on that edge, and the weight is readable after training.

An all-LS model is a classical transformation model per node, so `fit_classical` applies. It is a deterministic full-batch L-BFGS fit in float64, with no schedule and no minibatch noise.

Two things are worth reading off the result. The flow adds its shift on the latent scale, so a parent that raises a child gets a negative weight. And the weights carry the latent scale, so their ratio is what compares against the process, not their size. Here $x_3 = x_1 + 0.25x_2$, so the ratio should be near $1/0.25 = 4$.

`docs/interpretation.md` covers the ordinal case, where a weight is a log-odds ratio.

```
ls_spec = {
    "x1": ContinuousNode(),
    "x2": ContinuousNode(LS("x1")),
    "x3": ContinuousNode(LS("x1") + LS("x2")),
}
torch.manual_seed(0)
ls_flow = CausalFlowDAG(ls_spec, device=DEVICE)
report = ls_flow.fit_classical(sub)
coefs = ls_flow.ls_coefficients()

b1 = float(coefs"x3"[0])
b2 = float(coefs"x3"[0])
ratio = b1 / b2
nll_ls = sum(ls_flow.nll(val).values())
print(f"exact MLE in {report['seconds']:.1f}s\n")
print(f"x3 <- x1 : {b1:+.4f}")
print(f"x3 <- x2 : {b2:+.4f}")
print(f"ratio    : {ratio:.3f}   (the process has 1 / 0.25 = 4)")
print(f"\nheld-out NLL: all-LS {nll_ls:.4f}, flexible {nll_bernstein:.4f}")
```

```
# The bound is deliberately wide: this is one draw at one seed, and the
# measured deviation was 0.03.
assert abs(ratio - 4.0) < 0.25, f"the coefficient ratio came out {ratio:.3f}"
```

```
exact MLE in 18.5s

x3 <- x1 : -1.7361
x3 <- x2 : -0.4308
ratio    : 4.030   (the process has 1 / 0.25 = 4)

held-out NLL: all-LS 4.9409, flexible 4.9354
```

The all-LS model pays a little likelihood for being readable. The price is small here because the edges of this process really are linear, so the flexible terms have little extra structure to find. On a process with curved edges the gap is larger. `experiments/paper/` measures that case.

What this notebook checked

rung	call	checked against
L1	`flow.sample(n)`	the correlation matrix of the data
L2	`flow.sample(n, do=...)`	the analytic mean $-0.25 + 0.25a$
L3	`flow.abduct(df)`, then `flow.sample(do=..., u=u)`	the true per-unit counterfactual
transform	`I(transform="affine")`	held-out likelihood against Bernstein
interpretability	`LS(...)`, `fit_classical`, `ls_coefficients()`	the coefficient ratio of the process

Where to go next:

- `docs/model.md`, the model and its notation.
- `docs/interpretation.md`, reading a fitted model, including ordinal nodes and log-odds ratios.
- `notebooks/training_strategies.py`, the fitting API and every shipped strategy.
- `notebooks/classical_fit_tram_dag.py`, the agreement with `statsmodels` and R.
- `notebooks/varying_coefficients.py`, effects that vary with a covariate.
- repo · paper

```
elapsed = time.perf_counter() - t_total
print(f"whole notebook: {elapsed:.0f}s")
assert elapsed < 300, f"the notebook took {elapsed:.0f}s, over its five-minute claim"
```

```
whole notebook: 41s
```

Training strategies: driving `fit` from the API side

`fit` is one minibatch Adam loop that keeps the final weights. It owns three things only: the loop, the per-epoch validation score, and progress printing. Everything else is the caller's, through `optimizer=` and `callbacks=`.

This notebook runs every shipped strategy on one small workload, so the comparison is like for like, and prints what each one leaves behind in `flow.history`. It is about mechanics, not speed. For measured time-to-target on real workloads see `docs/training-speed.md`. For which strategy to pick see the table in `docs/fitting.md`.

This notebook is executed on every documentation build, so its recipes are checked against the current API. The snippets in `docs/fitting.md` are a quick reference copied from here; if the two ever disagree, this file is right.

```
import time
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import torch

from tramdag import CausalFlowDAG, ContinuousNode, I
from tramdag.callbacks import Callback, EarlyStopping, PerNodePlateau, per_node_adam
from tramdag.plots import plot_training

plt.rcParams["figure.dpi"] = 110

# repo-relative data, whether the notebook runs from the repo root or notebooks/
REPO = next(
    p for p in [Path.cwd(), *Path.cwd().parents] if (p / "pyproject.toml").exists()
)

# The tracked 1000-row VACA sample, the same benchmark the Colab demo uses.
df = pd.read_csv(REPO / "notebooks" / "data" / "vaca.csv")
train, val = df.iloc[:900], df.iloc[900:]

SPEC = {
    "x1": ContinuousNode(),
    "x2": ContinuousNode(I("x1")),
    "x3": ContinuousNode(I("x1", "x2")),
}
CEILING = 300  # epoch ceiling for every recipe below
scoreboard = []


def build():
    """Give a fresh unfitted flow, seeded so every recipe starts identically."""
    torch.manual_seed(0)
    return CausalFlowDAG(SPEC)


def record(name, flow, seconds):
    """Add one finished recipe to the scoreboard and print its line."""
    nll = sum(flow.nll(val).values())
    epochs = len(flow.history["train"])
    scoreboard.append(
        {"strategy": name, "val_nll": nll, "epochs": epochs, "seconds": seconds}
    )
    print(f"{name:22s} val NLL {nll:.4f}   {epochs:4d} epochs   {seconds:5.1f}s")
    return nll


print(f"{len(train)} train rows, {len(val)} validation rows")
```

```
900 train rows, 100 validation rows
```

1. What `fit` records without any callback

Pass `validation_data=` and `fit` scores the validation set once per epoch, centrally. Every shipped callback reads that one computation rather than repeating it. Three keys appear in `flow.history`:

- `train`, one dict of per-node negative log-likelihood per epoch,
- `val`, the same for the validation rows, only when validation is on,
- `lr`, the optimizer's rate after each epoch, recorded after the callbacks ran, so a schedule's decision for that epoch is what gets stored.

The per-node shape is not decoration. The joint likelihood decomposes per node, so a per-node score says which node is still improving.

```
flow = build()
flow.fit(train, epochs=20, batch_size=256, validation_data=val)

print("history keys:", sorted(flow.history))
print("epochs recorded:", len(flow.history["train"]))
print("per-node train NLL, last epoch:")
for node, value in flow.history"train".items():
    print(f"    {node}: {value:.4f}")
print("learning rate, last epoch:", flow.history"lr")

assert len(flow.history["train"]) == len(flow.history["val"]) == 20
assert set(flow.history"train") == set(SPEC)
```

```
history keys: ['lr', 'train', 'val']
epochs recorded: 20
per-node train NLL, last epoch:
    x1: 2.6743
    x2: 1.4033
    x3: 1.4267
learning rate, last epoch: 0.01
```

Without `validation_data=` there is no `val` key, and the shipped callbacks refuse rather than guess.

```
bare = build()
bare.fit(train, epochs=5, batch_size=256)
print("history keys without validation:", sorted(bare.history))
assert "val" not in bare.history

try:
    build().fit(train, epochs=5, batch_size=256, callbacks=EarlyStopping())
except RuntimeError as err:
    print("\nEarlyStopping without validation refuses:\n   ", err)
else:
    raise AssertionError("EarlyStopping should refuse without validation data")
```

```
history keys without validation: ['lr', 'train']



EarlyStopping without validation refuses:
   this callback reads flow.history['val'] — pass validation_data= or validation_split= to fit()
```

2. Plain Adam

One rate, one loop, and the weights at the last epoch. This is the baseline every other recipe is measured against. An all-LS model trained this way to convergence reaches the classical

maximum-likelihood estimate, which `classical_fit_tram_dag.py` checks against `statsmodels` and R.

```
flow = build()
t0 = time.perf_counter()
flow.fit(train, epochs=CEILING, batch_size=256, learning_rate=1e-2, validation_data=val)
nll_plain = record("plain Adam", flow, time.perf_counter() - t0)
history_plain = [sum(d.values()) for d in flow.history["val"]]
```

```
plain Adam              val NLL 4.9958    300 epochs    11.5s
```

3. Two phases

Calling `fit` again continues training. The optimizer is rebuilt unless you pass your own, so a second call at a lower rate is a coarse-then-fine schedule with no callback at all. History accumulates across the calls.

```
flow = build()
t0 = time.perf_counter()
flow.fit(train, epochs=200, batch_size=256, learning_rate=1e-2, validation_data=val)
after_coarse = sum(flow.nll(val).values())
flow.fit(train, epochs=100, batch_size=256, learning_rate=1e-3, validation_data=val)
nll_two_phase = record("two phases", flow, time.perf_counter() - t0)

print(f"    after the coarse phase: {after_coarse:.4f}")
print(f"    after the fine phase:   {nll_two_phase:.4f}")
assert len(flow.history["train"]) == 300, "the second fit did not continue the history"
```

```
two phases              val NLL 4.9781    300 epochs    11.3s
    after the coarse phase: 5.0057
    after the fine phase:   4.9781
```

4. `EarlyStopping`: keep the best weights

The loop keeps the final weights, which is what makes the classical agreement exact. A flexible model does not always want that: it can reach a better validation score part way through and then drift. `EarlyStopping` snapshots the best epoch and loads it back at the end of the fit.

The check below is the one a static code block cannot make. After the fit, the model's validation score equals the *minimum* over the recorded epochs, not the last one.

```
flow = build()
t0 = time.perf_counter()
stopper = EarlyStopping()  # restore_best is on by default, no patience
flow.fit(
    train,
    epochs=CEILING,
    batch_size=256,
    learning_rate=1e-2,
    validation_data=val,
    callbacks=stopper,
)
nll_best = record("EarlyStopping", flow, time.perf_counter() - t0)

recorded = [sum(d.values()) for d in flow.history["val"]]
print(f"    best epoch {stopper.best_epoch} of {len(recorded)}")
print(f"    minimum recorded: {min(recorded):.4f}")
print(f"    last epoch:       {recorded[-1]:.4f}")
```

```
assert abs(nll_best - min(recorded)) < 1e-4, (
    "the restored weights do not match the best recorded epoch"
)
```

```
EarlyStopping          val NLL 4.9564    300 epochs    11.4s
    best epoch 180 of 300
    minimum recorded: 4.9564
    last epoch:       4.9958
```

5. `EarlyStopping(patience=)`: stop as well as restore

Without `patience` the fit spends its whole budget and only the restoration happens. With it, the fit also stops once the best epoch is that many epochs old.

```
flow = build()
t0 = time.perf_counter()
stopper = EarlyStopping(patience=25)
flow.fit(
    train,
    epochs=CEILING,
    batch_size=256,
    learning_rate=1e-2,
    validation_data=val,
    callbacks=stopper,
)
record("EarlyStopping(25)", flow, time.perf_counter() - t0)

spent = len(flow.history["train"])
print(f"    stopped after {spent} of {CEILING} epochs, best was {stopper.best_epoch}")
assert spent < CEILING, "patience never triggered, so the ceiling bound instead"
assert spent - stopper.best_epoch >= 25
```

```
EarlyStopping(25)      val NLL 4.9564    205 epochs     7.9s
    stopped after 205 of 300 epochs, best was 180
```

6. Per-node rates: `per_node_adam` with `PerNodePlateau`

The per-node losses have independent gradients, so a rate per node is exactly independent per-node training. `per_node_adam` builds an Adam with one tagged parameter group per node. `PerNodePlateau` then decays each node's rate on that node's own validation score and freezes the node once the rate is low and flat. The fit stops when the last node freezes.

Do not attach a torch scheduler to the same optimizer. Two controllers would steer the same group rates against each other.

```
flow = build()
t0 = time.perf_counter()
plateau = PerNodePlateau(patience=10, freeze=30)
flow.fit(
    train,
    epochs=CEILING,
    batch_size=256,
    validation_data=val,
    optimizer=per_node_adam(flow, lr=1e-2),
    callbacks=plateau,
)
record("PerNodePlateau", flow, time.perf_counter() - t0)
```

```
print(f"    froze at: {dict(sorted(plateau.frozen.items()))}")
print("    per-node rates at the end:", flow.history"lr")
assert set(plateau.frozen) == set(SPEC), "not every node froze"
assert len(flow.history["train"]) < CEILING, "the plateau rule did not self-stop"
```

```
PerNodePlateau         val NLL 4.9985    227 epochs     8.6s
    froze at: {'x1': 227, 'x2': 85, 'x3': 58}
    per-node rates at the end: {'x1': 0.0, 'x2': 0.0, 'x3': 0.0}
```

`history["lr"]` is a dict per epoch when the groups are tagged, so the decay of each node is on record without a callback of your own. `plot_training` reads both that and the freeze epochs.

```
plot_training(flow, frozen=plateau.frozen)
plt.show()
```

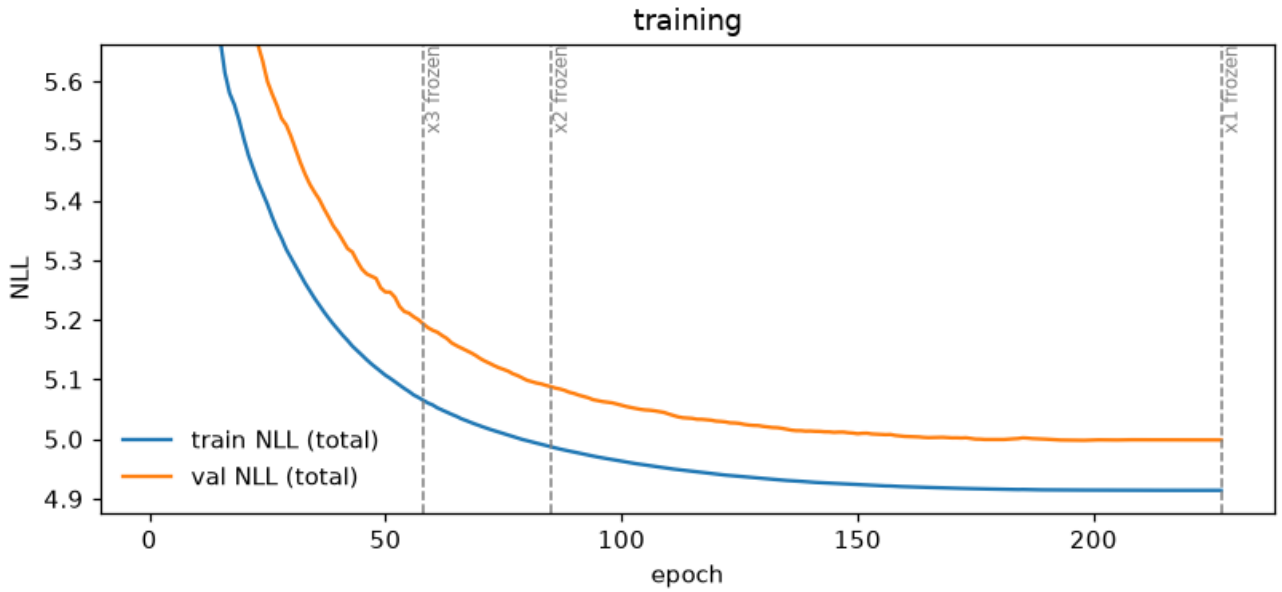


Figure 9: png

7. Writing your own

A bare callable in `callbacks=` is an `on_epoch_end` hook, called as `cb(flow, epoch, optimizer)`. Return `True` to stop the fit. That is enough to carry any torch scheduler, because `fit` has already put this epoch's validation score in `history["val"]`.

Subclass `Callback` instead when you need the other two hooks: `on_fit_begin` for state that must reset between fits, and `on_fit_end`, which runs before the varying-coefficient re-centering so restored weights still take part in it.

```
class GlobalPlateau(Callback):
    """One torch ReduceLROnPlateau over all nodes, driven by fit's own score."""

    def __init__(self, factor=0.3, patience=10):
        self.factor, self.patience = factor, patience
        self.scheduler = None

    def on_fit_begin(self, flow, optimizer):
        """Build the scheduler here: the optimizer exists only once fit runs."""
        self.scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
```

```
            optimizer, factor=self.factor, patience=self.patience
        )

    def on_epoch_end(self, flow, epoch, optimizer):
        """Step the scheduler, and stop once the rate has bottomed out."""
        self.scheduler.step(sum(flow.history"val".values()))
        return optimizer.param_groups0 < 1e-5


flow = build()
optimizer = torch.optim.Adam(flow.parameters(), lr=1e-2)
t0 = time.perf_counter()
flow.fit(
    train,
    epochs=CEILING,
    batch_size=256,
    validation_data=val,
    optimizer=optimizer,
    callbacks=GlobalPlateau(),
)
record("GlobalPlateau", flow, time.perf_counter() - t0)

final_lr = optimizer.param_groups0
print(f"    rate went 1.0e-02 -> {final_lr:.1e}")
assert final_lr < 1e-2, "the scheduler never decayed the rate"
```

```
GlobalPlateau          val NLL 4.9888    210 epochs     8.0s
    rate went 1.0e-02 -> 7.3e-06
```

A one-line callable is often enough. This one records a coefficient after every epoch, which is how the paper replications trace convergence:

```
trace = []
flow.fit(train, epochs=100, validation_data=val,
         callbacks=lambda f, epoch, opt: trace.append(f.nll(val)["x3"]))
```

A callable of the wrong shape is refused before the first epoch, so a mis-registered callback cannot waste a long run.

```
try:
    build().fit(train, epochs=5, batch_size=256, callbacks=lambda flow: None)
except TypeError as err:
    print("wrong arity refused up front:\n   ", err)
else:
    raise AssertionError("a one-argument callable should be refused")
```

```
wrong arity refused up front:
   a bare callable in callbacks= is called as cb(flow, epoch, optimizer); <function <lambda> at 0x7f63
for the other hooks subclass tramdag.callbacks.Callback
```

8. The scoreboard

One workload, one seed, six recipes. The numbers show mechanics on 900 rows, not a benchmark; `docs/training-speed.md` has the measured comparison.

```
board = pd.DataFrame(scoreboard).set_index("strategy")
print(board.to_string(float_format=lambda v: f"{v:.4f}"))

# Keeping the best weights cannot score worse than keeping the last ones.
```

```
assert nll_best <= nll_plain + 1e-6, (
    f"EarlyStopping scored {nll_best:.4f} against plain Adam's {nll_plain:.4f}"
)
```

```
                   val_nll  epochs  seconds
strategy
plain Adam          4.9958     300  11.4870
two phases          4.9781     300  11.3246
EarlyStopping       4.9564     300  11.4129
EarlyStopping(25)   4.9564     205   7.8907
PerNodePlateau      4.9985     227   8.6328
GlobalPlateau       4.9888     210   8.0218
```

```
fig, ax = plt.subplots(figsize=(7, 3.4))
ax.plot(np.arange(1, len(history_plain) + 1), history_plain, label="plain Adam")
ax.axhline(nll_best, ls="--", lw=1, color="C1", label="EarlyStopping, restored")
ax.set_xlabel("epoch")
ax.set_ylabel("validation NLL (total)")
ax.set_ylim(min(history_plain) - 0.02, min(history_plain) + 0.4)
ax.legend()
ax.set_title("keeping the best epoch against keeping the last")
fig.tight_layout()
plt.show()
```

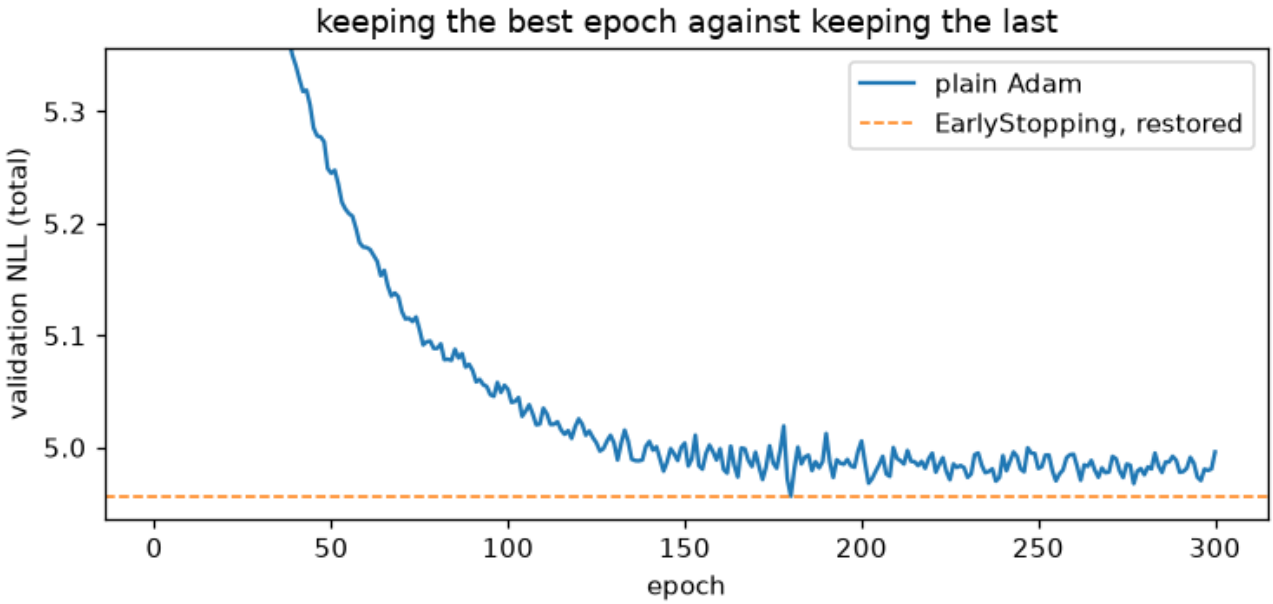


Figure 10: png

Which one, when

The decision table lives in `docs/fitting.md` and the measured time-to-target in `docs/training-speed.md`. Two rules are worth repeating here, because both are easy to get wrong:

- An all-LS model wants the final weights, not the best ones. That is what makes it match `statsmodels` and R exactly. Use `fit_classical` for it instead, which is deterministic and faster.
- A flexible model with confounded observational data can overfit at the maximum-likelihood point and lose the causal effect. There `EarlyStopping` is not a speed choice but a correctness

one.

Heterogeneous treatment effects: the `VC` term

Most causal questions are not "does the treatment help?" but "**whom** does it help, and by how much?". A TRAM-DAG answers that with a **varying-coefficient** term, which gives one edge of the DAG a treatment effect that depends on covariates:

$$\text{shift} \;+=\; \beta(\text{modifiers}) \cdot x_t, \qquad \beta(x) = \beta_0 + b_\Theta(x)$$

`beta0` is the constant part — an interpretable log-odds ratio, exactly as a linear shift would give — and `b_Theta` is a deliberately small, **penalized** network that bends it per individual.

This notebook builds a DGP whose effect function we know exactly, fits the model, and scores the recovered `beta(x)` against the truth. It then shows the two things the term exists for that a plain flexible shift does not give: a read-out you can plot, and a **propensity-centered** variant that survives confounding.

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

from tramdag import CS, LS, VC, CausalFlowDAG, ContinuousNode, I, OrdinalNode
from tramdag.callbacks import EarlyStopping

plt.rcParams["figure.dpi"] = 110
```

1. A DGP with a known effect function

Three covariates, a **confounded** treatment (`X1` and `X2` push assignment), and an outcome whose conditional is exactly a transformation model:

$$h(y) + g(x) + \beta(x)\, t = U, \qquad U \sim \text{Logistic}(0, 1)$$

with $h(y) = 2y$, a nonlinear prognostic part $g(x) = \frac{1}{2}X_1^2 + X_2 - \frac{1}{2}X_3$, and the effect function

$$\beta(x) = -1 + 0.8\, X_2 - 0.6\, X_3.$$

`X2` is deliberately *both* a confounder and an effect modifier — the case where a naive estimate fails hardest.

```
B0, B2, B3 = -1.0, 0.8, -0.6


def true_beta(df):
    """The effect function the model has to recover, on the latent scale."""
    return B0 + B2 * df["X2"].to_numpy() + B3 * df["X3"].to_numpy()


def simulate(n, seed):
    rng = np.random.default_rng(seed)
    x1, x2, x3 = (rng.normal(size=n) for _ in range(3))
    t = (rng.logistic(size=n) > -(0.4 * x1 + 0.4 * x2)).astype(float)
    g = 0.5 * x1**2 + x2 - 0.5 * x3
```

```
    beta = B0 + B2 * x2 + B3 * x3  # the same expression true_beta reports
    y = (rng.logistic(size=n) - g - beta * t) / 2.0
    return pd.DataFrame({"X1": x1, "X2": x2, "X3": x3, "T": t, "Y": y})


train, val, test = simulate(4500, 1), simulate(500, 2), simulate(2000, 3)
print(f"treated fraction: {train['T'].mean():.2f}")
print(
    f"true effect ranges from {true_beta(test).min():+.2f} to {true_beta(test).max():+.2f}"
)
```

```
treated fraction: 0.48
true effect ranges from -4.41 to +2.29
```

2. Which covariates modify the effect? (`effect_modifier_scan`)

Before committing to a set of modifiers, ask the data. Fit the **cheap all-ls model** — seconds, deterministic — and look at the per-observation scores of the treatment coefficient. If the effect really were constant, those scores would carry no structure in any covariate; a CUSUM statistic per covariate turns that into a ranking.

```
def cheap_all_ls():
    return {
        "X1": ContinuousNode(),
        "X2": ContinuousNode(),
        "X3": ContinuousNode(),
        "T": OrdinalNode(2, [LS("X1"), LS("X2")]),
        "Y": ContinuousNode([LS("X1"), LS("X2"), LS("X3"), LS("T")]),
    }


screen = CausalFlowDAG(cheap_all_ls(), seed=0)
screen.fit_classical(train)
print(screen.effect_modifier_scan(train, "Y", t="T"))
```

```
        stat       p_value  crit_5pct  flag
X2  4.163334  1.759830e-15     1.3581  True
X1  4.139814  2.600764e-15     1.3581  True
X3  3.162057  4.133820e-09     1.3581  True
```

`X2` and `X3` are the true modifiers and both flag — but so does `X1`, which does **not** enter $\beta(x)$ at all. That is not a bug, and it is worth understanding before you trust the shortlist: the statistic measures how unstable the *cheap model* is along a covariate, and instability has two sources — a coefficient that truly varies, and a **prognostic part the cheap model gets wrong**. Here g contains $\frac{1}{2}X_1^2$, which a linear term cannot represent.

The demonstration: re-generate with a *linear* prognostic X_1, change nothing else, and `X1` drops out of the shortlist.

```
def simulate_linear_x1(n, seed):
    rng = np.random.default_rng(seed)
    x1, x2, x3 = (rng.normal(size=n) for _ in range(3))
    t = (rng.logistic(size=n) > -(0.4 * x1 + 0.4 * x2)).astype(float)
    g = 0.5 * x1 + x2 - 0.5 * x3  # linear, so the cheap model fits it exactly
    y = (rng.logistic(size=n) - g - (B0 + B2 * x2 + B3 * x3) * t) / 2.0
    return pd.DataFrame({"X1": x1, "X2": x2, "X3": x3, "T": t, "Y": y})
```

```
well_specified = simulate_linear_x1(4500, 1)
screen2 = CausalFlowDAG(cheap_all_ls(), seed=0)
screen2.fit_classical(well_specified)
print(screen2.effect_modifier_scan(well_specified, "Y", t="T"))
```

```
        stat       p_value  crit_5pct   flag
X2  4.609058  7.066926e-19     1.3581   True
X3  3.476365  6.368441e-11     1.3581   True
X1  0.552602  9.201793e-01     1.3581  False
```

Read the scan as a **screening** step, then: it shortlists covariates worth giving to the effect head, and a flag can also mean "your prognostic part is wrong here". Both are things you want to know.

3. The spec: prognostic part and effect head are separate

`CS("X1", "X2", "X3")` absorbs the prognostic signal — as flexible as you like — while `VC("X2", "X3", t="T")` carries the effect. `X2` and `X3` appear twice on purpose: prognostically through the shift, and as modifiers through the head. Only the treatment named by `t=` owns its edge.

```
spec = {
    "X1": ContinuousNode(),
    "X2": ContinuousNode(),
    "X3": ContinuousNode(),
    "T": OrdinalNode(2, [LS("X1"), LS("X2")]),
    "Y": ContinuousNode([CS("X1", "X2", "X3"), VC("X2", "X3", t="T")]),
}
flow = CausalFlowDAG(spec, seed=0)
flow.fit(
    train,
    epochs=300,
    learning_rate=1e-2,
    batch_size=512,
    validation_data=val,
    seed=0,
    callbacks=EarlyStopping(),  # keep the best-validation weights
)
print(flow.to_matrix())
```

```
   X1 X2 X3   T                          Y
X1           LS      CS['X1', 'X2', 'X3']
X2           LS  CS['X1', 'X2', 'X3']+VCm
X3               CS['X1', 'X2', 'X3']+VCm
T                                     VC
Y
```

The adjacency view above is the paper's *meta-adjacency matrix*: every edge labelled with the term that carries it. `VC` marks the treatment edge and `VCm` the modifiers, which is how you can see at a glance that `X2` enters `Y` twice.

4. Reading the effect out

`varying_coef` gives $\beta(x)$ per row. It is **deterministic** and does not look at the outcome — it is a property of the fitted model, not of the rows' `Y` values.

```
beta_hat = flow.varying_coef(test, "Y")
beta_true = true_beta(test)
corr = float(np.corrcoef(beta_hat, beta_true)[0, 1])
beta0 = float(flow.nodes["Y"].shifts["T"].beta0)
```

```
print(f"corr(beta_hat, beta_true) = {corr:.3f}")
print(f"beta0 = {beta0:+.3f}   (true constant part {B0:+.1f})")
print(f"mean |error| = {np.abs(beta_hat - beta_true).mean():.3f}")

fig, axes = plt.subplots(1, 2, figsize=(10, 3.8))
axes[0].scatter(beta_true, beta_hat, s=6, alpha=0.35)
lims = [min(beta_true.min(), beta_hat.min()), max(beta_true.max(), beta_hat.max())]
axes[0].plot(lims, lims, "k--", lw=1)
axes[0].set_xlabel(r"true $\beta(x)$")
axes[0].set_ylabel(r"fitted $\hat\beta(x)$")
axes[0].set_title(f"recovery, corr = {corr:.3f}")

order = np.argsort(test["X2"].to_numpy())
axes[1].plot(test["X2"].to_numpy()[order], beta_true[order], "k-", lw=2, label="true")
axes[1].scatter(test["X2"], beta_hat, s=6, alpha=0.3, color="C0", label="fitted")
axes[1].set_xlabel("$X_2$")
axes[1].set_ylabel(r"$\beta$")
axes[1].set_title(r"$\beta$ against a modifier")
axes[1].legend()
fig.tight_layout()
plt.show()
```

```
corr(beta_hat, beta_true) = 0.996
beta0 = -0.960   (true constant part -1.0)
mean |error| = 0.098
```

```
/tmp/ipykernel_3989/2710869176.py:4: UserWarning: Converting a tensor with requires_grad=True to a s
Consider using tensor.detach() first. (Triggered internally at /__w/pytorch/pytorch/torch/csrc/auto
  beta0 = float(flow.nodes["Y"].shifts["T"].beta0)
```

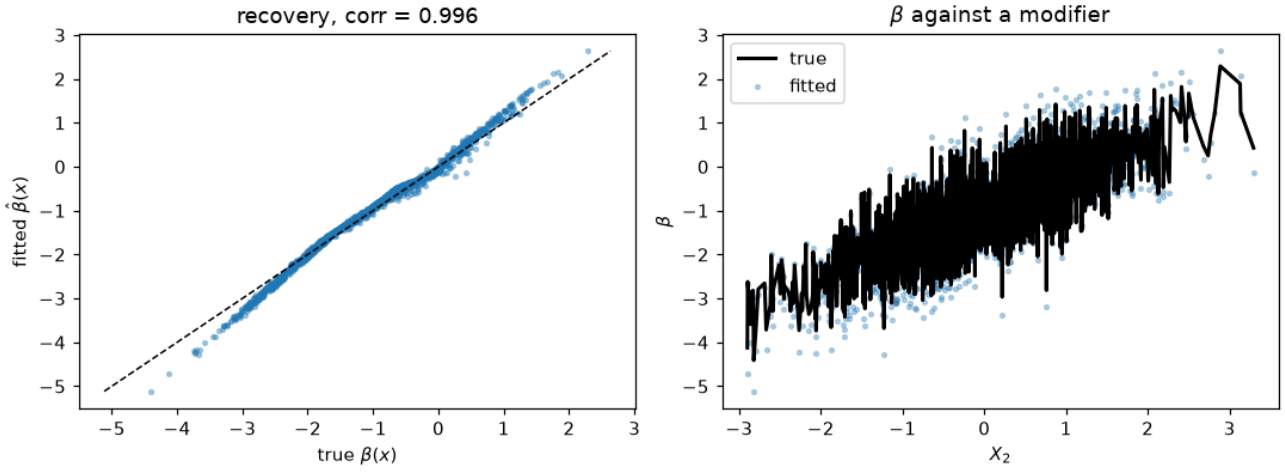


Figure 11: png

The scatter against `X2` alone is a band rather than a line, because the true effect also depends on `X3` — exactly as it should be.

5. The read-out is an identity, not a summary

For a binary treatment, $\beta(x)$ *equals* the difference of the abducted latents between the two arms, with the outcome held fixed. That is a definition the implementation must satisfy, and it does, to float precision:

```
u1 = flow.abduct(test.assign(T=1.0), seed=0)["Y"].to_numpy()
u0 = flow.abduct(test.assign(T=0.0), seed=0)["Y"].to_numpy()
print(f"max |beta(x) - (u(T=1) - u(T=0))| = {np.abs(beta_hat - (u1 - u0)).max():.2e}")
```

```
max |beta(x) - (u(T=1) - u(T=0))| = 4.77e-07
```

6. Confounding: why `center=` exists

Everything above had a prognostic part flexible enough to absorb g. Real models are misspecified, and when the misfit correlates with **treatment assignment**, the effect head absorbs the confounding instead of the effect.

The configuration where this bites hardest is deliberately simple: one covariate, a strong propensity $e(x) = \sigma(2x)$, a constant true effect $\tau = -1$, and a quadratic prognostic part fitted with a linear term. `center="ps"` replaces t by $t - \hat{e}(x)$ using **cross-fitted** (out-of-fold) propensities from the training-frame column `ps`, so the head sees the part of the treatment that the covariates do not explain. Stage 1 — the propensities — is yours: any classifier, predicted out of fold, merged into the frame as a column. Here, five classical fits of the treatment spec.

```
TAU = -1.0


def confounded(n, seed):
    rng = np.random.default_rng(seed)
    x = rng.normal(size=n)
    t = (rng.logistic(size=n) > -2.0 * x).astype(float)  # strong confounding
    y = (rng.logistic(size=n) - 1.2 * x**2 - TAU * t) / 2.0  # quadratic prognostic
    return pd.DataFrame({"X": x, "T": t, "Y": y})


c_train, c_val, c_test = confounded(5400, 0), confounded(600, 7), confounded(3000, 1000)

# stage 1 of the centered design, the caller's job: cross-fitted P(T=1|X), each
# fold predicted by a treatment model that never saw it (the DML requirement)
fold_id = np.random.default_rng(0).permutation(len(c_train)) % 5
e_oof = np.empty(len(c_train))
for j in range(5):
    t_spec = {
        "X": ContinuousNode([I(transform="affine")]),
        "T": OrdinalNode(2, [LS("X")]),
    }
    proxy = CausalFlowDAG(t_spec, seed=0)
    proxy.fit_classical(c_train.ilocfold_id != j])
    e_oof[fold_id == j] = proxy.pmf(c_train.iloc[fold_id == j], "T")[:, 1]

for center in (False, "ps"):
    spec_c = {
        "X": ContinuousNode([I(transform="affine")]),
        "T": OrdinalNode(2, [LS("X")]),
        # linear prognostic term, though the truth is quadratic
        "Y": ContinuousNode([LS("X"), VC("X", center=center, t="T")]),
    }
```

```
    fc = CausalFlowDAG(spec_c, seed=0)
    fc.fit(
        # the out-of-fold propensities ride the frame as the column center= names
        c_train.assign(ps=e_oof) if center else c_train,
        epochs=250,
        learning_rate=1e-2,
        batch_size=512,
        validation_data=c_val,
        seed=0,
        callbacks=EarlyStopping(),  # keep the best-validation weights
    )
    b = fc.varying_coef(c_test, "Y")
    print(
        f"center={bool(center)!s:5s}  mean |beta - tau| = {np.abs(b - TAU).mean():.3f}"
        f"   mean beta = {b.mean():+.3f}  (true tau {TAU:+.1f})"
    )
```

```
center=False  mean |beta - tau| = 1.284   mean beta = -0.028  (true tau -1.0)


center=True   mean |beta - tau| = 0.308   mean beta = -0.692  (true tau -1.0)
```

Without centering the confounding swallows the effect almost entirely — the average estimate lands near zero when the truth is −1. With centering it recovers most of it, and the mean absolute error falls by roughly a factor of four. Centering costs nothing when the prognostic part is adequate, which is why it is worth reaching for whenever assignment is not random.

What to take away

you want	use	read it with
one interpretable effect	`LS("T")`	`ls_coefficients()`
an effect that varies with covariates	`VC(*modifiers, t="T")`	`varying_coef(df, node)`
the same under confounding + a misspecified prognostic part	`VC(..., center="ps")`	the same read-out
a shortlist of modifiers before you commit	`effect_modifier_scan` on a cheap all-`ls` fit	its `flag` column, read as screening

The alternative of writing `CS("T", "X2", "X3")` is equally *expressive* — any shift decomposes into arms — but nothing in the likelihood rewards a smooth difference between them, so the implied effect is the difference of two unregularized networks. On this task class that reduced form measures corr ≈ 0.5 against the truth. The `VC` term exists to put the regularization where the question is. See `docs/varying-coefficients.md` for the semantics and `docs/scores.md` for the scan.